

Linear-Time Accumulation Schemes

Benedikt Bünz

`bb@nyu.edu`

NYU

Alessandro Chiesa

`alessandro.chiesa@epfl.ch`

EPFL

Giacomo Fenzi

`giacomo.fenzi@epfl.ch`

EPFL

William Wang

`ww@priv.pub`

NYU

May 28, 2025

Abstract

Proof-carrying data (PCD) is a powerful cryptographic primitive for computational integrity in a distributed setting. State-of-the-art constructions of PCD are based on accumulation schemes (and, closely related, folding schemes).

We present **WARP**, the first accumulation scheme with linear prover time and logarithmic verifier time. Our scheme is hash-based (secure in the random oracle model), plausibly post-quantum secure, and supports unbounded accumulation depth.

We achieve our result by constructing an interactive oracle reduction of proximity that works with *any* linear code over a sufficiently large field. We take a novel approach by constructing a straightline extractor that relies on *erasure correction*, rather than error-tolerant decoding like prior extractors. Along the way, we introduce a variant of straightline round-by-round knowledge soundness that is compatible with our extraction strategy.

Keywords: interactive oracle reductions; accumulation schemes; linear codes

Contents

1	Introduction	3
1.1	Our results	4
1.2	Related work	6
2	Technical overview	8
2.1	On hash-based accumulation	8
2.2	Limitations of prior work	8
2.3	Out-of-domain (and in-domain) sampling for every linear code	9
2.4	Codeword batching, with constraints	11
2.5	From PESAT to constrained codes	13
2.6	Twin constraint pseudo-batching	14
2.7	Straightline extraction for every linear code	16
2.8	Round-by-round extraction for every linear code	19
2.9	Summary and instantiations	21
3	Preliminaries	25
3.1	Relations	25
3.2	Interactive oracle reductions	26
3.3	Zero-evaders	27
3.4	Codes	28
3.5	Out-of-domain samples	29
3.6	Mutual correlated agreement for proximity generators	29
4	Round-by-round knowledge soundness for IORs	30
5	Polynomial equation satisfiability	31
5.1	Protocol	33
5.2	Security	33
6	Twin constraint pseudo-batching	35
6.1	Protocol	35
6.2	Security	38
7	Codeword batching	43
7.1	Protocol	43
7.2	Security	44
8	Multilinear constraint batching	46
8.1	Protocol	46
8.2	Security	48
9	Combining the IORs	51
10	Unbounded accumulation from any linear code	55
10.1	Accumulation schemes in oracle models	55
10.2	WARP: our accumulation scheme	55
A	Succinct arguments vs. accumulation schemes	64
B	RBR knowledge soundness implies SR knowledge soundness	65
C	Comparison with prior notions of RBR knowledge soundness	68
D	Extension fields	70
D.1	Definitions	70
D.2	Adapting the IORs	71
D.3	Adapting the accumulation scheme	71
	Acknowledgments	72
	References	72

1 Introduction

Proof-carrying data (PCD) [CT10] is a powerful cryptographic primitive that allows mutually distrustful parties to perform distributed computation in an efficiently verifiable manner. PCD generalizes incrementally verifiable computation (IVC) [Val08], and has applications to enforcing language semantics [CTV13], verifiable MapReduce computations [CTV15], image/video provenance [NT16; ZYORJ24], memory-efficient succinct arguments [NDCTB24], consensus protocols [BCG24], blockchains [Min; KB20; BMRS20; CCDW20], and more.

PCD was initially achieved via recursive proof composition of succinct arguments [BCCT13; BCTV14]. Subsequently, a successful line of work shows how to achieve highly-efficient PCD from a more lightweight primitive, *accumulation schemes* (and folding schemes) [BGH19; BCMS20; BCLMS21; KST22; BC23; KS24; EG23; BMNW25a; BC24; FKNP24; BMNW25b; BC25; NS25].

In this paper we present the first accumulation scheme with linear prover time and logarithmic verifier complexity. We review accumulation schemes below, and refer the reader to the cited papers for how PCD is obtained from them.

Accumulation schemes. Informally, an accumulation scheme $\text{ACC} = (\mathbf{P}_{\text{ACC}}, \mathbf{V}_{\text{ACC}}, \mathbf{D}_{\text{ACC}})$ for an NP relation $\mathcal{R}$ enables verifiably propagating the bit that equals the conjunction of any number of membership checks for $\mathcal{R}$, without holding onto the verified instance-witness pairs. This is done by maintaining an accumulator acc . For each instance-witness pair to be accumulated, the prover $\mathbf{P}_{\text{ACC}}$ updates acc and outputs a short proof pf attesting to the update’s correctness, which the verifier $\mathbf{V}_{\text{ACC}}$ checks. Afterwards, the accumulator acc is given to the decider $\mathbf{D}_{\text{ACC}}$, which outputs a bit. Provided all checks by $\mathbf{V}_{\text{ACC}}$ succeeded, $\mathbf{D}_{\text{ACC}}$ ’s output equals the conjunction of the membership checks on the accumulated instance-witness pairs.

In fact, we consider a refined notion known as a *split* accumulation scheme [BCLMS21], in which the accumulator acc is divided into a short “instance part” $\text{acc}.\mathbb{x}$ and long “witness part” $\text{acc}.\mathbb{w}$. The verifier $\mathbf{V}_{\text{ACC}}$ receives only instance parts, whereas the prover $\mathbf{P}_{\text{ACC}}$ and decider $\mathbf{D}_{\text{ACC}}$ receive both instance and witness parts. This refined notion enables particularly efficient accumulation, since the verifier $\mathbf{V}_{\text{ACC}}$ can be succinct in the witness parts.

More formally, an accumulation scheme $\text{ACC} = (\mathbf{P}_{\text{ACC}}, \mathbf{V}_{\text{ACC}}, \mathbf{D}_{\text{ACC}})$ for $\mathcal{R}$ works as follows.

- *Completeness.* Let $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ and acc_i be such that $\mathbf{D}_{\text{ACC}}(\text{acc}_i) = 1$. Then $\mathbf{P}_{\text{ACC}}(\text{acc}_i, \mathbb{x}, \mathbb{w})$ outputs $(\text{acc}_{i+1}, \text{pf})$ such that

$$\mathbf{V}_{\text{ACC}}(\text{acc}_i.\mathbb{x}, \mathbb{x}, \text{acc}_{i+1}.\mathbb{x}, \text{pf}) = 1 \quad \text{and} \quad \mathbf{D}_{\text{ACC}}(\text{acc}_{i+1}) = 1.$$

- *Knowledge soundness.* There exists an efficient deterministic extractor $\mathbf{E}_{\text{ACC}}$ such that, for every $\text{acc}_i.\mathbb{x}, \mathbb{x}, \text{acc}_{i+1}, \text{pf}$ that are efficiently generated, if

$$\mathbf{V}_{\text{ACC}}(\text{acc}_i.\mathbb{x}, \mathbb{x}, \text{acc}_{i+1}.\mathbb{x}, \text{pf}) = 1 \quad \text{and} \quad \mathbf{D}_{\text{ACC}}(\text{acc}_{i+1}) = 1,$$

then, except for a small error probability, the extractor $\mathbf{E}_{\text{ACC}}$ (given suitable inputs) outputs witnesses $(\mathbb{w}, \text{acc}_i.\mathbb{w})$ such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ and $\mathbf{D}_{\text{ACC}}(\text{acc}_i) = 1$ (for $\text{acc}_i := (\text{acc}_i.\mathbb{x}, \text{acc}_i.\mathbb{w})$).

More generally, an accumulation scheme supports any number ℓ_1 of instance-witness pairs and any number ℓ_2 of input accumulators; in the above informal description we focus on $\ell_1 = \ell_2 = 1$.

Prior art: superlinear provers. In PCD schemes obtained from accumulation schemes, the time complexity of the PCD prover depends linearly on the time complexities of the accumulation prover;

moreover, each PCD step *recursively* proves that the accumulation verifier accepts. While prior work has constructed accumulation schemes with asymptotically optimal accumulation verifiers, the same is not true for the prover side. Specifically, in prior constructions of accumulation schemes, *provers run in superlinear time*. Below we summarize the efficiency features of prior works, and postpone a more detailed discussion to Section 1.2.

Prior accumulation schemes. Most accumulation schemes rely on additive homomorphisms [BCMS20; BCLMS21; KST22; BC23; KS24; EG23] that arise from commitments over prime-order groups or bilinear groups [Ped92; KZG10]. These constructions require multiscalar multiplications that involve a superlinear amount of group operations (even with Pippenger’s algorithm [Pip80]).

Some accumulation schemes are based on lattices [BC24; FKNP24; BC25; NS25]. While lattice-based linear-time commitments are known [AHKV17], challenges arise when handling noise growth towards accumulation. In particular, no prior lattice constructions achieve linear-time provers.

Recent work constructs accumulation schemes in the random oracle model (ROM) by using, in particular, Merkle commitment schemes [BMNW25b]. Their prover runs in quasilinear time, due to the use of Reed–Solomon codes and polynomial quotients.

Separately, we note that any succinct non-interactive argument directly implies a corresponding accumulation scheme (where the accumulation decider always accepts). Qualitatively, this implication yields accumulation schemes that are less efficient than direct constructions of accumulation schemes, since succinct arguments rely on “heavier” tools (to achieve succinctness). Quantitatively, the efficiency tradeoffs are more nuanced, as explained in Appendix A.

1.1 Our results

We present WARP, an accumulation scheme in the ROM where the accumulation prover runs in linear time (performs a linear number of field operations and random oracle queries) and the accumulation verifier runs in logarithmic time (performs a logarithmic number of field operations and random oracle queries).

We state our main theorem for polynomial equation satisfaction (PESAT), an NP-complete problem that naturally captures R1CS [BCGTV13], GR1CS [DMS24], CCS [STW23], and more.

Definition 1 (informal). A **polynomial equation system** over a field $\mathbb{F}$ is a tuple $(\hat{\mathbf{p}}, M, N, k)$ specifying a list of N -variate polynomials $\hat{\mathbf{p}} = (\hat{p}_i)_{i \in [M]}$ over $\mathbb{F}$ of constant (total) degree. An instance-witness pair $\mathbf{z} := (\mathbf{x} \in \mathbb{F}^{N-k}, \mathbf{w} \in \mathbb{F}^k)$ is a **solution** if $\hat{p}_i(\mathbf{z}) = 0$ for every $i \in [M]$. We write $|\hat{\mathbf{p}}|$ for the number of field operations required for evaluating the equation system. (Without loss of generality, $k = O(|\hat{\mathbf{p}}|)$.) The indexed **PESAT relation** $\mathcal{R}_{\odot}(\mathbb{F})$ is the set of triples $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w})$ where $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$, $\mathfrak{x} = \mathbf{x}$, and $\mathfrak{w} = \mathbf{w}$ such that $(\mathbf{x}, \mathbf{w})$ is a solution to $\mathfrak{i}$.

Theorem 1 (informal). In the random oracle model (ROM) with security parameter λ_{ro} , there is an accumulation scheme $\text{ACC} = (\mathbf{P}_{\text{ACC}}, \mathbf{V}_{\text{ACC}}, \mathbf{D}_{\text{ACC}})$ for the PESAT relation $\mathcal{R}_{\odot}(\mathbb{F})$ with the following properties for accumulating ℓ instance-witness pairs and accumulators.

- $\mathbf{P}_{\text{ACC}}$ performs $O(\ell \cdot |\hat{\mathbf{p}}| + \lambda \cdot \log k)$ field operations and $O(k)$ random oracle queries of size $\ell \cdot \log |\mathbb{F}|$ and λ_{ro} .
 - $\mathbf{V}_{\text{ACC}}$ performs $O(\ell \cdot (\log N + \log M + \lambda) + \lambda \cdot \log k)$ field operations, $O(\ell \cdot \lambda \cdot \log k)$ random oracle queries of size λ_{ro} and $O(\ell \cdot \lambda)$ random oracle queries of size $\log |\mathbb{F}|$.
 - $\mathbf{D}_{\text{ACC}}$ performs $O(|\hat{\mathbf{p}}|)$ field operations and $O(k)$ random oracle queries of size $\log |\mathbb{F}|$ and λ_{ro} .
- The protocol has knowledge soundness error $\leq 2^{-\lambda}$, for sufficiently large $|\mathbb{F}|$ and λ_{ro} .

The accumulation prover runs in linear time because its time complexity is proportional to the time complexity to evaluate the polynomial equation system. Moreover, compared to [BMNW25b], our accumulation prover runs in linear time (instead of quasilinear time), and our accumulation verifier has smaller arithmetic complexity (while retaining the same random oracle query complexity). See Table 1 for a comparison with known hash-based accumulation schemes.

IOR over (almost) every code. To achieve Theorem 1 we follow the recipe of [BMNW25b], which compiles (suitable) *interactive oracle reductions* (IORs) into a corresponding accumulation scheme. Recall that an IOR is an interactive protocol between a prover and a verifier, in which the prover sends possibly long proof strings that the verifier does not read in their entirety, but rather queries as an oracle; after the interaction, the verifier queries the proof strings, and either rejects or outputs a new instance of a prescribed relation. In an IOR, instances themselves might contain oracle messages, which the verifier can also query. For our purposes, the efficiency metrics that we aim to minimize are prover time, verifier time, and query complexity.

Our main technical contribution is two IORs that work over any given linear code, inheriting the efficiency features of the code (up to constants). Theorem 1 follows by fixing a linear code with linear encoding time and constant relative distance (e.g., the expander code in [GLSTW23] or the repeat-accumulate-accumulate code in [BCFRRZ25]), and applying the aforementioned recipe.

Theorem 2 (informal). *Fix a linear code $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ with encoding time $\text{enc}_{\mathcal{C}}$ and a repetition parameter $t \in \mathbb{N}$. We define an accumulation relation $\mathcal{R}_{\mathcal{C}}$ (in Definition 5.7) and construct an $\text{IOR}_{\odot} = (\mathbf{P}_{\odot}, \mathbf{V}_{\odot})$ from $\mathcal{R}_{\odot}(\mathbb{F})$ to $\mathcal{R}_{\mathcal{C}}$, where:*

- $\mathbf{P}_{\odot}$ performs $O(\text{enc}_{\mathcal{C}})$ field operations and sends a single oracle of length $O(n)$.
- $\mathbf{V}_{\odot}$ performs no field operations and no queries.

Further, we construct an $\text{IOR}_{\mathcal{C}} = (\mathbf{P}_{\mathcal{C}}, \mathbf{V}_{\mathcal{C}})$ from $\mathcal{R}_{\mathcal{C}}^{\ell}$ to $\mathcal{R}_{\mathcal{C}}$, where:

- $\mathbf{P}_{\mathcal{C}}$ performs $O(\ell \cdot (n + |\hat{\mathbf{p}}|) + t \cdot \log n)$ field operations and sends a single oracle of length $O(n)$.
- $\mathbf{V}_{\mathcal{C}}$ performs $O(\ell \cdot (\log n + \log M + \log N + t) + \lambda \cdot \log n)$ field operations and $\ell \cdot t$ queries.

Both protocols have round-by-round knowledge soundness error $\leq 2^{-\lambda}$ provided $t \geq \frac{\lambda}{-\log(1-\delta_{\text{PG}})}$ and $\mathbb{F}$ is sufficiently large, where δ_{PG} is $\mathcal{C}$'s proximity radius for mutual correlated agreement (see below).

The field size requirement in Theorem 1 is directly inherited from Theorem 2. We defer the precise requirements to Theorem 9.1. Here it suffices to say that the field size requirements are similar to those incurred in state-of-the-art IOPs/IORs of proximity. Roughly, to ensure round-by-round knowledge soundness error $\leq 2^{-\lambda}$, the field size must be $|\mathbb{F}| = 2^{\lambda} \cdot \text{poly}(\ell, \log M, \log n, |\Lambda(\mathcal{C}, \delta_{\text{PG}})|)$, where $|\Lambda(\mathcal{C}, \delta_{\text{PG}})|$ denotes the list size of $\mathcal{C}$ at distance δ_{PG} .

Almost any code. Theorem 2 requires *no structure* from the underlying linear code $\mathcal{C}$, apart from being defined over a sufficiently large field; our IORs do not rely on interleaving or tensoring of $\mathcal{C}$, contrary to prior linear-time IOPs. The property of *mutual correlated agreement* [ACFY25] mentioned in Theorem 2 is known for every linear code $\mathcal{C}$ [AHIV17; ACFY25; GKL24; Zei24]; informally, this property states that, except for a small error, if the random linear combination of ℓ functions has a large agreement set with the code $\mathcal{C}$ then this set coincides with the agreement set of these ℓ functions with the interleaved code $\mathcal{C}^{\ell}$ (see Section 3.6 for a formal definition).

Out-of-domain samples for every linear code. Out-of-domain samples in the Reed–Solomon proximity testing literature [BGKS20; ACFY24; ACFY25] are widely used to achieve efficient

proximity testing in the list decoding regime. [BGKS20] suggested an abstract definition of out-of-domain samples for arbitrary linear codes, but did not use it beyond Reed–Solomon codes. Our results crucially rely on out-of-domain samples obtained via a carefully chosen specialization of [BGKS20]’s definition, leading to a *first application of out-of-domain sampling for arbitrary codes*.

Straightline extraction without efficient decoding. In the probabilistic proof literature, achieving straightline knowledge soundness is relatively straightforward when the underlying code has an efficient (error-tolerant) decoding algorithm (e.g., this is true for Reed–Solomon codes). Roughly, the extractor runs the decoding algorithm on a function that, by the security analysis, is sufficiently close to an encoding of a valid witness. When the code is not known to have an efficient decoding algorithm (this is the case for certain linear-time encodable codes), showing knowledge soundness is significantly more involved, and usually involves rewinding extraction.

We introduce a weakening of round-by-round knowledge soundness, with straightline extraction, that we show our IORs satisfy *regardless of whether the given code has an efficient decoding algorithm*. Moreover, we show that our notion implies (straightline) state-restoration knowledge soundness, thus sufficing for compilation into a (non-interactive) protocol in the ROM according to the recipe in [BMNW25b]. Our strategy is not specific to IORs, and we believe that the definitions and tools in this paper can be applied to obtain straightline extraction for existing IOPs that use linear-time encodable codes (e.g., [BCGGHJ17; BCG20; BCL22; GLSTW23; ZCF24; BCFRRZ25]).

Small fields. The field size requirements in Theorem 1 and Theorem 2 (which imply that $|\mathbb{F}| \geq 2^\lambda$) can be relaxed, enabling the use of smaller fields. Given a field $\mathbb{B}$ and linear code $\mathcal{C}: \mathbb{B}^k \rightarrow \mathbb{B}^n$, WARP can be adapted to accumulate instances of $\mathcal{R}_\otimes(\mathbb{B})$. We describe this adaption in Appendix D, which involves sampling challenges from a sufficiently large field extension $\mathbb{F}$ of the field $\mathbb{B}$ (the requirements on $|\mathbb{F}|$ are as before). Note that WARP is *not* necessarily linear time with this adaption, since the accumulation prover would perform a linear number of field operations in the *extension field* $\mathbb{F}$. In the (common) setting where $|\mathbb{B}| = O(1)$ and $|\mathbb{F}| \geq 2^\lambda$, a field operation in $\mathbb{F}$ induces $O(\lambda)$ field operations in $\mathbb{B}$, and thus the prover cost is superlinear. Nevertheless, using extension fields may be concretely efficient and well-suited for many relations of interest. Achieving an accumulation scheme that, for every field $\mathbb{B}$, performs a linear number of $\mathbb{B}$ -operations remains an open question.

1.2 Related work

The most relevant prior work is hash-based accumulation [BMNW25a; BMNW25b], which we discuss in Sections 2.1 and 2.2. Below we summarize other prior work on accumulation schemes.

Group-based accumulation. Group-based accumulation schemes (e.g., [BCMS20; BCLMS21; KST22; KS24]) achieve asymptotically optimal verifier complexity ($O(1)$ group operations). These constructions rely on a Pedersen commitment [Ped92] or a KZG commitment [KZG10] of the witness, which involves a large multi-scalar-multiplication. This takes $O(\frac{\lambda \cdot n}{\log(\lambda \cdot n)})$ group operations via Pippenger’s algorithm [Pip80], making them unsuitable for linear-time accumulation.

Lattice-based accumulation. Lattice-based accumulation schemes can use [Ajt96] commitments, which are partially homomorphic. Nevertheless, the challenge of lattice-based accumulation lies in controlling the norm growth of the committed (and extracted) witness (thereby enabling unbounded accumulation). A recent exciting line of work [BC24; FKNP24; BC25; NS25] constructs accumulation schemes overcoming this challenge, but, to our knowledge, no construction achieves a linear-time prover (either because of superlinear commitment costs or superlinearity in the accumulation prover). Achieving a lattice-based accumulation scheme that controls norm growth while

maintaining a linear-time prover remains an open problem.

Remark 1.1 (concurrent work). [BMMS25] gives a linear-time accumulation scheme (in the ROM) that matches the verifier complexity of WARP. A few differences limit its concrete efficiency.

- The accumulation relation in [BMMS25] is less expressive than ours, which incurs concrete overheads. Specifically, their analogue of Construction 5.10 must invoke the sumcheck protocol. In practice, this requires the prover and verifier to perform more field operations.
- In their analogue of Construction 9.4, the prover sends multiple oracles (at least three, compared to one in our case). This requires the prover and verifier to make more random oracle queries.
- [BMMS25] only supports proximity bounds within the unique decoding regime. In practice, this requires the verifier to make more random oracle queries.

Finally, [BMMS25] does not achieve straightline knowledge soundness when the underlying code is not known to have an efficient error-tolerant decoder. We believe that our techniques can be applied to their protocols to obtain straightline extraction.

2 Technical overview

We discuss the main ideas behind our results.

2.1 On hash-based accumulation

We review the general framework that underlies hash-based accumulation schemes.

SNARGs in the ROM. Succinct non-interactive arguments in the ROM are obtained via the BCS transformation [BCS16]. The starting point is an *interactive oracle proof* (IOP) [BCS16; RRR16], which is an interactive protocol between a prover and a verifier in which the prover sends possibly long messages that the verifier can query as oracles. The BCS transformation relies on: (i) Merkle commitment schemes (in the ROM) to enable the argument prover to succinctly commit and locally decommit to the prover’s long messages; and (ii) the Fiat–Shamir transformation (in the ROM) to enable the argument prover to derive the IOP verifier’s (random) messages. In practice, by heuristically instantiating the random oracle, this leads to so-called *hash-based SNARGs*.

Accumulation in the ROM. Accumulation schemes in the ROM are obtained in a way that mirrors the above strategy [BMNW25a; BMNW25b]; in practice, this leads to *hash-based accumulation schemes*. Such constructions share many benefits with hash-based SNARGs: a transparent setup (the choice of hash function to instantiate the random oracle), fast proving (due to flexibility in field choice and size), plausible post-quantum security,¹ and more.

The starting point is relaxed from an IOP to an *interactive oracle reduction* (IOR) [BCGGRS19; BMNW25b]. Informally, an IOR is an interactive oracle protocol in which the verifier, rather than outputting a decision bit, either rejects or outputs an instance for the target relation; instances may contain oracle messages, which the verifier can also query. See Section 3.2 for a definition.

The BCS transformation can then be adapted to map an IOR into a corresponding *non-interactive reduction* in the ROM [BMNW25a; BMNW25b]. As discussed in Section 1, we seek an accumulation scheme $\text{ACC} = (\mathbf{P}_{\text{ACC}}, \mathbf{V}_{\text{ACC}}, \mathbf{D}_{\text{ACC}})$ for $\mathcal{R}$ in the ROM. This directly follows from two IORs that are the main target in this paper.

- An IOR that reduces checking membership in $\mathcal{R}$ to checking that a sent oracle (typically, an alleged encoding of a valid witness) is close to a certain code-based accumulation relation $\mathcal{R}_c$.
- An IOR that reduces ℓ proximity claims to $\mathcal{R}_c$ to a single proximity claim to $\mathcal{R}_c$.

These two IORs are akin to the two main components of many IOPs: an “IOP preamble” that reduces deciding the relation to a proximity claim, and an IOP of proximity for that claim.

Linear-time proving. A line of work [BCGGHJ17; BCG20; BCL22; GLSTW23] constructs IOPs with a linear-time prover. These directly lead to linear-time succinct arguments in the ROM (or in the plain model from linear-time collision resistant hash-function [AHIKV17]). From a technical perspective, we seek to replicate this success, by designing suitable IORs with a linear-time prover.

2.2 Limitations of prior work

We review the main features and limitations of prior IORs.

¹The BCS transformation is secure in the quantum random oracle model [CMS19]. We conjecture that a similar result holds for the strategy in [BMNW25a; BMNW25b], which adapts the BCS transformation to reductions.

IORs for every linear code, but with distance loss. [BMNW25a] constructs IORs for batching proximity claims to a general linear code (via random linear combinations and spot-checking). However the IORs *do not preserve distance*: the output proximity claim is always for a (strictly) smaller distance than for the input proximity claims. This limitation translates (after hash-based compilation) into achieving a *bounded-depth* accumulation scheme (a weaker notion of accumulation). Moreover, the IORs are proven secure only in the *unique-decoding* regime and the IOR prover sends two oracle messages (all other messages are short); this hinders performance.²

Distance-preserving IORs, but only for Reed–Solomon codes. [BMNW25b] replaces the core component of [BMNW25a] with a protocol inspired by [ACFY24], using quotients to enforce in-domain and out-of-domain constraints. The resulting IOR is *distance-preserving* and is proved secure in the *list-decoding* regime, improving the number of queries performed by the verifier. However, quotienting has drawbacks. First, the protocol is restricted to Reed–Solomon codes, which have quasilinear encoding time. Second, polynomial quotients require the prover to run an additional quasilinear-time FFT, and the verifier performs $O(\ell \cdot \lambda^2)$ field operations. Finally, the IOR prover sends two oracle messages (all other messages are short), which hinders performance.

This paper. We design IORs for proximity claims to a general linear code, enforcing in-domain and out-of-domain constraints in a new way that avoids quotienting altogether. We address the other shortcomings as well: our IORs are secure in the list-decoding regime, and the IOR prover sends a single oracle message (all other messages are short).

2.3 Out-of-domain (and in-domain) sampling for every linear code

We review *out-of-domain sampling* [BGKS20] and *in-domain sampling* [ACFY24; ACFY25], techniques that are useful towards constructing IOPPs for Reed–Solomon codes with good soundness in the list-decoding regime. Then we extend these notions to *any* linear code $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$.

Throughout, we fix a function $f: [n] \rightarrow \mathbb{F}$, and let $\Lambda(\mathcal{C}, f, \delta)$ be the set of codewords in $\mathcal{C}$ at relative distance $\leq \delta$ from f ; $\Lambda(\mathcal{C}, \delta)$ denotes the maximum of $\Lambda(\mathcal{C}, f, \delta)$ across all functions f .

Out-of-domain (and in-domain) sampling for RS codes. Consider the special case of a Reed–Solomon code: $\mathcal{C}$ consists of all evaluations on $[n] \subseteq \mathbb{F}$ of univariate polynomials over $\mathbb{F}$ of degree less than d (interpret $[n]$ as a domain L in $\mathbb{F}$).

Out-of-domain sampling is a technique that forces the prover to “choose” a single codeword in $\Lambda(\mathcal{C}, f, \delta)$ (if $\delta \geq \delta(\mathcal{C})/2$ then it can be that $|\Lambda(\mathcal{C}, f, \delta)| > 1$). The verifier sends a random element $\tau \in \mathbb{F}$ (the out-of-domain sample) and the prover answers with an element $\sigma \in \mathbb{F}$; in the honest case $f \in \mathcal{C}$ and $\sigma := \hat{f}(\tau)$ where $\hat{f}$ is the univariate polynomial whose evaluation on $[n]$ equals f . The verifier then considers the claim “there exists $u \in \Lambda(\mathcal{C}, f, \delta)$ such that $\hat{u}(\tau) = \sigma$ ”. This forces the prover to “choose” (at most) one codeword in $\Lambda(\mathcal{C}, f, \delta)$ because, with high probability over τ , no two codewords $u_1, u_2 \in \Lambda(\mathcal{C}, f, \delta)$ are such that $\hat{u}_1(\tau) = \hat{u}_2(\tau)$.

In-domain sampling refers to a claim of the form “ $f(\tau) = \sigma$ ” for a random element $\tau \in [n]$ (the in-domain sample) and a given response $\sigma \in \mathbb{F}$. Such claims are used to enforce consistency between different oracles sent by the prover (e.g., see domain shifting in [ACFY24; ACFY25]).

Moreover, prior protocols exploit the fact that, for Reed–Solomon codes, in-domain sampling can be viewed as a special case of out-of-domain sampling [ACFY24; ACFY25]. In the honest case where $f \in \mathcal{C}$, it holds that $f = \hat{f}([n])$, so $f(\tau) = \hat{f}(\tau)$ for every $\tau \in [n]$. Hence, a claim such as “ $f(\tau) = \sigma$ ” can be replaced by the claim “there is a codeword $u \in \Lambda(\mathcal{C}, f, \delta)$ such that

²To be precise, the prover can commit to both messages as a single interleaved oracle, but this still incurs overhead.

$\hat{u}(\tau) = \sigma$ ” (where $\hat{u}$ is the polynomial that evaluates to u). This enables batching out-of-domain and in-domain samples by using quotienting or the sumcheck protocol.

What happens in general? For a general linear code $\mathcal{C}$, an *in-domain claim* is a statement of the form “given $\tau \in [n]$ and $\sigma \in \mathbb{F}$, there exists $u \in \Lambda(\mathcal{C}, f, \delta)$ such that $u(\tau) = \sigma$ ”, namely, it is a claim on the value at a given position of a nearby codeword. Less clear is what an *out-of-domain claim* should be: while a polynomial can be evaluated everywhere on the underlying field, there is no polynomial underlying a codeword in an arbitrary linear code. Nevertheless, we seek a notion of out-of-domain sampling for general linear codes that satisfies several requirements: (i) it should be a constraint on elements of $\Lambda(\mathcal{C}, f, \delta)$ that forces the prover to “choose” a single codeword in $\Lambda(\mathcal{C}, f, \delta)$ (up to a small error); (ii) it should be compatible with in-domain samples (so both can be batched together); and, finally, (iii) it should be compatible with our goal of a linear-time prover.

Our approach: multilinear extensions. We impose a polynomial structure *on top* of the linear code $\mathcal{C}$: we rely on the *multilinear extension* of a codeword.³ For a codeword $u \in \mathcal{C}$, we write $\hat{u}$ to be the $(\log n)$ -variate multilinear polynomial that agrees with u on $\{0, 1\}^{\log n} \equiv [n]$. The following (straightforward) lemma tells us that the multilinear extensions of distinct codewords in the list $\Lambda(\mathcal{C}, f, \delta)$ agree at a random point with small probability.

Lemma 1 (informal). *Let $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ be a linear-code. For every $f: [n] \rightarrow \mathbb{F}$ and $\delta \in (0, 1)$,*

$$\Pr_{\tau \leftarrow \mathbb{F}^{\log n}} \left[\begin{array}{c} \exists \text{ distinct } u, v \in \Lambda(\mathcal{C}, f, \delta) \\ \hat{u}(\tau) = \hat{v}(\tau) \end{array} \right] \leq \frac{|\Lambda(\mathcal{C}, f, \delta)|^2 \cdot \log n}{|\mathbb{F}|} \leq \frac{|\Lambda(\mathcal{C}, \delta)|^2 \cdot \log n}{|\mathbb{F}|}.$$

Proof sketch. The multilinear extensions $\hat{u}, \hat{v}$ of distinct codewords $u, v \in \mathcal{C}$ are distinct. Hence, by the polynomial identity lemma, $\Pr_{\tau \leftarrow \mathbb{F}^{\log n}} [\hat{u}(\tau) = \hat{v}(\tau)] \leq \frac{\log n}{|\mathbb{F}|}$. The lemma follows by taking a union bound over the elements of $\binom{\Lambda(\mathcal{C}, f, \delta)}{2}$ (all distinct pairs of codewords in $\Lambda(\mathcal{C}, f, \delta)$). $\square$

We consider out-of-domain claims to be statements of the form “given $\tau \in \mathbb{F}^{\log n}$ and $\sigma \in \mathbb{F}$, there exists $u \in \Lambda(\mathcal{C}, f, \delta)$ such that $\hat{u}(\tau) = \sigma$ ”. This ensures several desiderata. First, $\hat{u}(\tau)$ is computable in $O(n)$ field operations (given u and τ), which is compatible with linear-time proving. Second, in-domain claims can be viewed as a special case of out-of-domain claims, as we now explain. For $\tau \in [n]$, $\hat{u}(\text{binary}(\tau)) = u(\tau)$, where $\text{binary}: [n] \rightarrow \{0, 1\}^{\log n}$ converts an integer to its binary representation embedded in $\mathbb{F}^{\log n}$. This enables reinterpreting the statement “given $\tau \in [n]$ and $\sigma \in \mathbb{F}$, there exists $u \in \Lambda(\mathcal{C}, f, \delta)$ such that $u(\tau) = \sigma$ ” as a statement “given $\tau \in [n]$ and $\sigma \in \mathbb{F}$, there exists $u \in \Lambda(\mathcal{C}, f, \delta)$ such that $\hat{u}(\text{binary}(\tau)) = \sigma$ ”. (Interestingly, these choices differ from those in prior work, even if the linear code $\mathcal{C}$ is a Reed–Solomon code.)

Towards batching and more. The above choices and considerations are the starting point of our work. We will show that, by working with multilinear extensions of codewords, we can design IORs that batch proximity claims for the (arbitrary) linear code $\mathcal{C}$. In the next few subsections we outline these ideas. Throughout, we use the notion of *constrained codes* [ACFY25].

Definition 2. *For $\tau \in \mathbb{F}^{\log n}$ and $\sigma \in \mathbb{F}$, the **constrained code** $\mathcal{C}[(\tau, \sigma)]$ is the set of codewords $u \in \mathcal{C}$ such that $\hat{u}(\tau) = \sigma$. $\mathcal{C}[(\tau_1, \sigma_1), \dots, (\tau_t, \sigma_t)]$ is similarly defined but with multiple constraints.*

³One can, e.g., also consider a univariate extension of a codeword. However, that would later lead to us to use tools such that as the univariate sumcheck protocol, which is not compatible with linear-time proving.

Remark 2.1 (linear constraints in action). [BGKS20] suggests a generic definition of out-of-domain sampling for an arbitrary linear code $\mathcal{C}$: they define linearly constrained codes where constraints on codewords $u \in \mathcal{C}$ have the form $\langle \mathbf{z}, \mathcal{C}^{-1}(u) \rangle = \mu$ for $\mathbf{z} \in \mathbb{F}^k$.

A special case is the out-of-domain sampling for Reed–Solomon codes introduced in [BGKS20]: if $\mathcal{C}$ is a Reed–Solomon code and $\mathbf{z} = (L_i(\tau))_{i \in [k]}$ contains the evaluations at $\tau \in \mathbb{F}$ of the Lagrange polynomials $(L_i(X))_{i \in [k]}$ that define $\mathcal{C}$, then $\langle \mathbf{z}, \mathcal{C}^{-1}(u) \rangle$ equals the univariate extension of u evaluated at τ . However, [BGKS20] demonstrated the use only of constraints $\langle (L_i(\tau))_{i \in [k]}, \mathcal{C}^{-1}(u) \rangle = \mu$ when $\mathcal{C}$ is the Reed–Solomon code, relying on the fact that quotients enable translating such constraints to “pure” proximity claims about the Reed–Solomon code. For general constraints $\langle \mathbf{z}, \mathcal{C}^{-1}(u) \rangle = \mu$, [BGKS20] proved a proximity gaps statement but provided no applications.

The multilinear extension constraints that we use are a different special case, and our results are a powerful application of this class of constraints. Specifically, if $\mathbf{z} = (\widehat{\mathbf{G}}_{\mathcal{C}}(\boldsymbol{\tau}, \mathbf{b}))_{\mathbf{b} \in \{0,1\}^{\log k}}$ where $\boldsymbol{\tau} \in \mathbb{F}^{\log n}$ and $\widehat{\mathbf{G}}_{\mathcal{C}}$ is the multilinear extension of a generator matrix $\mathbf{G}_{\mathcal{C}} \in \mathbb{F}^{n \times k}$ of $\mathcal{C}$, then $\langle \mathbf{z}, \mathcal{C}^{-1}(u) \rangle$ equals the multilinear extension of u evaluated at $\boldsymbol{\tau}$ (i.e., the constraints we consider). In our work we design protocols that directly reason about constrained codes for a general linear code $\mathcal{C}$; moreover, we depart from [BGKS20] in that we separately exploit out-of-domain sampling and proximity gaps statements (we separately rely on Lemma 3.14 and Definition 3.15).

2.4 Codeword batching, with constraints

We outline a (simplified) protocol for *codeword batching*, a key subroutine in our IOR. The prover and verifier have access to two (or more) oracles, along with claims about a linear combination of said oracles. The prover and verifier interact and, at the end of the interaction, the verifier outputs a list of claims about a *single* oracle. This is essential for constructing a many-to-one reduction whose new instance size is independent of the number of old instances.

Definition 3 (informal). *A codeword batching protocol with proximity radius δ^* is an interactive (oracle) protocol $(\mathbf{P}, \mathbf{V})$ with the following properties.*

- **Syntax:** *the verifier $\mathbf{V}$ gets as input randomness $\gamma_1, \gamma_2 \in \mathbb{F}$, multilinear evaluation claim $(\boldsymbol{\alpha}, \mu)$, and oracle access to $f_1, f_2: [n] \rightarrow \mathbb{F}$. The prover $\mathbf{P}$ gets as (explicit) input $(\gamma_1, \gamma_2, \boldsymbol{\alpha}, \mu, f_1, f_2)$. At the end of the interaction, $\mathbf{V}$ outputs a list of multilinear evaluation claims $(\boldsymbol{\tau}_i, \sigma_i)_i$ and (implicitly) an oracle $g: [n] \rightarrow \mathbb{F}$.*
- **Completeness:** *if $\gamma_1 \cdot f_1 + \gamma_2 \cdot f_2 \in \mathcal{C}[(\boldsymbol{\alpha}, \mu)]$ then $g \in \mathcal{C}[(\boldsymbol{\tau}_i, \sigma_i)_i]$.*
- **Soundness:** *for every $\delta \in (0, \delta^*)$, if $\Delta(\gamma_1 \cdot f_1 + \gamma_2 \cdot f_2, \mathcal{C}[(\boldsymbol{\alpha}, \mu)]) > \delta$ then, except for a small error probability, $\Delta(g, \mathcal{C}[(\boldsymbol{\tau}_i, \sigma_i)_i]) > \delta$.*

We describe a codeword batching protocol with proximity radius $\delta^* = 1$ (i.e., distance is preserved for every $\delta \in (0, 1)$) that has a linear-time prover and small soundness error.

Construction 1 (sketch). *Define $(\mathbf{P}, \mathbf{V})$ as follows.*

1. $\mathbf{P}$ sends $g: [n] \rightarrow \mathbb{F}$. In the honest case, $g := \gamma_1 \cdot f_1 + \gamma_2 \cdot f_2$.
2. $\mathbf{V}$ samples and sends $\boldsymbol{\tau} \leftarrow \mathbb{F}^{\log n}$.
3. $\mathbf{P}$ sends $\sigma \in \mathbb{F}$. In the honest case, $\sigma := \hat{g}(\boldsymbol{\tau})$.
4. $\mathbf{V}$ samples $x_1, \dots, x_t \leftarrow [n]$. For $i \in [t]$, define $\boldsymbol{\tau}_i := \text{binary}(x_i)$.
5. $\mathbf{V}$, for every $i \in [t]$, queries f_1 and f_2 at τ_i and computes $\sigma_i := \gamma_1 \cdot f_1(x_i) + \gamma_2 \cdot f_2(x_i)$.

6. $\mathbf{V}$ outputs the list of claims $((\boldsymbol{\alpha}, \mu), (\boldsymbol{\tau}, \sigma), (\boldsymbol{\tau}_i, \sigma_i)_{i \in [t]})$ and (implicitly) the oracle g .

Regardless of the linear code $\mathcal{C}$ used, the prover runs in Construction 1 in linear time: computing the function $g := \gamma_1 \cdot f_1 + \gamma_2 \cdot f_2$ and field element $\sigma := \hat{g}(\boldsymbol{\tau})$ can be done in $O(n)$ field operations. The soundness of Construction 1 is captured by the following lemma.

Lemma 2 (informal). *For every distance $\delta \in (0, 1)$, Construction 1 has soundness error*

$$\frac{|\Lambda(\mathcal{C}, \delta)|^2 \cdot \log n}{|\mathbb{F}|} + (1 - \delta)^t.$$

Proof sketch. Suppose that $\Delta(\gamma_1 \cdot f_1 + \gamma_2 \cdot f_2, \mathcal{C}[(\boldsymbol{\alpha}, \mu)]) > \delta$. Let $\mathcal{C}' := \mathcal{C}[(\boldsymbol{\alpha}, \mu), (\boldsymbol{\tau}, \sigma), (\boldsymbol{\tau}_i, \sigma_i)_{i \in [t]}]$. We argue that, except for the above error probability, $\Delta(g, \mathcal{C}') > \delta$ over the choices of $\boldsymbol{\tau}, \tau_1, \dots, \tau_t$.

By Lemma 1, except with probability at most $\frac{|\Lambda(\mathcal{C}, \delta)|^2 \cdot \log n}{|\mathbb{F}|}$, there is at most one codeword $u \in \Lambda(\mathcal{C}, g, \delta)$ such that $\hat{u}(\boldsymbol{\tau}) = \sigma$. Next, fix an arbitrary $u \in \Lambda(\mathcal{C}, g, \delta)$.

- If $\hat{u}(\boldsymbol{\tau}) \neq \sigma$, then $u \notin \mathcal{C}'$.
- If $\hat{u}(\boldsymbol{\tau}) = \sigma$ and $\hat{u}(\boldsymbol{\alpha}) \neq \mu$, then again $u \notin \mathcal{C}'$.
- If $\hat{u}(\boldsymbol{\tau}) = \sigma$ and $\hat{u}(\boldsymbol{\alpha}) = \mu$, it must be that $\Delta(\gamma_1 \cdot f_1 + \gamma_2 \cdot f_2, u) > \delta$ (as otherwise $\Delta(\gamma_1 \cdot f_1 + \gamma_2 \cdot f_2, \mathcal{C}[(\boldsymbol{\alpha}, \mu)]) \leq \delta$, contradicting our hypothesis). Hence, except with probability $(1 - \delta)^t$, there exists $i \in [t]$ such that $\hat{u}(\text{binary}(x_i)) \neq \sigma_i$ (i.e., $u(x_i) \neq \sigma_i$), in which case once again $u \notin \mathcal{C}'$.

The soundness error in the lemma statement follows from the fact that the last case (the only one with an error) needs to be considered only once, if out-of-domain sampling succeeded (the prover was forced to choose a single codeword in $\Lambda(\mathcal{C}, g, \delta)$). $\square$

Our complete protocol, presented in Construction 7.2, extends Construction 1 to handle an arbitrary number of input codewords $f_1, \dots, f_\ell$ and multiple out-of-domain samples $\boldsymbol{\tau}_1, \dots, \boldsymbol{\tau}_s$.

Batching $\Theta(\lambda)$ evaluation claims. Construction 1 receives a single multilinear evaluation claim $(\boldsymbol{\alpha}, \mu)$ as input, and outputs $r := t + 2$ multilinear evaluation claims $((\boldsymbol{\alpha}, \mu), (\boldsymbol{\tau}, \sigma), (\boldsymbol{\tau}_i, \sigma_i)_{i \in [t]})$. We cannot afford this increase in claims (as this would result in an accumulator whose size grows with the number of accumulations). Hence we seek to batch these r claims into a single one.

Standard techniques to batch r multilinear evaluation claims into a single one would result in a prover that runs in time $\Theta(r \cdot n)$ [RR22; CBBZ23]. In our setting, however, this is too expensive. The soundness analysis of the codeword batching protocol shows that we need $t \geq \frac{\lambda}{-\log(1-\delta)} = \Omega(\lambda)$ to achieve λ bits of security. This would imply a prover that, to batch $r = t + 2$ multilinear evaluation claims, runs in time $\Omega(r \cdot n) = \Omega(t \cdot n) = \Omega(\lambda \cdot n)$, which is superlinear.⁴

We overcome this problem by taking advantage of the fact that, in our setting, in-domain claims correspond to multilinear evaluation claims *inside the boolean hypercube*. In more detail, the t claims $(\boldsymbol{\tau}_i, \sigma_i)_{i \in [t]}$ in Construction 1 are special: $\boldsymbol{\tau}_i \in \{0, 1\}^{\log n}$ for every $i \in [t]$ (while the remaining 2 claims are for general locations in $\mathbb{F}^{\log n}$). We show that, by adapting ideas in [CBBZ23, Sec 3.8.1], we obtain a batching protocol for r multilinear evaluation claims, of which t are in the boolean hypercube, where the prover runs in time $O((r - t) \cdot n + t \cdot \log n)$. This is $O(n + t \cdot \log n)$ when $r = t + 2$, yielding the desired linear prover time. We describe and analyze this IOR in Section 8.

⁴This superlinear time is analogous to the one incurred by the quotienting procedure in [BMNW25b], where the prover is required to compute a $O(\lambda)$ -multi-point quotient of a witness polynomial, which requires an FFT. When batching via quotients, it is unknown how to avoid this additional computational overhead.

2.5 From PESAT to constrained codes

We reduce membership in the NP relation PESAT $\mathcal{R}_\circ(\mathbb{F})$ (see Definition 1) to proximity to a constrained code derived from a given arbitrary linear code $\mathcal{C}$. The ideas for this reduction are relatively standard (see [BMNW25b]). We sketch them here in order to motivate the definition of the constrained code relation $\mathcal{R}_\mathcal{C}$ that we accumulate via our IOR (our main technical contribution). See Section 5 for technical details of the IOR from $\mathcal{R}_\circ(\mathbb{F})$ to $\mathcal{R}_\mathcal{C}$.

Bundling PESAT. The PESAT relation $\mathcal{R}_\circ(\mathbb{F})$ consists of triples $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = ((\hat{\mathbf{p}}, M, N, k), \mathbf{x}, \mathbf{w})$ such that $\hat{p}_i((\mathbf{x}, \mathbf{w})) = 0$ for every $i \in [M]$. We compress the M polynomial constraints into a single one by defining the *bundling of* $(\hat{\mathbf{p}}, M, N, k)$:

$$\hat{P}(\mathbf{R}, \mathbf{Z}) := \sum_{\mathbf{i} \in \{0,1\}^{\log M}} \text{eq}(\mathbf{R}, \mathbf{i}) \cdot \hat{p}_{\mathbf{i}}(\mathbf{Z}).$$

The polynomial $\hat{P}: \mathbb{F}^{\log M} \times \mathbb{F}^N \rightarrow \mathbb{F}$ satisfies the following: (i) if $(\mathbf{x}, \mathbf{w})$ satisfies $(\hat{\mathbf{p}}, M, N, k)$ then $\hat{P}(\boldsymbol{\tau}, (\mathbf{x}, \mathbf{w})) = 0$ for every $\boldsymbol{\tau} \in \mathbb{F}^{\log M}$; (ii) if $(\mathbf{x}, \mathbf{w})$ does not satisfy $(\hat{\mathbf{p}}, M, N, k)$ then $\hat{P}(\boldsymbol{\tau}, (\mathbf{x}, \mathbf{w})) \neq 0$ with high probability over $\boldsymbol{\tau} \in \mathbb{F}^{\log M}$.

Twin constraints. We extend the constrained code in Definition 2 to additionally include (bundled) PESAT constraints.

Definition 4. For $(\boldsymbol{\alpha}, \mu) \in \mathbb{F}^{\log n} \times \mathbb{F}$ and $(\beta, \eta) \in \mathbb{F}^{\log M+N-k} \times \mathbb{F}$, the **twin constrained code** $\mathcal{C}[(\boldsymbol{\alpha}, \mu), (\hat{P}, \beta, \eta)]$ is the set of codewords $u \in \mathcal{C}$ such that $\hat{u}(\boldsymbol{\alpha}) = \mu$ and $\hat{P}(\beta, \mathcal{C}^{-1}(u)) = \eta$. $\mathcal{C}[(\boldsymbol{\alpha}_i, \mu_i)_{i \in [r]}, (\hat{P}, \beta, \eta)]$ is similarly defined but with multiple constraints.

The codeword batching protocol discussed in Section 2.4 can be straightforwardly extended to carry forward a (bundled) PESAT constraint, with only minor syntactical changes in the soundness analysis. The main relation that we accumulate, denoted by $\mathcal{R}_\mathcal{C}$ (and defined formally in Definition 5.7), consists of codewords satisfying the twin constraints.

From PESAT to twin constraints. The IOR from $\mathcal{R}_\circ(\mathbb{F})$ to $\mathcal{R}_\mathcal{C}$ is immediate.

Construction 2 (sketch). The prover and verifier receive an explicit instance $\mathfrak{x} = \mathbf{x} \in \mathbb{F}^{N-k}$, and the prover additionally receives a witness $\mathfrak{w} = \mathbf{w} \in \mathbb{F}^k$.

1. The prover sends $f: [n] \rightarrow \mathbb{F}$. In the honest case, $f := \mathcal{C}(\mathbf{w})$.
2. The verifier samples $\boldsymbol{\tau} \leftarrow \mathbb{F}^{\log M}$.
3. The verifier outputs the explicit instance $\mathfrak{x}' := ((\boldsymbol{\tau}, \mathbf{x}), 0)$ and implicit instance $\mathfrak{y}' := f$, which represents the claim “ $f \in \mathcal{C}[(\boldsymbol{\tau}, \mathbf{x}), 0]$ ” (namely, the claim “ $\hat{P}(\boldsymbol{\tau}, \mathbf{x}, \mathcal{C}^{-1}(f)) = 0$ ”).

The prover performs $O(\text{enc}_\mathcal{C})$ field operations, where $\text{enc}_\mathcal{C}$ is the number of field operations required to encode a message for the linear code $\mathcal{C}$; in particular, the prover performs $O(|\mathbf{w}|) = O(k)$ field operations (i.e., runs in linear time) if $\mathcal{C}$ is linear-time encodable.

The following lemma bounds the soundness error of Construction 2.

Lemma 3 (informal). Fix a PESAT index $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$. Suppose that the PESAT instance $\mathfrak{x} = \mathbf{x} \in \mathbb{F}^{N-k}$ is such that there exists no PESAT witness $\mathfrak{w} = \mathbf{w} \in \mathbb{F}^k$ for which $\hat{p}_i((\mathbf{x}, \mathbf{w})) = 0$ for every $i \in [M]$. Then, for every distance $\delta \in (0, 1)$, the verifier in Construction 2 outputs explicit instance $\mathfrak{x}' := ((\boldsymbol{\tau}, \mathbf{x}), 0)$ and implicit instance $\mathfrak{y}' := f$ such that $\Delta(f, \mathcal{C}[(\boldsymbol{\tau}, \mathbf{x}), 0]) > \delta$, except with probability at most

$$\frac{|\Lambda(\mathcal{C}, \delta)| \cdot \log M}{|\mathbb{F}|}.$$

2.6 Twin constraint pseudo-batching

We discuss *twin constraint pseudo-batching*, the problem of reducing multiple proximity claims for the twin constrained code (Definition 4) to a single proximity claim of a linear combination of the input functions. We call this *pseudo-batching* to emphasize the fact that the (twin) constraints are batched but not the codewords; this is compatible with the input of the codeword batching protocol discussed in Section 2.4 (after the modification to accommodate PESAT constraints).

Definition 5 (informal). *A twin constraint pseudo-batching protocol with proximity radius δ^* is an interactive (oracle) protocol $(\mathbf{P}, \mathbf{V})$ with the following properties.*

- **Syntax:** *The verifier $\mathbf{V}$ gets as input multilinear evaluation claims $(\alpha_0, \mu_0), (\alpha_1, \mu_1)$ and (bundled) PESAT constraints $(\beta_0, \eta_0), (\beta_1, \eta_1)$, and oracle access to $f_0, f_1: [n] \rightarrow \mathbb{F}$. The prover $\mathbf{P}$ gets as (explicit) input $(\alpha_0, \mu_0, \alpha_1, \mu_1, \beta_0, \eta_0, \beta_1, \eta_1, f_0, f_1)$. At the end of the interaction, $\mathbf{V}$ outputs randomness $\gamma_0, \gamma_1 \in \mathbb{F}$, a multilinear evaluation claim (α, μ) , a (bundled) PESAT constraint (β, η) , and (implicitly) oracles f_0, f_1 .*
- **Completeness:** *if $\forall b \in \{0, 1\} f_b \in \mathcal{C}[(\alpha_b, \mu_b), (\hat{P}, \beta_b, \eta_b)]$ then $\gamma_0 \cdot f_0 + \gamma_1 \cdot f_1 \in \mathcal{C}[(\alpha, \mu), (\hat{P}, \beta, \eta)]$.*
- **Soundness:** *for every $\delta \in (0, \delta^*)$, if $\exists b \in \{0, 1\} \Delta(f_b, \mathcal{C}[(\alpha_b, \mu_b), (\hat{P}, \beta_b, \eta_b)]) > \delta$ then, except for a small error probability, $\Delta(\gamma_0 \cdot f_0 + \gamma_1 \cdot f_1, \mathcal{C}[(\alpha, \mu), (\hat{P}, \beta, \eta)]) > \delta$.*

We describe a protocol for the above that has a linear-time prover and small soundness error. Our approach roughly follows an approach in [BMNW25b] that, in turn, is inspired by [EG23]. There are two main differences. First, in order to facilitate a linear-time prover, we use a multilinear (rather than univariate) sumcheck by adapting ideas from [EG23]. Second, we simultaneously batch multilinear evaluation claims and (bundled) PESAT constraints with a *single* oracle. In contrast, the quotient strategy in [BMNW25b] requires two oracles (intuitively this is because a quotiented codeword no longer satisfies PESAT constraints).

Construction 3 (sketch). *Define $(\mathbf{P}, \mathbf{V})$ as follows.*

1. $\mathbf{V}$ samples and sends $\tau \leftarrow \mathbb{F}$.
2. $\mathbf{P}$ and $\mathbf{V}$ define the linear (collections of) univariate polynomials

$$\begin{aligned} \hat{A}(\mathbf{X}) &:= (1 - \mathbf{X}) \cdot \alpha_0 + \mathbf{X} \cdot \alpha_1 && \in (\mathbb{F}^{<2}[\mathbf{X}])^{\log n}, \\ \hat{B}(\mathbf{X}) &:= (1 - \mathbf{X}) \cdot \beta_0 + \mathbf{X} \cdot \beta_1 && \in (\mathbb{F}^{<2}[\mathbf{X}])^{\log M + \kappa - k}, \\ \hat{F}(\mathbf{X}) &:= (1 - \mathbf{X}) \cdot f_0 + \mathbf{X} \cdot f_1 && \in (\mathbb{F}^{<2}[\mathbf{X}])^n, \\ \hat{U}(\mathbf{X}) &:= (1 - \mathbf{X}) \cdot \mathcal{C}^{-1}(f_0) + \mathbf{X} \cdot \mathcal{C}^{-1}(f_1) && \in (\mathbb{F}^{<2}[\mathbf{X}])^k. \end{aligned}$$

3. $\mathbf{P}$ sends univariate polynomials $\hat{h}_{\mathcal{C}} \in \mathbb{F}^{<\log n + 2}[\mathbf{X}]$, $\hat{h}_{\hat{P}} \in \mathbb{F}^{<\log M + d + 1}[\mathbf{X}]$. In the honest case:

$$\begin{aligned} \hat{h}_{\mathcal{C}}(\mathbf{X}) &:= \text{eq}(\mathbf{X}, \tau) \cdot \sum_{b \in \{0, 1\}^{\log n}} \text{eq}(\hat{A}(\mathbf{X}), b) \cdot \hat{F}_b(\mathbf{X}), \\ \hat{h}_{\hat{P}}(\mathbf{X}) &:= \text{eq}(\mathbf{X}, \tau) \cdot \hat{P}(\hat{B}(\mathbf{X}), \hat{U}(\mathbf{X})). \end{aligned}$$

4. $\mathbf{V}$ checks that $\hat{h}_{\mathcal{C}}(0) + \hat{h}_{\mathcal{C}}(1) = (1 - \tau) \cdot \mu_0 + \tau \cdot \mu_1$ and $\hat{h}_{\hat{P}}(0) + \hat{h}_{\hat{P}}(1) = (1 - \tau) \cdot \eta_0 + \tau \cdot \eta_1$.

5. $\mathbf{V}$ samples $\gamma \leftarrow \mathbb{F}$, and sets $\gamma_0 := 1 - \gamma$, $\gamma_1 := \gamma$, and

$$\begin{aligned}\alpha &:= \hat{\mathbf{A}}(\gamma), & \mu &:= \frac{\hat{h}_{\mathcal{C}}(\gamma)}{\text{eq}(\gamma, \tau)}, \\ \beta &:= \hat{\mathbf{B}}(\gamma), & \eta &:= \frac{\hat{h}_{\hat{\mathcal{P}}}(\gamma)}{\text{eq}(\gamma, \tau)}.\end{aligned}$$

6. $\mathbf{V}$ outputs the new instance $((\gamma_0, \gamma_1), (\alpha, \mu), (\beta, \eta))$ and f_0, f_1 (as oracles).

The prover's computational effort is dominated by computing the polynomials $\hat{h}_{\mathcal{C}}, \hat{h}_{\hat{\mathcal{P}}}$. This can be done in linear time (specifically, $O(n + |\hat{\mathbf{p}}|)$ field operations), by adapting ideas from [EG23].

The soundness analysis of the protocol is non-trivial, and relies on the notion of *mutual correlated agreement* [ACFY25] (see Definition 3.15). Informally, a linear code $\mathcal{C}$ has mutual correlated agreement with proximity radius δ_{PG} and error err_{PG} if, for every $f_0, f_1: [n] \rightarrow \mathbb{F}$ and $\delta \in (0, \delta_{\text{PG}})$, the following holds with high probability at least $1 - \text{err}_{\text{PG}}$ over a random choice of $\gamma \in \mathbb{F}$: if $(1 - \gamma) \cdot f_0 + \gamma \cdot f_1$ agrees with $\mathcal{C}$ on $S \subseteq [n]$ with $|S| \geq (1 - \delta) \cdot n$ then f_0 and f_1 agree with the interleaved code $\mathcal{C}^2$ on the *same* agreement set S . We prove that the protocol preserves distance, with small soundness error, and has proximity radius $\delta^* := \delta_{\text{PG}}$.

Lemma 4 (informal). *Suppose that the linear code $\mathcal{C}$ has mutual correlated agreement with proximity radius δ_{PG} and error err_{PG} . For every $\delta \in (0, \delta_{\text{PG}})$, Construction 3 has soundness error*

$$\frac{|\Lambda(\mathcal{C}^2, \delta)| \cdot \text{poly}(\log n, \log M)}{|\mathbb{F}|} + \text{err}_{\text{PG}}(\delta).$$

Proof sketch. For simplicity, we only consider PESAT constraints in this sketch. Extending the analysis to include codeword evaluation constraints is straightforward. As we are in the soundness case, for every $(u_0, u_1) \in \Lambda(\mathcal{C}, f_0, \delta) \times \Lambda(\mathcal{C}, f_1, \delta)$ it must be that $\hat{P}(\beta_0, \mathcal{C}^{-1}(u_0)) \neq \eta_0$ or $\hat{P}(\beta_1, \mathcal{C}^{-1}(u_1)) \neq \eta_1$. This implies that the same condition holds for every codeword $(u_0, u_1) \in \Lambda(\mathcal{C}^2, (f_0, f_1), \delta)$ in the interleaved code.

1. The initial statement consists of the proximity claim of the two input oracles to the respective constrained code. The first verifier message reduces these claims to a single proximity claim to the *interleaved code*. We argue that, with probability at most $\frac{|\Lambda(\mathcal{C}^2, \delta)|}{|\mathbb{F}|}$ over the choice of τ , there is a pair of codewords $(u_0, u_1) \in \Lambda(\mathcal{C}^2, (f_0, f_1), \delta)$ such that

$$\sum_{b \in \{0,1\}} \text{eq}(b, \tau) \cdot \hat{P}(\beta_b, \mathcal{C}^{-1}(u_b)) = (1 - \tau) \cdot \eta_0 + \tau \cdot \eta_1.$$

Fix a pair of codewords $(u_0, u_1) \in \Lambda(\mathcal{C}^2, (f_0, f_1), \delta)$; by soundness, $\exists b \in \{0,1\}$ such that $\hat{P}(\beta_b, \mathcal{C}^{-1}(u_b)) \neq \eta_b$. Hence $\sum_{b \in \{0,1\}} \text{eq}(b, \mathbf{x}) \cdot \hat{P}(\beta_b, \mathcal{C}^{-1}(u_b)) \neq \sum_{b \in \{0,1\}} \text{eq}(b, \mathbf{x}) \cdot \eta_b$, so by the polynomial identity lemma the equality at τ holds with probability at most $\frac{1}{|\mathbb{F}|}$. Then we take a union bound over all $|\Lambda(\mathcal{C}^2, \delta)|$ pairs of codewords $(u_0, u_1) \in \Lambda(\mathcal{C}^2, (f_0, f_1), \delta)$.

2. We convert the proximity claim about interleaved codewords to the final proximity claim, again with respect to the constrained code. First, we show that, with high probability, the random combination of interleaved codewords does not satisfy the new claim. Second, mutual correlated agreement allows replacing this random linear combination of interleaved codewords with codewords close to the random linear combination of the original words.

- (a) We argue that, with probability at most $\frac{|\Lambda(\mathcal{C}^2, \delta)| \cdot \text{poly}(\log n, \log M)}{|\mathbb{F}|}$, there is a pair of codewords $(u_0, u_1) \in \Lambda(\mathcal{C}^2, (f_0, f_1), \delta)$ such that

$$\hat{h}_{\hat{P}}(\gamma) = \text{eq}(\gamma, \tau) \cdot \hat{P}(\hat{\mathbf{B}}(\gamma), \mathcal{C}^{-1}((1 - \gamma) \cdot u_0 + \gamma \cdot u_1)).$$

Fix $(u_0, u_1) \in \Lambda(\mathcal{C}^2, (f_0, f_1), \delta)$. It must be that $\hat{h}_{\hat{P}}(\mathbf{X}) \neq \text{eq}(\mathbf{X}, \tau) \cdot \hat{P}(\hat{\mathbf{B}}(\mathbf{X}), \mathcal{C}^{-1}((1 - \mathbf{X}) \cdot u_0 + \mathbf{X} \cdot u_1))$ as the left hand side sums to $(1 - \tau) \cdot \eta_0 + \tau \cdot \eta_1$ (else the verifier immediately rejects) and the right hand side cannot (or else the conditions of Item 1 are violated). By the polynomial identity lemma the probability the two sides agree at γ is at most $\frac{\text{poly}(\log n, \log M)}{|\mathbb{F}|}$. Then we take a union bound over all $|\Lambda(\mathcal{C}^2, \delta)|$ pairs of codewords $(u_0, u_1) \in \Lambda(\mathcal{C}^2, (f_0, f_1), \delta)$.

- (b) By [ACFY25, Lemma 4.13], mutual correlated agreement of $\mathcal{C}$ implies that, except with probability at most $\text{err}_{\text{PC}}(\delta)$ over the choice of γ ,

$$\Lambda(\mathcal{C}, (1 - \gamma) \cdot f_0 + \gamma \cdot f_1, \delta) = \{(1 - \gamma) \cdot u_0 + \gamma \cdot u_1 : (u_0, u_1) \in \Lambda(\mathcal{C}^2, (f_0, f_1), \delta)\}.$$

Let $\beta := \hat{\mathbf{B}}(\gamma)$ and $\eta := \hat{h}(\gamma)/\text{eq}(\gamma, \tau)$ (as in the protocol). If $\Delta((1 - \gamma) \cdot f_0 + \gamma \cdot f_1, \mathcal{C}[(\cdot), (\hat{P}, \beta, \eta)]) \leq \delta$ then there is $u \in \Lambda(\mathcal{C}, f_0 + \gamma \cdot f_1, \delta)$ such that $\hat{P}(\beta, \mathcal{C}^{-1}(u)) = \eta$. But then, by Item 2b, $u = (1 - \gamma) \cdot u_0 + \gamma \cdot u_1$ for some $(u_0, u_1) \in \Lambda(\mathcal{C}^2, (f_0, f_1), \delta)$ and so $\hat{P}(\beta, \mathcal{C}^{-1}((1 - \gamma) \cdot u_0 + \gamma \cdot u_1)) = \eta$, which contradicts Item 2a. Thus, no codeword close to the random linear combination of the original input words can satisfy the updated claim.

□

See Section 6 for technical details about twin constraint pseudo-batching, including a more general protocol that handles ℓ oracles $f_1, \dots, f_\ell$, each with its own twin constraint.

2.7 Straightline extraction for every linear code

The IORs that we presented are used to obtain a corresponding accumulation scheme ACC in the ROM, which is a *non-interactive* protocol. This requires the IORs to satisfy (an appropriate flavor of) *state-restoration security* [BCS16; CY24]. Specifically, in order to ensure that ACC satisfies straightline knowledge soundness in the ROM, the IORs must satisfy *straightline state-restoration knowledge soundness*.⁵

We prove that the IORs in this paper satisfy the aforementioned security notion, *even if the underlying code $\mathcal{C}$ does not have an efficient error-tolerant decoder*. This is notable in the linear-time prover setting because there are linear-time-encodable codes of practical interest for which there are no known efficient error-tolerant decoders.

Achieving straightline extraction for every linear code $\mathcal{C}$, however, demands a different approach compared to prior work. Indeed, prior work on IOPs/IORs achieves straightline extraction via error-tolerant decoders (or else, when no error-tolerant decoder is available, prior work only achieves the weaker notion of rewinding extraction, e.g., as in [BCGGHJ17; GLSTW23; DP24; ZCF24]).

Below we sketch our approach, and then in Section 2.8 we explain how a new notion of round-by-round knowledge soundness aids in carrying out our approach.

⁵Alternatively, one could aim for the weaker notion of *rewinding* knowledge soundness for ACC, which in turn would require the IORs to satisfy the weaker notion of *rewinding* state-restoration knowledge soundness. However rewinding extraction typically comes with downsides, such as worse error bounds and expected-time extraction.

Straightline extraction in an SR game. The security property that we target considers a state-restoration game, which models the (idealized) Fiat–Shamir security of the public-coin interactive protocol at hand. Informally, an adversary $\tilde{\mathbf{P}}_{\text{sr}}$ makes a number of moves (up to a budget) that model queries to the random oracle and receives corresponding responses, and ultimately outputs a final choice of statement and prover messages. Straightline state-restoration knowledge soundness considers the probability that $\tilde{\mathbf{P}}_{\text{sr}}$ ’s output convinces the interactive protocol’s verifier and a straightline extractor $\mathbf{E}_{\text{sr}}$ fails to find a witness for the statement. The extractor $\mathbf{E}_{\text{sr}}$ is straightline in that it receives as input the statement, prover messages, and move-response trace from the state-restoration game (and, crucially, does *not* receive black-box access to the adversary).

Prior work has adapted these notions and definitions from IOPs to IORs [BMNW25b].

The challenge. Prior work on IOPs/IOs that achieves straightline extraction relies on efficient error-tolerant decoding for the underlying code. Most notably, the Reed–Solomon code has efficient error-tolerant decoders (both in the unique-decoding and list-decoding regimes), which are then used by straightline extractors (see, e.g., [BBHR18; ACFY24; ACFY25]). Typically, the first prover message in the protocol allegedly encodes a valid witness, and one proves that, except for a small error probability, if the adversary outputs an accepting transcript in the state-restoration game then the first prover message is close to the encoding of a valid witness. The straightline extractor $\mathbf{E}_{\text{sr}}$ then uses an error-tolerant decoder on that message to find the valid witness. Crucially, *an efficient extractor $\mathbf{E}_{\text{sr}}$ cannot do so if there is no efficient error-tolerant decoder* (which is the case for a number of codes of interest). We wish to achieve an efficient straightline extractor for every linear code $\mathcal{C}$, without relying on error-tolerant decoding.

The hope. In the state-restoration game the straightline extractor $\mathbf{E}_{\text{sr}}$ receives additional information that we may be able to use to avoid error-tolerant decoding, as we now explain.

Informally (and with simplifications), state-restoration knowledge soundness for an IOR from a relation $\mathcal{R}$ to a relation $\mathcal{R}'$ considers the following event (the probability is taken over the randomness of the state-restoration game SRGame):

$$\left[\begin{array}{l} \nexists y_{\star} \in B_{\delta}(y), (\mathbf{x}, y_{\star}, \mathbf{w}) \notin \mathcal{R} \\ \wedge \exists y'_{\star} \in B_{\delta}(y'), (\mathbf{x}', y'_{\star}, \mathbf{w}') \in \mathcal{R}' \end{array} \middle| \begin{array}{l} (\mathbf{x}, y, (\pi_i)_{i \in [k]}, \mathbf{w}', (\rho_i)_{i \in [k]}) \xleftarrow{\text{tr}_{\text{sr}}} \text{SRGame}(\tilde{\mathbf{P}}_{\text{sr}}) \\ (\mathbf{x}', y') \leftarrow \mathbf{V}^{y, (\pi_i)_{i \in [k]}}(\mathbf{x}, (\rho_i)_{i \in [k]}) \\ \mathbf{w} \leftarrow \mathbf{E}_{\text{sr}}(\mathbf{x}, y, (\pi_i)_{i \in [k]}, \mathbf{w}', (\rho_i)_{i \in [k]}, \text{tr}_{\text{sr}}) \end{array} \right].$$

where $B_{\delta}(\cdot)$ denotes the ball of strings at relative distance $\leq \delta$ from the input.

Observe that the straightline extractor $\mathbf{E}_{\text{sr}}$ is given as input a *valid witness* $\mathbf{w}'$ for the new statement $(\mathbf{x}', y')$ (which considers the target relation $\mathcal{R}'$). This is because the definition of straightline knowledge soundness of a non-interactive reduction also has such a witness, which in turn is justified by the fact that, in an accumulation scheme, straightline knowledge soundness is predicated on the decider accepting. See [BMNW25b] for details.

We outline how a valid witness $\mathbf{w}'$ for $(\mathbf{x}', y')$ helps $\mathbf{E}_{\text{sr}}$ find a valid witness $\mathbf{w}$ for $(\mathbf{x}, y)$ for the two main IORs that we construct in this paper.

- Consider the IOR from the PESAT relation $\mathcal{R}_{\odot}(\mathbb{F})$ to the twin constraint relation $\mathcal{R}_{\mathcal{C}}$ discussed in Section 2.5 (Construction 2). In this case, for a given index PESAT index $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$, $\mathbf{E}_{\text{sr}}$ receives the following inputs:
 - $\mathbf{x} = \mathbf{x} \in \mathbb{F}^{N-k}$ (explicit instance);
 - $y = \perp$ (there is no implicit instance for $\mathcal{R}_{\odot}(\mathbb{F})$);
 - $\pi_1 = f: [n] \rightarrow \mathbb{F}$ (the prover sends one oracle);

- $\mathbf{w}' = u: [n] \rightarrow \mathbb{F}$ (a witness for $(\mathbf{x}', \mathbf{y}')$ where $\mathbf{x}' = ((\boldsymbol{\tau}, \mathbf{x}), 0)$ and $\mathbf{y}' = f$); and
- $\rho_1 = \boldsymbol{\tau} \in \mathbb{F}^{\log M}$ (the verifier sends one random message).

The condition “ $\exists \mathbf{y}'_* \in B_\delta(\mathbf{y}'), (\mathbf{x}', \mathbf{y}'_*, \mathbf{w}') \in \mathcal{R}_c$ ” tells us that $u \in \mathcal{C}[(\hat{P}, (\boldsymbol{\tau}, \mathbf{x}), 0)]$ is δ -close to the output oracle f . Therefore $\mathbf{E}_{\text{sr}}$ can obtain a PESAT witness $\mathbf{w} = \mathbf{w}' \in \mathbb{F}^k$ simply by decoding the codeword $u \in \mathcal{C}$ (and ignoring the potentially corrupted function f).

- Consider the IOR from $\mathcal{R}_c^\ell$ to $\mathcal{R}_c$ that is obtained by combining the IORs discussed in Sections 2.4 and 2.6. In this case, for a given index PESAT index $\mathfrak{i} = (\hat{P}, M, N, k)$, $\mathbf{E}_{\text{sr}}$ receives the following inputs:

- $(\mathbf{x}_1, \dots, \mathbf{x}_\ell)$ where $\mathbf{x}_i = (\boldsymbol{\alpha}_i, \mu_i, \boldsymbol{\beta}_i, \eta_i)$ (explicit instances);
- $(\mathbf{y}_1, \dots, \mathbf{y}_\ell)$ where $\mathbf{y}_i = f_i: [n] \rightarrow \mathbb{F}$ (implicit instances);
- $(\pi_i)_{i \in [k]}$ (the oracles sent by the prover);
- $\mathbf{w}' = u: [n] \rightarrow \mathbb{F}$ (a witness for $(\mathbf{x}', \mathbf{y}')$ where $\mathbf{x}' := (\boldsymbol{\alpha}, \mu, \boldsymbol{\beta}, \eta)$ and $\mathbf{y}' := f: [n] \rightarrow \mathbb{F}$); and
- $(\rho_i)_{i \in [k]}$ (the verifier random messages).

The condition “ $\exists \mathbf{y}'_* \in B_\delta(\mathbf{y}'), (\mathbf{x}', \mathbf{y}'_*, \mathbf{w}') \in \mathcal{R}_c$ ” tells us that $u \in \mathcal{C}[(\boldsymbol{\alpha}, \mu), (\hat{P}, \boldsymbol{\beta}, \eta)]$ is δ -close to the output oracle f . To succeed, the extractor $\mathbf{E}_{\text{sr}}$ must find $u_1, \dots, u_\ell$, with each $u_i \in \mathcal{C}[(\boldsymbol{\alpha}_i, \mu_i), (\hat{P}, \boldsymbol{\beta}_i, \eta_i)]$, that are δ -close to $f_1, \dots, f_\ell$, respectively.

Our key insight is to *leverage mutual correlated agreement for this extraction task*. Specifically, due to the mutual correlated agreement property of $\mathcal{C}$, our IOR is such that, with high probability over the choice of verifier randomness, each f_i agrees with a codeword $u_i \in \mathcal{C}[(\boldsymbol{\alpha}_i, \mu_i), (\hat{P}, \boldsymbol{\beta}_i, \eta_i)]$ at (exactly) the locations where f agrees with the codeword $u \in \mathcal{C}[(\boldsymbol{\alpha}, \mu), (\hat{P}, \boldsymbol{\beta}, \eta)]$.⁶

This motivates the following extraction strategy. First, $\mathbf{E}_{\text{sr}}$ finds the set $S \subseteq [n]$ of locations where f and u agree; note that S contains at least $(1 - \delta) \cdot n$ locations. Next, $\mathbf{E}_{\text{sr}}$ restricts each f_i to S , by replacing symbols in $[n] \setminus S$ with $\perp$. Finally, for each $i \in [\ell]$, $\mathbf{E}_{\text{sr}}$ *erasure corrects* the restriction of f_i to S , thereby obtaining the corresponding codeword u_i . The extractor $\mathbf{E}_{\text{sr}}$ outputs the desired witnesses $(\mathbf{w}_1, \dots, \mathbf{w}_\ell) := (u_1, \dots, u_\ell)$.

Crucially, every linear code has an efficient erasure correction algorithm (which consists of solving a linear system induced by the generator matrix); see [GRS12, Exercise 2.16].

(Note that, in both cases, the extractor $\mathbf{E}_{\text{sr}}$ does not take advantage of the move-response trace tr_{sr} .) Formalizing the above intuition into an analysis requires some care, as we discuss next.

Remark 2.2 (efficiency benefits of erasure correction). Erasure correction is *beneficial even for linear codes with efficient error-tolerant decoding*, because erasure correctors can be (and typically are) faster than error-tolerant decoders. Fixing a distance $\delta \in (0, \delta(\mathcal{C}))$, an error tolerant decoder returns all codewords at distance δ from the input word, which implies that the decoder time is at least $\Omega(|\Lambda(\mathcal{C}, \delta)| \cdot n)$. Instead, an erasure corrector is only required to output the unique erasure correction of the input word, and thus may run in time $O(n)$. In fact, Reed–Solomon codes can be erasure corrected in time $O(n) \cdot \text{polylog}(n)$. Since the (straightline state-restoration) extractor $\mathbf{E}_{\text{sr}}$ uses either the decoder or the corrector as a subroutine, the improvement in asymptotics implies a more efficient accumulation scheme (by way of a faster security reduction).

⁶Prior notions of correlated agreement do not suffice, since they do not relate the agreement sets between the input oracles and the output oracle.

2.8 Round-by-round extraction for every linear code

Directly proving state-restoration security for an IOP/IOR can be cumbersome, so one typically proves that the IOP/IOR satisfies a stronger property that is known to imply (the desired flavor of) state-restoration security (see [CY24] for more). Turning to our setting, the common approach to show that an IOP/IOR satisfies straightline state-restoration knowledge soundness is to show that the IOP/IOR satisfies round-by-round knowledge soundness [CCHLRR18; CMS19; CY24; BMNW25b]. Prior definitions, however, appear too strong. We explain this subtle point and propose a relaxation that we submit as the “right” notion of RBR knowledge soundness for IOPs/IORs.⁷ Crucially, we prove the following: (i) our relaxation still implies straightline state-restoration knowledge soundness; and (ii) our IORs satisfy the relaxation.

Warm-up: RBR soundness. We review the notion of RBR soundness [CCHLRR18], generalized to IORs [BMNW25b]. Informally, an IOR from $\mathcal{R}$ to $\mathcal{R}'$ is RBR sound if its soundness can be modeled as a sequential composition of one-round reductions (each individually sound). This is expressed via a *state function* that captures whether a given instance-transcript pair is good for the prover.

- Before the interaction (the transcript is empty), the instances that are good for the prover are those close to the language $\mathcal{L}(\mathcal{R})$.
- In intermediate rounds of the protocol, the probability (over the verifier’s randomness for that round) that a state transitions from bad into good (for the prover) is small.
- After the interaction (the transcript is full), the instance-transcript pairs that are good for the prover are those that cause the verifier to output instances close to the language $\mathcal{L}(\mathcal{R}')$.

The *round-by-round soundness error* upper bounds the transition probability in each round.

Definition 2.3 (informal). *A state function for an IOR from $\mathcal{R}$ to $\mathcal{R}'$ with proximity radius δ^* is a (possibly inefficient) function State parameterized by a proximity bound $\delta \in (0, \delta^*)$ that, on input an explicit instance $\mathbf{x}$, implicit instance $\mathbf{y}$, and interaction transcript tr , outputs a bit.*

Definition 2.4 (informal). *An IOR from $\mathcal{R}$ to $\mathcal{R}'$ with proximity radius δ^* has **round-by-round soundness error** ε_{rbr} if there exist a state function State for the IOR with such that, for every $\delta \in (0, \delta^*)$, statement $(\mathbf{x}, \mathbf{y})$, and interaction transcript tr , the following holds.*

- Empty transcript: if $\text{tr} = \emptyset$ is the empty transcript, then $\text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}) = 1$ if and only if there exists $\mathbf{y}_\star \in B_\delta(\mathbf{y})$ and $\mathbf{w}$ with $(\mathbf{x}, \mathbf{y}_\star, \mathbf{w}) \in \mathcal{R}$.
- Prover moves: if tr is a transcript where the prover is about to move and $\text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}) = 0$, then $\text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \pi) = 0$ for any prover message π .
- Verifier moves: if tr is a transcript where the verifier is about to move, then

$$\Pr_{\rho_i} \left[\begin{array}{l} \text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}) = 0 \\ \wedge \text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_i) = 1 \end{array} \right] \leq \varepsilon_{\text{rbr}}.$$

- Full transcript: if $\text{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}) = 1$ if and only if $\mathbf{V}^{\mathbf{y}, (\pi_i)_{i \in [k]}}(\mathbf{x}, (\rho_i)_{i \in [k]})$ outputs $(\mathbf{x}', \mathbf{y}')$ such that there exist $\mathbf{y}'_\star \in B_\delta(\mathbf{y}')$ and $\mathbf{w}'$ with $(\mathbf{x}', \mathbf{y}'_\star, \mathbf{w}') \in \mathcal{R}'$.

RBR knowledge soundness in prior work. RBR knowledge soundness, introduced for IOPs in [CMS19], strengthens the above notion to enable (straightline) witness extraction. The definition

⁷IORs generalize IOPs in the sense that an IOP for a relation $\mathcal{R}$ is an IOR from $\mathcal{R}$ to the trivial language $\{1\}$. Even restricted to IOPs, our relaxation differs from prior notions of RBR knowledge soundness.

is motivated by the fact that, in many IOPs/IORs of interest, the prover sends an encoding of the witness as a message in some (usually, the first) round; hence the extractor's only responsibility is to decode this message. Roughly, while RBR soundness upper bounds the state function's transition probability, RBR knowledge soundness upper bounds the state function's transition probability *provided that the extractor fails*. The definition below is adapted to IORs [BMNW25b].

Definition 2.5 (informal). *An IOR from $\mathcal{R}$ to $\mathcal{R}'$ with proximity radius δ^* has **round-by-round knowledge soundness error** ε_{rbr} if there exist a state function State for the IOR and an efficient deterministic extractor $\mathbf{E}_{\text{rbr}}$ such that, for every $\delta \in (0, \delta^*)$, statement $(\mathbf{x}, \mathbf{y})$, and interaction transcript tr , the following holds.*

- Empty transcript: *If $\text{tr} = \emptyset$ is the empty transcript, then $\text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}) = 0$.*
- Prover moves: *if tr is a transcript where the prover is about to move and $\text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}) = 0$, then $\text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \pi) = 0$ for any prover message π .*
- Verifier moves: *if tr is a transcript where the verifier is about to move, then*

$$\Pr_{\rho_i} \left[\begin{array}{l} \text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}) = 0 \\ \wedge \text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_i) = 1 \\ \wedge \forall \mathbf{y}_\star \in B_\delta(\mathbf{y}), (\mathbf{x}, \mathbf{y}_\star, \mathbf{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr})) \notin \mathcal{R} \end{array} \right] \leq \varepsilon_{\text{rbr}}.$$

- Full transcript: *if $\text{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\text{State}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}) = 1$ if and only if $\mathbf{V}^{\mathbf{y}, (\pi_i)_{i \in [k]}}(\mathbf{x}, (\rho_i)_{i \in [k]})$ outputs $(\mathbf{z}', \mathbf{y}')$ such that there exist $\mathbf{y}'_\star \in B_\delta(\mathbf{y}')$ and $\mathbf{w}'$ with $(\mathbf{z}', \mathbf{y}'_\star, \mathbf{w}') \in \mathcal{R}'$.*

The state function in Definition 2.5 plays a different role than the state function in Definition 2.4: it encodes whether the extractor should succeed in outputting a valid witness, not just whether the current state is good for the prover.

We outline how protocols typically satisfy RBR knowledge soundness. For simplicity, assume the prover sends an encoding of the witness in the first round of the protocol. There will exist some critical round of the protocol where: (i) the extractor is defined to output a decoding of the prover's first message; and (ii) the transition probability can be upper bounded assuming the extractor's output is not a valid witness. Outside of the critical round, the transition probability can be upper bounded, *regardless* of whether or not the extractor succeeds (indeed, the extractor can simply be defined to output $\perp$ outside of the critical round). See [ACFY24, Section 7] for an example of how RBR knowledge soundness is established for an IOP.

Unfortunately, our protocols do not satisfy Definition 2.5.⁸ As discussed in Section 2.7, our extraction strategy relies on using a witness for the output instance in order to deduce (via erasure correction) a witness for the input instance. This is simply not possible in Definition 2.5, because the extractor sees only the prover's messages up to the current round.

The main problem here is that, once the critical round occurs, the extractor is expected to immediately output a valid witness. Relaxing this necessitates a new definition.

A relaxation: RBR extraction. We propose a relaxation of RBR knowledge soundness that is compatible with our extraction strategy outlined in Section 2.7. Below, for clarity, we informally refer to this notion as *RBR extraction*.

Recall that an IOP/IOR is RBR sound if it can be expressed as a sequential composition of one-round reductions each of which is individually sound. In analogy, we say that an IOP/IOR is

⁸To be precise, our protocols are not known to satisfy Definition 2.5 when instantiated with an arbitrary linear code. If the code has an efficient error-tolerant decoder, then they satisfy Definition 2.5.

RBR extractable if it can be expressed as a sequential composition of one-round reductions each of which is individually knowledge sound. This is arguably more natural than relying on a “global” extractor as in Definition 2.5.

In more detail, we consider a *knowledge state function*, which captures each round’s intermediate relation by taking as additional input a candidate witness (for that intermediate relation); this is in analogy to a state function, which captures each round’s intermediate language. Then we define a notion of RBR knowledge soundness, where in every round, the extractor is given a valid witness for this round’s relation and is tasked with deducing a witness for the prior round’s relation.

Definition 2.6 (informal). *A knowledge state function for an IOR from $\mathcal{R}$ to $\mathcal{R}'$ with proximity radius δ^* is a (possibly inefficient) function KState parameterized by a proximity bound $\delta \in (0, \delta^*)$ that, on input an explicit instance $\mathbf{x}$, implicit instance $\mathbf{y}$, interaction transcript tr , and knowledge state witness $\mathbf{w}$, outputs a bit.*

Definition 2.7 (informal). *An IOR from $\mathcal{R}$ to $\mathcal{R}'$ with proximity radius δ^* has **round-by-round knowledge soundness error** ε_{rbr} if there exist a knowledge state function KState for the IOR and an efficient deterministic extractor $\mathbf{E}_{\text{rbr}}$ such that, for every $\delta \in (0, \delta^*)$, statement $(\mathbf{x}, \mathbf{y})$, and interaction transcript tr , and knowledge state witness $\mathbf{w}$, the following holds.*

- Empty transcript: if $\text{tr} = \emptyset$ is the empty transcript, then $\text{KState}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if there exists $\mathbf{y}_\star \in B_\delta(\mathbf{y})$ with $(\mathbf{x}, \mathbf{y}_\star, \mathbf{w}) \in \mathcal{R}$.
- Prover moves: if tr is a transcript where the prover is about to move and $\text{KState}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$, then $\text{KState}_\delta(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \pi, \mathbf{w}) = 0$ for every prover message π .
- Verifier moves: if tr is a transcript where the verifier is about to move, then

$$\Pr_{\rho_i} \left[\exists \mathbf{w} : \begin{array}{l} \text{KState}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_i, \mathbf{w})) = 0 \\ \wedge \text{KState}_\delta(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_i, \mathbf{w}) = 1 \end{array} \right] \leq \varepsilon_{\text{rbr}}.$$

- Full transcript: if $\text{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\text{KState}_\delta(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $\mathbf{V}^{\mathbf{y}, (\pi_i)_{i \in [k]}}(\mathbf{x}, (\rho_i)_{i \in [k]})$ outputs $(\mathbf{x}', \mathbf{y}')$ such that there exists $\mathbf{y}'_\star \in B_\delta(\mathbf{y}')$ with $(\mathbf{x}', \mathbf{y}'_\star, \mathbf{w}) \in \mathcal{R}'$.

All IORs in this paper are proved to satisfy RBR extraction, regardless of the underlying linear code. Note that Definition 2.7 is a (strict) relaxation of Definition 2.5 (see Appendix C). Nevertheless, we prove that RBR extraction still implies (straightline) state-restoration knowledge soundness (the low-level notion of straightline knowledge soundness that ensures Fiat–Shamir security).

Theorem 2.8 (informal). *If a k -round IOR has round-by-round knowledge soundness error ε_{rbr} (per our Definition 2.7), then the IOR has straightline state-restoration knowledge soundness error*

$$\varepsilon_{\text{sr}}(t) \leq (t + k) \cdot \varepsilon_{\text{rbr}}.$$

Technical details. In Section 4 we formally define (our relaxation of) round-by-round knowledge soundness for IORs; all IORs in this paper are proved to satisfy this notion. Separately, in Appendix B we state and prove a formal version of Theorem 2.8.

2.9 Summary and instantiations

In the prior sections we discussed the IORs, based on a given linear code $\mathcal{C}$, claimed in Theorem 2:

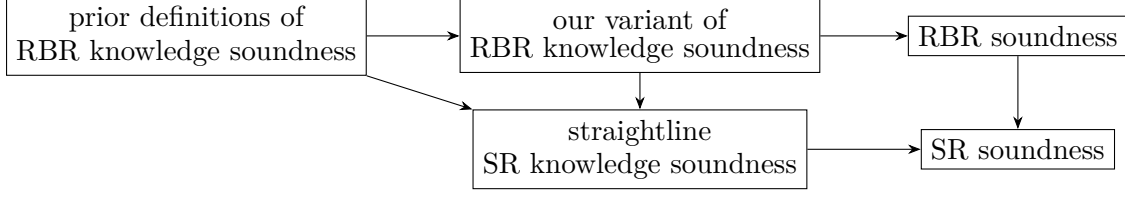


Figure 1: Implications between various notions of round-by-round (RBR) and state-restoration (SR) soundness for IOPs/IORs. In Appendix B we prove that our variant of RBR knowledge soundness implies straightline SR knowledge soundness. In Appendix C we prove that prior definitions of RBR knowledge soundness imply our variant of RBR knowledge soundness. Other implications are known.

- an IOR from $\mathcal{R}_\otimes(\mathbb{F})$ to $\mathcal{R}_\mathcal{C}$ (outlined in Section 2.5), where the prover performs $O(\text{enc}_\mathcal{C})$ field operations and the verifier makes no queries; and
- an IOR from $\mathcal{R}_\mathcal{C}^\ell$ to $\mathcal{R}_\mathcal{C}$ (resulting from discussions in Sections 2.4 and 2.6), where the prover performs $O(\ell \cdot (n + |\hat{p}|) + t \cdot \log n)$ field operations and the verifier makes $\ell \cdot t$ queries.

Both IORs have error $\leq 2^{-\lambda}$ provided $\mathbb{F}$ is sufficiently large and $t > \frac{\lambda}{-\log(1-\delta_{\text{PG}})}$, where δ_{PG} is the proximity radius for the mutual correlated agreement property of $\mathcal{C}$.

In Section 10 we explain how, via compilation in the ROM, these two IORs imply a corresponding accumulation scheme $\text{ACC} = (\mathbf{P}_{\text{ACC}}, \mathbf{V}_{\text{ACC}}, \mathbf{D}_{\text{ACC}})$ for $\mathcal{R}_\otimes(\mathbb{F})$. In general, the efficiency parameters of ACC depend on properties of the linear code $\mathcal{C}$: (i) the encoding time $\text{enc}_\mathcal{C}$ affects the proving time of the accumulation prover $\mathbf{P}_{\text{ACC}}$ and accumulation decider $\mathbf{D}_{\text{ACC}}$; and (ii) the proximity radius δ_{PG} and error err_{PG} of the mutual correlated agreement property affect the query complexity of the accumulation verifier $\mathbf{V}_{\text{ACC}}$ and the minimum size of the field $\mathbb{F}$. See Theorem 10.3 for a formal theorem statement that details efficiency and security parameters.

Statements for mutual correlated agreement are known for every linear code, both in the unique decoding and list-decoding regime.

- *Unique decoding:* every linear code $\mathcal{C}$ has mutual correlated agreement with proximity radius $\delta_{\text{PG}} = \frac{\delta(\mathcal{C})}{3}$ and error $\text{err}_{\text{PG}} = \frac{n}{|\mathbb{F}|}$ [AHIV17; ACFY25].
- *List decoding:* every linear code $\mathcal{C}$ has mutual correlated agreement with proximity radius $\delta_{\text{PG}} = 1 - \sqrt[3]{1 - \delta(\mathcal{C}) + \eta}$ (for $\eta > 0$) and error $\text{err}_{\text{PG}} = \frac{O(n) + \frac{2}{\sqrt[3]{1 - \delta(\mathcal{C}) + \eta} - \sqrt{1 - \delta(\mathcal{C}) + \eta}}}{\eta \cdot |\mathbb{F}|}$ [GKL24; Zei24].

Any improvements in these bounds imply corresponding efficiency improvements in our protocols.

Instantiation with linear-time encodable codes. If we fix $\mathcal{C}$ to be a linear-time encodable code (with constant relative distance) then we obtain Theorem 1.⁹ Codes satisfying these requirement include, e.g., expander-based codes [Spi96; GI05; DI14; GLSTW23] and repeat-accumulate-accumulate codes [BCFRRZ25]. Deriving improved mutual correlated agreement parameters for these special cases (compared to the generic bounds) is an important research direction.

Instantiation with Reed–Solomon codes. Reed–Solomon codes are a popular code choice, because they have quasilinear encoding time and they are maximum distance separable (i.e., they meet the Singleton bound, so they enjoy an optimal rate-distance tradeoff). Better results on mutual correlated agreement hold (or are believed to hold) for this case.

⁹Both the unique decoding and list decoding parameters yield the claimed asymptotic efficiency.

- *Unique decoding:* Reed–Solomon codes have mutual correlated agreement with proximity radius $\delta_{\text{PG}} = \frac{\delta(\mathcal{C})}{2}$ and error $\text{err}_{\text{PG}} = \frac{n}{|\mathbb{F}|}$ [BCIKS20; ACFY25].
- *List decoding:* Reed–Solomon codes have correlated agreement with proximity radius $\delta_{\text{PG}} = 1 - \sqrt{\delta(\mathcal{C})} - \eta$ for $\eta > 0$ with error $\text{err}_{\text{PG}} = \frac{n^{2 \cdot (1 - \delta(\mathcal{C}))^2}}{(2\eta)^7 \cdot |\mathbb{F}|}$ [BCIKS20]; moreover, [ACFY25] conjectures that this result extends (with similar parameters) to mutual correlated agreement.

Choice of parameters. Configuring the WARP protocol requires care, due to tradeoffs that arise in the choice of linear code $\mathcal{C}$ and proximity bound δ .

- *Which code $\mathcal{C}$?* Choosing a linear-time encodable code is necessary to achieve linear-time accumulation. However, the situation is more nuanced when considering concrete efficiency.

Linear-time encodable codes of practical interest typically have some drawbacks: (i) smaller minimum distance than a Reed–Solomon code of the same rate (Reed–Solomon codes are maximum distance separable); and (ii) worse (known) bounds for mutual correlated agreement compared to Reed–Solomon codes. This results in an IOR verifier with larger query complexity and, consequently, an accumulation verifier with larger hash complexity; in turn, this impacts the overall *prover time* of the PCD construction obtained from the accumulation scheme (as the PCD construction proves correctness of a circuit that includes the accumulation verifier).

In light of this, the quasilinear encoding time of Reed–Solomon codes might be (concretely) small enough that the better query complexity offsets the performance gains in the prover that come from using a linear-time encodable code.

- *Which proximity bound δ ?* Suppose $\mathcal{C}$ has mutual correlated agreement with proximity radius δ_{PG} and error err_{PG} . Our IORs preserve distance for every proximity bound $\delta \in (0, \delta_{\text{PG}})$ (i.e., the IORs have proximity radius $\delta^* := \delta_{\text{PG}}$). We generally recommend setting the proximity bound δ to be as large as possible (using the largest proximity radius δ_{PG} known for $\mathcal{C}$), because larger δ results in fewer queries and thereby a smaller accumulation verifier.

Smaller choices of δ can, however, offer useful tradeoffs. First, known mutual correlated agreement errors worsen as δ increases, which translates into using larger fields to ensure soundness. Using a smaller δ might then allow saving on field size, improving the efficiency of field arithmetic for both the prover and verifier. Second, when $\delta \leq \delta(\mathcal{C})/2$ (i.e., the proximity radius is at most the code’s unique decoding radius), the protocols we presented can be simplified to avoid using out-of-domain samples. This improves the accumulation prover’s efficiency (and round complexity). Nevertheless, we suspect that the benefits of choosing a smaller proximity radius δ are not sufficient to offset the increased verifier complexity.

Carefully navigating these tradeoffs requires an implementation (and suitable optimizations). We leave such efforts to future work.

Comparison of hash-based accumulation schemes. Table 1 compares WARP with other hash-based accumulation schemes [BMNW25b; BMMS25] and a bounded-depth accumulation scheme [BMNW25a]. WARP achieves the smallest prover and verifier arithmetic costs, matched only by concurrent work [BMMS25]. Moreover, WARP achieves the smallest cryptographic costs, in the number of Merkle commitments computed by the prover and the number of Merkle openings checked by the verifier. The latter is due to working in the list-decoding regime (as does [BMNW25b]).

	prover arithmetic cost	prover Merkle commitments	verifier arithmetic cost	verifier Merkle openings	repetition parameter t	list decoding
[BMNW25a][RS]	$O\left(\begin{array}{c} \ell_1 \cdot n \log n \\ + \ell \cdot \log \ell \cdot n \end{array}\right)$	$\text{MT}(n, \mathbb{F}^{\ell_1})$ + $\text{MT}(n, \mathbb{F})$	$O(\ell \cdot d_{\text{acc}} \cdot t)$	$(2 + \ell_2) \cdot t$	$\frac{\lambda}{-\log(1 - d_{\text{acc}} \cdot \delta_{\text{ud}}(\mathcal{C}))}$	
[BMNW25a][C]	$O\left(\begin{array}{c} \ell_1 \cdot \text{enc}_{\mathcal{C}} \\ + \ell \cdot \log \ell \cdot n \end{array}\right)$	$\text{MT}(n, \mathbb{F}^{\ell_1})$ + $\text{MT}(n, \mathbb{F}^2)$	$O(\ell \cdot d_{\text{acc}} \cdot t)$	$(2 + \ell_2) \cdot t$	$\frac{\lambda}{-\log(1 - d_{\text{acc}} \cdot \delta_{\text{ud}}(\mathcal{C}))}$	
[BMNW25b]	$O\left(\begin{array}{c} \ell_1 \cdot n \log n \\ + \ell \cdot \log \ell \cdot n \end{array}\right)$	$\text{MT}(n, \mathbb{F}^{\ell_1})$ + $\text{MT}(n, \mathbb{F}^2)$	$O(\ell \cdot t^2)$	$(1 + \ell_2) \cdot t$	$\frac{\lambda}{-\log(1 - \delta_{\text{ca}}(\mathcal{C}))}$	✓
WARP[C]	$O\left(\begin{array}{c} \ell_1 \cdot \text{enc}_{\mathcal{C}} \\ + \ell \cdot n + \lambda \cdot \log n \end{array}\right)$	$\text{MT}(n, \mathbb{F}^{\ell_1})$ + $\text{MT}(n, \mathbb{F})$	$O\left(\begin{array}{c} \ell \cdot (\log n + t) \\ + t \cdot \log n \end{array}\right)$	$(1 + \ell_2) \cdot t$	$\frac{\lambda}{-\log(1 - \delta_{\text{mca}}(\mathcal{C}))}$	✓
[BMMS25][C]	$O\left(\begin{array}{c} (\ell_1 + 1/\rho(\mathcal{C})) \cdot \text{enc}_{\mathcal{C}} \\ + \ell \cdot n + \lambda \cdot \log n \end{array}\right)$	$\text{MT}(n, \mathbb{F}^{\ell_1})$ + $2 \cdot \text{MT}(n, \mathbb{F})$ + $\text{MT}(n, \mathbb{F}^{1/\rho(\mathcal{C})})$	$O\left(\begin{array}{c} \ell \cdot (\log n + t) \\ + t \cdot \log n \end{array}\right)$	$(3 + \ell_2) \cdot t$	$\frac{\lambda}{-\log(1 - \delta_{\text{ud}}(\mathcal{C}))}$	

Table 1: Comparison of hash-based accumulation schemes. For ease of notation we assume that $N, M, |\hat{\mathbf{p}}| = O(n)$ and $d = O(1)$. In the first two rows, d_{acc} denotes the depth for the bounded-depth accumulation scheme in [BMNW25a]. For the given code $\mathcal{C}$, $\rho(\mathcal{C})$ is the rate, $\delta_{\text{ca}}(\mathcal{C}), \delta_{\text{mca}}(\mathcal{C})$ are the proximity radii for correlated agreement and mutual correlated agreement, and $\delta_{\text{ud}}(\mathcal{C}) := \min\{\delta_{\text{ca}}(\mathcal{C}), \delta(\mathcal{C})/2\}$. Moreover, $\text{MT}(n, \Sigma)$ denotes the cost of computing a Merkle commitment to a vector in Σ^n , and Merkle openings refers to the number of authentication paths.

3 Preliminaries

We define objects and state results that we use in this paper. We use the following notation.

- The set of powers of two is $2^{\mathbb{N}} := \{2^n : n \in \mathbb{N}\}$.
- The preimage set of $t \in T$ under a function $f : S \rightarrow T$ is $f^{-1}(t) := \{s \in S : f(s) = t\}$.
- For two functions $f, g : [n] \rightarrow \mathbb{F}$ and a set $S \subseteq [n]$, we write $f(S) = g(S)$ to mean that $f(x) = g(x)$ for every $x \in S$. Conversely, if $f(S) \neq g(S)$, then there exists $x \in S$ such that $f(x) \neq g(x)$.
- For two functions $f, g : [n] \rightarrow \mathbb{F}$, $\Delta(f, g)$ is the fractional Hamming distance between f and g (the fraction of points in which they disagree). For a set $S \subseteq \mathbb{F}^{[n]}$, $\Delta(f, S) := \min_{h \in S} \Delta(f, h)$ (or if S is empty, $\Delta(f, S) := 1$).
- For $i \in [n]$, let $\text{binary}(i) \in \mathbb{F}^{\log n}$ denote the binary representation of i (embedded in $\mathbb{F}$).
- For a polynomial $\hat{p}$, let $\deg_{\text{tot}}(\hat{p}) \in \mathbb{N}$ denote the total degree of $\hat{p}$.
- The equality polynomial eq for m variables is $\text{eq}(\mathbf{X}, \mathbf{Y}) := \prod_{i=1}^m (\mathbf{X}_i \cdot \mathbf{Y}_i + (1 - \mathbf{X}_i) \cdot (1 - \mathbf{Y}_i))$.
- For a function $f : S \rightarrow \mathbb{F}$ where $|S| \in 2^{\mathbb{N}}$, let $\hat{f}$ denote the multilinear extension of f , implicitly identifying S with $\{0, 1\}^{\log |S|}$.

Claim 3.1. For every $\hat{f} \in \mathbb{F}^{<2}[\mathbf{X}_1, \dots, \mathbf{X}_m]$ and $\boldsymbol{\tau} \in \mathbb{F}^m$,

$$\hat{f}(\boldsymbol{\tau}) = \sum_{\mathbf{b} \in \{0,1\}^m} \hat{f}(\mathbf{b}) \cdot \text{eq}(\boldsymbol{\tau}, \mathbf{b}).$$

Lemma 3.2 (Ore–DeMillo–Lipton–Schwartz–Zippel lemma). Let $\hat{p} \in \mathbb{F}^{<d}[X_1, \dots, X_m]$ be a non-zero polynomial. Then,

$$\Pr_{\boldsymbol{\tau} \leftarrow \mathbb{F}^m} [\hat{p}(\boldsymbol{\tau}) = 0] \leq \frac{m \cdot (d-1)}{|\mathbb{F}|}.$$

3.1 Relations

For the purposes of this work, a *relation* $\mathcal{R}$ consists of quadruples $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \mathfrak{w}) \in \mathcal{R}$, where $\mathfrak{i}$ is the index, $\mathfrak{x}$ the explicit instance, $\mathfrak{y}$ the implicit instance, and $\mathfrak{w}$ the witness. For a relation $\mathcal{R} = \{(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \mathfrak{w})\}$, let $\mathcal{L}(\mathcal{R}) := \{(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}) : \exists \mathfrak{w}, (\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \mathfrak{w}) \in \mathcal{R}\}$ be the language induced by $\mathcal{R}$.

Multi-instance relations. For $\ell \in \mathbb{N}$, we define the *multi-instance relation*

$$\mathcal{R}^\ell := \{(\mathfrak{i}, (\mathfrak{x}_1, \dots, \mathfrak{x}_\ell), (\mathfrak{y}_1, \dots, \mathfrak{y}_\ell), (\mathfrak{w}_1, \dots, \mathfrak{w}_\ell)) : \forall i \in [\ell], (\mathfrak{i}, \mathfrak{x}_i, \mathfrak{y}_i, \mathfrak{w}_i) \in \mathcal{R}\}.$$

Distance from a relation. Let $\mathcal{R}$ be a relation. For any statement $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y})$ (not necessarily in the language), we associate an *oracle*, denoted $\pi_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y})$. Unless stated otherwise, we define this to be the implicit instance (if $\mathfrak{y} = (\mathfrak{y}_1, \dots, \mathfrak{y}_\ell)$ is a tuple of words, then $\pi_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y})$ should be interpreted as an interleaved word). Let $\Pi_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}) := \{\pi_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}) : (\mathfrak{i}, \mathfrak{x}, \mathfrak{y}) \in \mathcal{L}(\mathcal{R})\}$ be the set of oracles in the relation induced by a given index and explicit instance. We define the distance of a statement $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y})$ from the relation $\mathcal{R}$ to be

$$\Delta_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}) := \Delta(\pi_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}), \Pi_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x})).$$

3.2 Interactive oracle reductions

Interactive Oracle Reductions (IORs) [BCGGRS19; BMNW25b] are a generalization of Interactive Oracle Proofs (IOPs) [BCS16; RRR16]. Below we describe the flavor of IOR relevant to this paper.

A public-coin holographic IOR of proximity from a relation $\mathcal{R}$ to $\mathcal{R}'$ is a triple $\text{IOR} = (\mathbf{I}, \mathbf{P}, \mathbf{V})$ that works as follows.

- *Offline phase.* The indexer $\mathbf{I}$ is a deterministic algorithm that receives as input an index $\mathfrak{i}$ and outputs a tuple $(\mathfrak{i}', \iota, \mathbb{I})$ consisting of a new index $\mathfrak{i}'$, a short index ι (an explicit input for $\mathbf{V}$), and a long index $\mathbb{I}$ (an implicit input for $\mathbf{V}$).

In this paper, all IORs satisfy $\mathfrak{i}' = \mathfrak{i}$ (the index is unchanged) and $\mathbb{I} = \perp$ (no long index). To simplify notation, we write $\mathbf{I}(\mathfrak{i}) = \iota$ ($\mathbf{I}$ is tasked to output the short index ι only).

- *Online phase.* The prover $\mathbf{P}$ is an interactive algorithm that receives as input $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \mathfrak{w}) \in \mathcal{R}$, and the verifier $\mathbf{V}$ is an interactive oracle algorithm that receives as (explicit) input $(\iota, \mathfrak{x})$ and query access to $\mathfrak{y}$. They interact over k rounds as follows. In each round $i \in [k]$, $\mathbf{P}$ sends a proof string π_i and $\mathbf{V}$ sends a uniformly random message $\rho_i \in \{0, 1\}^{r_i}$. After k rounds of interaction, $\mathbf{V}$ makes queries to $\mathfrak{y}$ and proof strings $\pi_1, \dots, \pi_k$ sent by $\mathbf{P}$. After that:
 - $\mathbf{V}$ either rejects or outputs a new explicit instance $\mathfrak{x}'$ and implicit instance $\mathfrak{y}'$, and
 - $\mathbf{P}$ outputs a new witness $\mathfrak{w}'$ such that $(\mathfrak{i}, \mathfrak{x}', \mathfrak{y}', \mathfrak{w}') \in \mathcal{R}'$.

Without loss of generality, $\mathbf{V}$'s output is a deterministic function of its input and the transcript. Note that $\mathbf{V}$ specifies $\mathfrak{y}'$ implicitly, in terms of $\mathfrak{y}$ and $\pi_1, \dots, \pi_k$.

We say that IOR has (perfect) completeness and soundness error β with proximity radius $\delta^* \in [0, 1]$ if the following hold.

- **Completeness.** For every $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \mathfrak{w}) \in \mathcal{R}$,

$$\Pr_{\rho_1, \dots, \rho_k} \left[(\mathfrak{i}, \mathfrak{x}', \mathfrak{y}', \mathfrak{w}') \in \mathcal{R}' \mid \begin{array}{l} \pi_1 \leftarrow \mathbf{P}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \mathfrak{w}) \\ \rho_1 \leftarrow \{0, 1\}^{r_1} \\ \vdots \\ (\pi_k, \mathfrak{w}') \leftarrow \mathbf{P}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \mathfrak{w}, \rho_1, \dots, \rho_{k-1}) \\ \rho_k \leftarrow \{0, 1\}^{r_k} \\ (\mathfrak{x}', \mathfrak{y}') \leftarrow \mathbf{V}^{\mathfrak{y}, \pi_1, \dots, \pi_k}(\mathbf{I}(\mathfrak{i}), \mathfrak{x}, \rho_1, \dots, \rho_k) \end{array} \right] = 1.$$

- **Soundness.** For every proximity bound $\delta \in (0, \delta^*)$, statement $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}) \notin \mathcal{L}(\mathcal{R})$ with $\Delta_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}) > \delta$, and unbounded malicious prover $\tilde{\mathbf{P}}$,

$$\Pr_{\rho_1, \dots, \rho_k} \left[\Delta_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}', \mathfrak{y}') \leq \delta \mid \begin{array}{l} \pi_1 \leftarrow \tilde{\mathbf{P}} \\ \rho_1 \leftarrow \{0, 1\}^{r_1} \\ \vdots \\ \pi_k \leftarrow \tilde{\mathbf{P}}(\rho_1, \dots, \rho_{k-1}) \\ \rho_k \leftarrow \{0, 1\}^{r_k} \\ (\mathfrak{x}', \mathfrak{y}') \leftarrow \mathbf{V}^{\mathfrak{y}, \pi_1, \dots, \pi_k}(\mathbf{I}(\mathfrak{i}), \mathfrak{x}, \rho_1, \dots, \rho_k) \end{array} \right] \leq \beta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \delta).$$

Sometimes we leave implicit some parameters of β if they are clear from context.

Remark 3.3. The above soundness property describes a **distance-preserving IOR**, in the sense that both the old and new statements use the same proximity bound δ . More generally, an IOR may be specified with a new proximity bound δ' (which is a function of δ and possibly other parameters). All IORs in this paper are distance-preserving, so we have specialized the definition to this case.

In this paper we construct IORs that satisfy a stronger security notion, **round-by-round knowledge soundness**, that we introduce in Section 4. This ensures Fiat–Shamir security of the corresponding non-interactive reduction in the ROM, as we prove in Appendix B.

Efficiency measures. We study several efficiency measures, which are implicitly functions of the index $\mathfrak{i}$ and explicit instance $\mathfrak{x}$.

- *Rounds* k : there are k rounds of interaction (a round is a prover message and verifier message).
- *Alphabet* Σ : the implicit instance $\mathfrak{y}$ and each prover message π_i is over the alphabet Σ ; in this paper, the alphabet is always a finite field $\mathbb{F}$.
- *Proof length* l : the total number of symbols in the proofs $\pi_1, \dots, \pi_k$.
- *Input queries* q_y : the number of alphabet elements read by the verifier from $\mathfrak{y}$.
- *Proof queries* q_π : the number of alphabet elements read by the verifier from $\pi_1, \dots, \pi_k$.
- *Randomness* r : the verifier's i -th message ρ_i has length r_i and $r := \sum_{i=1}^k r_i$ is the total number of random bits sent by the verifier.
- *Prover time* pt : $\mathbf{P}$ runs in time pt measured in algebraic field operations.
- *Verifier time* vt : $\mathbf{V}$ runs in time vt measured in algebraic field operations.

3.3 Zero-evaders

Definition 3.4. A **zero-evader** over $\mathbb{F}$ with error $\varepsilon_{\text{zero}}$ is a function $\text{ZE}: D_{\text{ZE}} \rightarrow \mathbb{F}^m$ such that

$$\forall \mathbf{v} \in \mathbb{F}^m \setminus \{\mathbf{0}\}, \quad \Pr_{\rho \leftarrow D_{\text{ZE}}} [\langle \text{ZE}(\rho), \mathbf{v} \rangle = 0] \leq \varepsilon_{\text{zero}}.$$

Lemma 3.5. Let $\mathbf{G} \in \mathbb{F}^{n \times k}$ be an injective linear map. Let $\hat{\mathbf{G}}$ be the multilinear extension of $\mathbf{G}$. The function $\text{ZE}: \mathbb{F}^{\log n} \rightarrow \mathbb{F}^k$ defined by

$$\text{ZE}(\alpha) := (\hat{\mathbf{G}}(\alpha, \mathbf{b}))_{\mathbf{b} \in \{0,1\}^{\log k}}$$

is a zero-evader with error $\frac{\log n}{|\mathbb{F}|}$.

Proof. For every $\alpha \in \mathbb{F}^{\log n}$ and $\mathbf{v} \in \mathbb{F}^k$,

$$\begin{aligned} \sum_{\mathbf{b} \in \{0,1\}^{\log k}} \hat{\mathbf{G}}(\alpha, \mathbf{b}) \cdot \mathbf{v}(\mathbf{b}) &= \sum_{\mathbf{b} \in \{0,1\}^{\log k}, \mathbf{b}' \in \{0,1\}^{\log n}} \text{eq}(\alpha, \mathbf{b}') \cdot \mathbf{G}(\mathbf{b}', \mathbf{b}) \cdot \mathbf{v}(\mathbf{b}) \\ &= \sum_{\mathbf{b}' \in \{0,1\}^{\log n}} \text{eq}(\alpha, \mathbf{b}') \cdot (\mathbf{G}\mathbf{v})(\mathbf{b}'). \end{aligned}$$

If $\mathbf{v} \neq \mathbf{0}$ then $\mathbf{G}\mathbf{v} \neq \mathbf{0}$, since $\mathbf{G}$ is linear and injective. Then, by Lemma 3.2:

$$\Pr_{\alpha} [\langle \text{ZE}(\alpha), \mathbf{v} \rangle = 0] = \Pr_{\alpha} \left[\sum_{\mathbf{b}' \in \{0,1\}^{\log n}} \text{eq}(\alpha, \mathbf{b}') \cdot (\mathbf{G}\mathbf{v})(\mathbf{b}') = 0 \right] \leq \frac{\log n}{|\mathbb{F}|}.$$

□

3.4 Codes

We state several definitions and facts about (error-correcting) codes.

Definition 3.6. A **code** over an alphabet Σ with message length k and codeword length n is an injective map $\mathcal{C}: \Sigma^k \rightarrow \Sigma^n$. A code $\mathcal{C}$ is a **linear code** if $\Sigma = \mathbb{F}$ is a field and $\mathcal{C}$ is linear. In this paper we assume that $n \in 2^{\mathbb{N}}$.

We often view $\mathcal{C} \subseteq \Sigma^n$ as the map's image and codewords $u \in \mathcal{C}$ as functions $u: [n] \rightarrow \Sigma$.

Definition 3.7. The **minimum distance** of a code $\mathcal{C}$ is defined as $\delta(\mathcal{C}) := \min_{u \neq v \in \mathcal{C}} \Delta(u, v)$.

Definition 3.8. A code $\mathcal{C}: \Sigma^k \rightarrow \Sigma^n$ is **systematic** if, for all $x \in \Sigma^k$, the first k entries of $\mathcal{C}(x)$ are equal to x .

Algorithmic measures. We consider several algorithmic measures associated to $\mathcal{C}$.

- $\text{enc}_{\mathcal{C}}$ is the number of field operations required to encode a message $x \in \Sigma^k$ into its corresponding codeword $u := \mathcal{C}(x)$.
- $\text{dec}_{\mathcal{C}}$ is the number of field operations required to decode a codeword $u \in \mathcal{C}$ into its corresponding message $x := \mathcal{C}^{-1}(u)$; if $\mathcal{C}$ is systematic, then $\text{dec}_{\mathcal{C}} = 0$ (since x is the first k elements of u).
- $\text{ecor}_{\mathcal{C}}$ is the number of field operations required to *erasure correct* a codeword (Definition 3.9); every linear code has efficient erasure correction (Lemma 3.10).

Definition 3.9. A code $\mathcal{C} \subseteq \Sigma^n$ supports **erasure correction with correction time** $\text{ecor}_{\mathcal{C}}$ if there exists a deterministic algorithm $\mathbf{E}_{\mathcal{C}}$ (the erasure corrector) that, given input $f: [n] \rightarrow \Sigma \cup \{\perp\}$:

- if $|f^{-1}(\perp)| < \delta(\mathcal{C}) \cdot n$ and there exists a codeword $u \in \mathcal{C}$ such that $f(i) = u(i)$ for all $i \in [n] \setminus f^{-1}(\perp)$, then $\mathbf{E}_{\mathcal{C}}(f) = u$ (note that such a codeword u is unique); and
- otherwise, $\mathbf{E}_{\mathcal{C}}(f) = \perp$.

Moreover, $\mathbf{E}_{\mathcal{C}}$ performs at most $\text{ecor}_{\mathcal{C}}$ field operations.

Lemma 3.10 ([GRS12]). Every linear code $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ has erasure correction with correction time

$$\text{ecor}_{\mathcal{C}} = O(n^3).$$

Definition 3.11. For a code $\mathcal{C} \subseteq \Sigma^n$, function $f: [n] \rightarrow \Sigma$, and proximity parameter $\delta \in [0, 1]$, we define the **list decoding** of f within distance δ as

$$\Lambda(\mathcal{C}, f, \delta) := \{u \in \mathcal{C} \mid \Delta(f, u) \leq \delta\}.$$

We also define $|\Lambda(\mathcal{C}, \delta)| := \max_f |\Lambda(\mathcal{C}, f, \delta)|$.

Definition 3.12. The **m -interleaving** of a code $\mathcal{C} \subseteq \Sigma^n$ is the code $\mathcal{C}^m \subseteq (\Sigma^m)^n$. The alphabet of $\mathcal{C}^m$ is Σ^m , so distance is measured with respect to this alphabet.

The following lemma states that if a code is list-decodable then so is its interleaving.

Lemma 3.13 ([GGR11]). Let $\mathcal{C}$ be a code and let $\delta \in [0, \delta(\mathcal{C})]$. Let $b := \lceil \frac{\delta}{\delta(\mathcal{C}) - \delta} \rceil$ and $r := \lceil \log \frac{\delta(\mathcal{C})}{\delta(\mathcal{C}) - \delta} \rceil$. For every $m \geq 1$, $|\Lambda(\mathcal{C}^m, \delta)| \leq \binom{b+r}{r} \cdot |\Lambda(\mathcal{C}, \delta)|^r$.

3.5 Out-of-domain samples

We describe a generalization of out-of-domain sampling for general linear codes.

Lemma 3.14. *Let $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ be a linear code, $f: [n] \rightarrow \mathbb{F}$ be a function, $s \in \mathbb{N}$ be a repetition parameter, and $\delta \in [0, 1]$ be a distance parameter. Let $\mathbf{ZE}: D_{\mathbf{ZE}} \rightarrow \mathbb{F}^k$ be a zero-evader with error $\varepsilon_{\text{zero}}$ (Definition 3.4). Then,*

$$\Pr_{\rho_1, \dots, \rho_s \leftarrow D_{\mathbf{ZE}}} \left[\begin{array}{l} \exists \text{ distinct } u, v \in \Lambda(\mathcal{C}, f, \delta) \\ \forall i \in [s], \langle \mathbf{ZE}(\rho_i), \mathcal{C}^{-1}(u) \rangle = \langle \mathbf{ZE}(\rho_i), \mathcal{C}^{-1}(v) \rangle \end{array} \right] \leq \frac{|\Lambda(\mathcal{C}, \delta)|^2}{2} \cdot \varepsilon_{\text{zero}}^s.$$

Proof. Fix two distinct codewords $u, v \in \Lambda(\mathcal{C}, f, \delta)$. Since $u \neq v$ and $\mathcal{C}$ is injective, it must be that $\mathcal{C}^{-1}(u) \neq \mathcal{C}^{-1}(v)$. It follows that $\Pr_{\rho \leftarrow D_{\mathbf{ZE}}} [\langle \mathbf{ZE}(\rho), \mathcal{C}^{-1}(u) \rangle = \langle \mathbf{ZE}(\rho), \mathcal{C}^{-1}(v) \rangle] \leq \varepsilon_{\text{zero}}$. Hence the probability that $\langle \mathbf{ZE}(\rho_i), \mathcal{C}^{-1}(u) \rangle = \langle \mathbf{ZE}(\rho_i), \mathcal{C}^{-1}(v) \rangle$ for every $i \in [s]$ is at most $\varepsilon_{\text{zero}}^s$ (since the seeds $\rho_1, \dots, \rho_s$ are sampled independently). The lemma follows from a union bound over the $\binom{|\Lambda(\mathcal{C}, \delta, f)|}{2} \leq \frac{|\Lambda(\mathcal{C}, \delta)|^2}{2}$ pairs of distinct codewords in $\Lambda(\mathcal{C}, \delta, f)$. $\square$

Lemma 3.14 implies a corresponding statement for Reed–Solomon codes, in particular [ACFY24, Lemma 4.5] in STIR and [ACFY25, Lemma 4.25] in WHIR.

3.6 Mutual correlated agreement for proximity generators

We rely on *mutual correlated agreement*, a strengthening of correlated agreement for proximity generators introduced in [ACFY25], which for simplicity we refer to as *strong* proximity generators. These ensure that, with high probability, the agreement domain of the function resulting from the proximity generator is identical to the set of correlated agreement domains between the functions.

Definition 3.15 ([ACFY25]). *Let $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ be a linear code. We say that $\text{PG}(\ell, \cdot): D_{\text{PG}} \rightarrow \mathbb{F}^\ell$ is a **strong proximity generator** for $\mathcal{C}$ with proximity radius δ_{PG} and error err_{PG} if, for all $f_1 \dots, f_\ell: [n] \rightarrow \mathbb{F}$ and $\delta \in (0, \delta_{\text{PG}}(\ell))$, the following holds:*

$$\Pr_{\substack{\alpha \leftarrow D_{\text{PG}} \\ \gamma := \text{PG}(\ell, \alpha)}} \left[\begin{array}{l} \exists S \subseteq [n] \text{ s.t.} \\ |S| \geq (1 - \delta) \cdot n \\ \wedge \exists u \in \mathcal{C}, u(S) = \sum_{j \in [\ell]} \gamma_j \cdot f_j(S) \\ \wedge \exists i \in [\ell], \forall v \in \mathcal{C}, v(S) \neq f_i(S) \end{array} \right] \leq \text{err}_{\text{PG}}(\mathcal{C}, \ell, \delta).$$

4 Round-by-round knowledge soundness for IORs

We define our variant of round-by-round knowledge soundness for IORs.

Definition 4.1. Let $\text{IOR} = (\mathbf{I}, \mathbf{P}, \mathbf{V})$ be an IOR from a relation $\mathcal{R}$ to $\mathcal{R}'$ with proximity radius δ^* . A **knowledge state function** for IOR is a (possibly inefficient) function KState , parameterized by a proximity bound $\delta \in (0, \delta^*)$, that, on input a statement $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y})$, interaction transcript tr , and **knowledge state witness** $\mathbf{w}$, outputs a bit and has the following properties.

- *Empty transcript:* if $\text{tr} = \emptyset$ is the empty transcript, then $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, \mathbf{w}) = 1$ if and only if there exists $\mathfrak{y}_\star$ satisfying $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}_\star, \mathbf{w}) \in \mathcal{R}$ and $\Delta(\pi_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}), \pi_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}_\star)) \leq \delta$.
- *Prover moves:* if tr is a transcript where the prover is about to move and $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, \mathbf{w}) = 0$, then $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \pi, \mathbf{w}) = 0$ for every prover message π .
- *Full transcript:* if $\text{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $\mathbf{V}^{\mathfrak{y}, \pi_1, \dots, \pi_k}(\mathbf{I}(\mathfrak{i}), \mathfrak{x}, \rho_1, \dots, \rho_k)$ outputs $(\mathfrak{x}', \mathfrak{y}')$ such that there exists $\mathfrak{y}'_\star$ satisfying $(\mathfrak{i}, \mathfrak{x}', \mathfrak{y}'_\star, \mathbf{w}) \in \mathcal{R}'$ and $\Delta(\pi_{\mathcal{R}'}(\mathfrak{i}, \mathfrak{x}', \mathfrak{y}'), \pi_{\mathcal{R}'}(\mathfrak{i}, \mathfrak{x}', \mathfrak{y}'_\star)) \leq \delta$.

Definition 4.2. $\text{IOR} = (\mathbf{I}, \mathbf{P}, \mathbf{V})$ from a relation $\mathcal{R}$ to $\mathcal{R}'$ with proximity radius δ^* has **round-by-round knowledge soundness with errors** $(\varepsilon_1, \dots, \varepsilon_k)$ and **extraction times** $(\text{et}_1, \dots, \text{et}_k)$ if there exist a knowledge state function KState for IOR and a deterministic extractor $\mathbf{E}_{\text{rbr}}$ with the following property: for every proximity bound $\delta \in (0, \delta^*)$, statement $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y})$, round index $i \in [k]$, and interaction transcript $\text{tr} = (\pi_1, \rho_1, \dots, \pi_{i-1}, \rho_{i-1}, \pi_i)$ where the verifier is about to move, $\mathbf{E}_{\text{rbr}}$ runs in time at most et_i and

$$\Pr_{\rho_i} \left[\exists \mathbf{w} : \begin{array}{l} \text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \rho_i, \mathbf{w})) = 0 \\ \wedge \text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \rho_i, \mathbf{w}) = 1 \end{array} \right] \leq \varepsilon_i(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \delta).$$

The **total extraction time** is defined to $\sum_{i \in [k]} \text{et}_i$.

In Appendix B we prove that (our variant of) round-by-round knowledge soundness implies straightline state-restoration knowledge soundness.

Remark 4.3. Similar to the soundness property given in Section 3.2, Definitions 4.1 and 4.2 are for a distance-preserving IOR. In full generality, as discussed in Remark 3.3, an IOR may be specified with a new proximity bound δ' for $\mathcal{R}'$. In this case, in Definition 4.1 the knowledge state function would use δ and δ' in the empty and full transcript requirements, respectively; Definition 4.2 would be updated accordingly.

5 Polynomial equation satisfiability

We describe *polynomial equations satisfiability* (PESAT), the NP-complete problem that we use.

Definition 5.1. A **system of polynomial equations** over the finite field $\mathbb{F}$ is a triple $(\hat{\mathbf{p}}, M, N)$ where $\hat{\mathbf{p}} = (\hat{p}_i)_{i \in [M]}$ is a list of M polynomials in N variables over $\mathbb{F}$. The **degree** of the system is the maximum total degree of any individual polynomial: $d := \max_i \deg_{\text{tot}}(\hat{p}_i)$. We denote by $|\hat{\mathbf{p}}|$ the number of field operations needed to evaluate the polynomial system on a vector $\mathbf{z} \in \mathbb{F}^N$.

Definition 5.2. For a field $\mathbb{F}$, the **polynomial equations satisfiability (PESAT) relation** $\mathcal{R}_{\odot}(\mathbb{F})$ is defined as

$$\mathcal{R}_{\odot}(\mathbb{F}) := \left\{ (\mathfrak{i}, \mathfrak{x}, \perp, \mathfrak{w}) \left| \begin{array}{l} \mathfrak{i} = (\hat{\mathbf{p}}, M, N, k) \\ \mathfrak{x} = \mathbf{x} \in \mathbb{F}^{N-k} \\ \mathfrak{w} = \mathbf{w} \in \mathbb{F}^k \\ \hat{\mathbf{p}}(\mathbf{z}) = \mathbf{0} \text{ where } \mathbf{z} := (\mathbf{x}, \mathbf{w}) \end{array} \right. \right\}.$$

We assume that $M \in 2^{\mathbb{N}}$. We write $\kappa := N - k$ to denote the length of $\mathfrak{x} = \mathbf{x}$.

Throughout, when we write $\mathfrak{i}$ we refer to the index of the PESAT relation: $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$. All the IORs that we consider have indexer $\mathbf{I}(\mathfrak{i}) = (M, N, k)$, so we omit this in protocol description.

Remark 5.3 (R1CS). We outline how PESAT over $\mathbb{F}$ captures R1CS [BCGTV13], which consists of triples $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = ((\mathbf{A}, \mathbf{B}, \mathbf{C}, k), \mathbf{x}, \mathbf{w})$ where $\mathbf{A}, \mathbf{B}, \mathbf{C}$ are $M \times N$ matrices over $\mathbb{F}$, $\mathbf{x} \in \mathbb{F}^{N-k}$, and $\mathbf{w} \in \mathbb{F}^k$ such that $\mathbf{A}\mathbf{z} \circ \mathbf{B}\mathbf{z} = \mathbf{C}\mathbf{z}$ for $\mathbf{z} := (\mathbf{x}, \mathbf{w})$; here $\circ$ denotes the Hadamard (entry-wise) product. R1CS is a degree-2 PESAT: for every $i \in [M]$, we define $\hat{p}_i(\mathbf{z}) := \langle \mathbf{a}_i, \mathbf{z} \rangle \cdot \langle \mathbf{b}_i, \mathbf{z} \rangle - \langle \mathbf{c}_i, \mathbf{z} \rangle$ where $\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i$ are the i -th rows of $\mathbf{A}, \mathbf{B}, \mathbf{C}$ respectively. Note that $|\hat{\mathbf{p}}| = O(S)$ where S is a bound on the number of non-zero entries in each of $\mathbf{A}, \mathbf{B}, \mathbf{C}$.

Remark 5.4 (CCS). We outline how PESAT over $\mathbb{F}$ captures CCS [STW23], which consists of triples $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = ((\mathbf{M}_1, \dots, \mathbf{M}_t, S_1, \dots, S_q, c_1, \dots, c_q, k), \mathbf{x}, \mathbf{w})$ where $\mathbf{M}_1, \dots, \mathbf{M}_t$ are $M \times N$ matrices over $\mathbb{F}$, $S_1, \dots, S_q$ are multisets in $[t]$ with $|S_i| \leq d$, $c_1, \dots, c_q \in \mathbb{F}$, $\mathbf{x} \in \mathbb{F}^{N-k}$, and $\mathbf{w} \in \mathbb{F}^k$ such that $\sum_{j \in [q]} c_j \cdot \bigcirc_{a \in S_j} \mathbf{M}_a \mathbf{z} = \mathbf{0}$ for $\mathbf{z} := (\mathbf{x}, \mathbf{w})$. CCS is a degree- d PESAT: for every $i \in [M]$, we define $\hat{p}_i(\mathbf{z}) := \sum_{j \in [q]} c_j \cdot \prod_{a \in S_j} \langle \mathbf{m}_{a,i}, \mathbf{z} \rangle$, where $\mathbf{m}_{a,i}$ is the i -th row of $\mathbf{M}_a$.

We work with a bundled form of PESAT where the M constraints are reduced to a single one.

Definition 5.5. Let $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$ and let $\mathbf{ZE}: D_{\mathbf{ZE}} \rightarrow \mathbb{F}^M$ be a zero-evader (Definition 3.4). The **ZE-bundling** of $\hat{\mathbf{p}}$ is the function $\hat{P}[\mathbf{ZE}, \hat{\mathbf{p}}]: D_{\mathbf{ZE}} \times \mathbb{F}^N \rightarrow \mathbb{F}$ defined as

$$\hat{P}[\mathbf{ZE}, \hat{\mathbf{p}}](\rho, \mathbf{z}) := \langle \mathbf{ZE}(\rho), \hat{\mathbf{p}}(\mathbf{z}) \rangle.$$

In this paper, we fix $\mathbf{ZE}: \mathbb{F}^{\log M} \rightarrow \mathbb{F}^M$ where $\tau \mapsto (\text{eq}(\tau, \mathbf{i}))_{\mathbf{i} \in \{0,1\}^{\log M}}$, in which case $\hat{P}[\mathbf{ZE}, \hat{\mathbf{p}}]$ is the polynomial

$$\hat{P}(\tau, \mathbf{z}) := \sum_{\mathbf{i} \in \{0,1\}^{\log M}} \text{eq}(\tau, \mathbf{i}) \cdot \hat{p}_{\mathbf{i}}(\mathbf{z}).$$

We often write $\hat{P}$ instead of $\hat{P}[\mathbf{ZE}, \hat{\mathbf{p}}]$ when $\hat{\mathbf{p}}$ is clear from context (and $\mathbf{ZE}$ is fixed as above).

We define notation for codewords on which evaluation and bundled constraints are enforced.

Definition 5.6. Let $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ be a linear code and $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$. For an evaluation point $\alpha \in \mathbb{F}^{\log n}$, circuit evaluation point $\beta \in \mathbb{F}^{\log M + \kappa}$, and targets $\mu, \eta \in \mathbb{F}$, we define

$$\mathcal{C}[(\alpha, \mu), (\hat{P}, \beta, \eta)] = \left\{ u \in \mathcal{C} : \begin{array}{l} \hat{u}(\alpha) = \mu \\ \wedge \hat{P}(\beta, \mathcal{C}^{-1}(u)) = \eta \end{array} \right\}.$$

Moreover, we define a multi-constraint version as

$$\mathcal{C}[(\alpha_i, \mu_i)_{i \in [r]}, (\hat{P}, \beta, \eta)] = \bigcap_{i \in [r]} \mathcal{C}[(\alpha_i, \mu_i), (\hat{P}, \beta, \eta)].$$

We write $m := \log M + \kappa$ to denote the length of β .

We use the bundled form to define the main relation that our protocols accumulate. The relation is defined with respect to a linear code which is fixed throughout this work.¹⁰

Definition 5.7. Let $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ be a linear code and $r \in \mathbb{N}$. The **multi-evaluation relation** $\mathcal{R}_{c,r}$ is

$$\mathcal{R}_{c,r} = \left\{ (\mathfrak{i}, ((\alpha_i, \mu_i)_{i \in [r]}, \beta, \eta), u, u) \mid \begin{array}{l} \mathfrak{i} = (\hat{\mathbf{p}}, M, N, k) \\ u \in \mathcal{C}[(\alpha_i, \mu_i)_{i \in [r]}, (\hat{P}, \beta, \eta)] \end{array} \right\}.$$

The **twin constraint relation** is $\mathcal{R}_c := \mathcal{R}_{c,1}$.

We present an IOR that reduces an arbitrary PESAT relation to (a simpler version of) the main relation.

Theorem 5.8. Fix a linear code $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$. Construction 5.10 is a public-coin IOR from $\mathcal{R}_{\odot}(\mathbb{F})$ (Definition 5.2) to $\mathcal{R}_{c,0}$ (Definition 5.7) with the following parameters.

- Rounds. The protocol has 1 round.
- Proof length. The prover sends a single message consisting of n field elements.
- Query complexity. The verifier makes no queries.
- Prover time. The prover performs $O(\text{enc}_{\mathcal{C}})$ field operations.
- Verifier time. The verifier performs no field operations.

For every $\delta \in (0, 1)$, if

$$|\mathbb{F}| \geq 2^\lambda \cdot \log M \cdot |\Lambda(\mathcal{C}, \delta)|, \tag{1}$$

the IOR has round-by-round knowledge soundness error $\leq 2^{-\lambda}$. Moreover, the total extraction time is $O(\text{dec}_{\mathcal{C}})$ field operations.

In Section 5.1 we describe the protocol and in Section 5.2 we prove its security.

Remark 5.9. Theorem 5.8 provides an IOR from $\mathcal{R}_{\odot}(\mathbb{F})$ to $\mathcal{R}_{c,0}$. Later, Theorem 6.2 provides an IOR for $\mathcal{R}_c = \mathcal{R}_{c,1} \neq \mathcal{R}_{c,0}$. This mismatch can be resolved in a few ways.

- Construction 6.3 can be adapted to handle $\mathcal{R}_{c,0}$ straightforwardly; we do not explicitly perform the adaptation, as it complicates notation.
- The prover can send an evaluation of $\hat{f}$ at an arbitrary point (say $\mathbf{0}$, this does not have to be chosen at random).

These do not affect the asymptotics of our constructions, nor significantly the efficiency, so we ignore them for the purpose of this work.

¹⁰In full generality, we fix a family of linear codes parameterized by the message length k .

5.1 Protocol

Construction 5.10. We describe an IOR from $\mathcal{R}_\otimes(\mathbb{F})$ (Definition 5.2) to $\mathcal{R}_{C,0}$ (Definition 5.7).

- **Inputs.** The verifier receives the explicit instance $\mathfrak{x} := \mathbf{x} \in \mathbb{F}^\kappa$. In the honest case, the prover additionally receives a witness $\mathfrak{w} := \mathbf{w} \in \mathbb{F}^k$ such that $(\mathfrak{i}, \mathbf{x}, \perp, \mathbf{w}) \in \mathcal{R}_\otimes(\mathbb{F})$.
- **Interaction phase.**
 1. **Encoded witness:** The prover sends $f: [n] \rightarrow \mathbb{F}$. In the honest case, $f := \mathcal{C}(\mathbf{w})$.
 2. **Zerocheck randomness:** The verifier samples $\boldsymbol{\tau} \leftarrow \mathbb{F}^{\log M}$.
- **Decision phase.** Define $\boldsymbol{\beta} := (\boldsymbol{\tau}, \mathbf{x})$. The verifier outputs the explicit instance $\mathfrak{x}' := (\boldsymbol{\beta}, 0)$ and implicit instance $\mathfrak{y}' := f$. In the honest case, the prover outputs the new witness $\mathfrak{w}' := f \in \mathcal{C}$.

5.2 Security

Lemma 5.11. *Construction 5.10 is an IOR from $\mathcal{R}_\otimes(\mathbb{F})$ to $\mathcal{R}_{C,0}$ with round-by-round knowledge soundness error ε_{zc} where:*

- $\varepsilon_{zc} \leq \frac{|\Lambda(\mathcal{C}, \delta)| \cdot \log M}{|\mathbb{F}|}$.

Moreover, the total extraction time is $O(\text{dec}_C)$ field operations.

Proof. First we define the knowledge state function and extractor; then we upper bound the round-by-round knowledge soundness error. Throughout, $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$, $\mathfrak{x} = \mathbf{x}$, and $\mathfrak{y} = \perp$.

Knowledge state function and extractor. We define the knowledge state function and extractor's behavior in each round:

0. **Initial transcript:** We set $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \emptyset, \mathbf{w}) = 1$ if and only if $\mathbf{w} \in \mathbb{F}^k$ and $\hat{\mathbf{p}}(\mathbf{x}, \mathbf{w}) = \mathbf{0}$.
1. **Zerocheck randomness:** At this stage the transcript tr has the form $\text{tr} = (f)$. The verifier samples $\boldsymbol{\tau} \leftarrow \mathbb{F}^{\log \ell}$.

Knowledge state function: We set $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \boldsymbol{\tau}, u) = 1$ if and only if $u \in \Lambda(\mathcal{C}, f, \delta)$ and

$$\hat{P}(\boldsymbol{\tau}, \mathbf{x}, \mathcal{C}^{-1}(u)) = 0.$$

This corresponds to whether the verifier's output is δ -close to $\mathcal{R}_{C,0}$.

Extractor: The extractor is $\mathbf{E}_{\text{rbr}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \boldsymbol{\tau}, u) := \mathcal{C}^{-1}(u)$ and runs in time $\text{et}_1 = O(\text{dec}_C)$.

RBR knowledge soundness. We upper bound the round-by-round knowledge soundness errors based on the knowledge state function and extractor described above.

1. **Zerocheck randomness:** We show that

$$\Pr_{\boldsymbol{\tau}} \left[\exists u : \begin{array}{l} \text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, \mathcal{C}^{-1}(u)) = 0 \\ \text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \boldsymbol{\tau}, u) = 1 \end{array} \right] \leq \frac{|\Lambda(\mathcal{C}, \delta)| \cdot \log M}{|\mathbb{F}|}.$$

Fix $u \in \Lambda(\mathcal{C}, f, \delta)$ and assume that $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, \mathcal{C}^{-1}(u)) = 0$. We show that

$$\Pr_{\boldsymbol{\tau}} [\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \boldsymbol{\tau}, u) = 1] \leq \frac{\log M}{|\mathbb{F}|}. \quad (2)$$

The error bound follows from a union bound over the codewords in $\Lambda(\mathcal{C}, f, \delta)$, as these are the only choices of u which allow $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \boldsymbol{\tau}, u) = 1$ for some choice of $\boldsymbol{\tau}$. Since $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, \mathcal{C}^{-1}(u)) = 0$, it must be that $\hat{\boldsymbol{p}}(\boldsymbol{x}, \mathcal{C}^{-1}(u)) \neq \mathbf{0}$. Equation 2's bound follows from the definition of $\hat{P}$ as the ZE-bundling of $\hat{\boldsymbol{p}}$, where ZE is a zero-evader with error $\frac{\log M}{|\mathbb{F}|}$.

□

6 Twin constraint pseudo-batching

Definition 6.1. Let $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ be a linear-code and $\ell, r \in \mathbb{N}$. The **pseudo-batching accumulation relation** $\mathcal{R}_{\mathcal{C}, r, \ell}$ is

$$\mathcal{R}_{\mathcal{C}, r, \ell} = \left\{ (\mathfrak{i}, (\gamma, (\alpha_i, \mu_i)_{i \in [r]}, \beta, \eta), (u_1, \dots, u_\ell), u) \mid \begin{array}{l} \mathfrak{i} = (\hat{\mathbf{p}}, M, N, k) \\ u = \sum_{i \in [\ell]} \gamma_i \cdot u_i \in \mathcal{C}[(\alpha_i, \mu_i)_{i \in [r]}, (\hat{P}, \beta, \eta)] \end{array} \right\}.$$

For any statement $(\mathfrak{i}, \mathbf{x} = (\gamma, \dots), \mathbf{y} = (f_1, \dots, f_\ell))$ (not necessarily in the language), the associated oracle is defined to be $\pi_{\mathcal{R}_{\mathcal{C}, r, \ell}}(\mathfrak{i}, \mathbf{x}, \mathbf{y}) := \sum_{i \in [\ell]} \gamma_i \cdot f_i$.

Theorem 6.2. Fix a linear code $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ and parameter $\ell \in 2^{\mathbb{N}}$, and let $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$ be of degree d . Construction 6.3 is a public-coin IOR from $\mathcal{R}_{\mathcal{C}}^\ell$ (Definition 5.7) to $\mathcal{R}_{\mathcal{C}, 1, \ell}$ (Definition 6.1) with the following parameters.

- **Rounds.** The protocol has $\log \ell + 1$ rounds.
- **Proof length.** The prover sends $\log \ell \cdot (1 + \max\{\log n + 1, \log M + d\}) + 2$ field elements.
- **Query complexity.** The verifier makes no queries.
- **Prover time.** The prover performs $O(\ell \cdot (n + d \cdot (|\hat{\mathbf{p}}| + M)))$ field operations (see Lemma 6.4).
- **Verifier time.** The verifier performs $O(\ell \cdot (\log n + \log M + \kappa) + \log \ell \cdot d)$ field operations.

Suppose that $\text{PG}(2, \alpha) = (1 - \alpha, \alpha)$ is a strong proximity generator for $\mathcal{C}$ with proximity radius δ_{PG} and error err_{PG} . For every $\delta \in (0, \min\{\delta(\mathcal{C}), \delta_{\text{PG}}(2)\})$, as long as

$$|\mathbb{F}| \geq 2^\lambda \cdot \max \left\{ \begin{array}{l} (\log \ell + 1) \cdot |\Lambda(\mathcal{C}^\ell, \delta)|, \\ (1 + \max\{\log n + 1, \log M + d\}) \cdot |\Lambda(\mathcal{C}^\ell, \delta)| + \ell/2 \cdot \text{err}_{\text{PG}}(\mathcal{C}, 2, \delta) \cdot |\mathbb{F}| \end{array} \right\},$$

the IOR has round-by-round knowledge soundness error $\leq 2^{-\lambda}$. Moreover, the total extraction time is $O(\ell \cdot \text{ecor}_{\mathcal{C}})$ field operations.

In Section 6.1 we describe the protocol and in Section 6.2 we prove its security.

6.1 Protocol

Construction 6.3. We describe an IOR from $\mathcal{R}_{\mathcal{C}}^\ell$ (Definition 5.7) to $\mathcal{R}_{\mathcal{C}, 1, \ell}$ (Definition 6.1).

- **Inputs.** For $\mathbf{i} \in \{0, 1\}^{\log \ell}$, the verifier receives the explicit instance $\mathbf{x}_{\mathbf{i}} = (\alpha_{\mathbf{i}}, \mu_{\mathbf{i}}, \beta_{\mathbf{i}}, \eta_{\mathbf{i}})$ and oracle access to the implicit instance $\mathbf{y}_{\mathbf{i}} := f_{\mathbf{i}}: [n] \rightarrow \mathbb{F}$. In the honest case, $(\mathfrak{i}, \mathbf{x}_{\mathbf{i}}, \mathbf{y}_{\mathbf{i}}) \in \mathcal{L}(\mathcal{R}_{\mathcal{C}})$ for every $\mathbf{i} \in \{0, 1\}^{\log \ell}$.
- **Interaction phase.**
 1. **Zerocheck randomness:** The verifier samples $\tau \leftarrow \mathbb{F}^{\log \ell}$ and $\xi \leftarrow \mathbb{F}$.

2. **Sumcheck protocol:** Define the following $(\log \ell)$ -variate multilinear polynomials:

$$\begin{aligned}\hat{\mathbf{F}}(\mathbf{I}) &:= \sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot f_{\mathbf{i}} && \in (\mathbb{F}^{<2}[\mathbf{I}])^n, \\ \hat{\mathbf{w}}(\mathbf{I}) &:= \sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot \mathcal{C}^{-1}(f_{\mathbf{i}}) && \in (\mathbb{F}^{<2}[\mathbf{I}])^k, \\ \hat{\mathbf{A}}(\mathbf{I}) &:= \sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot \boldsymbol{\alpha}_{\mathbf{i}} && \in (\mathbb{F}^{<2}[\mathbf{I}])^{\log n}, \\ \hat{\mathbf{B}}(\mathbf{I}) &:= \sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot \beta_{\mathbf{i}} && \in (\mathbb{F}^{<2}[\mathbf{I}])^m.\end{aligned}$$

Note that $\hat{\mathbf{w}}$ is well-defined in the honest case and, assuming the code is systematic, equivalent to the first k entries of $\hat{\mathbf{F}}$. Define the following:

$$\begin{aligned}\hat{G}(\mathbf{F}, \mathbf{W}, \mathbf{A}, \mathbf{B}) &:= \left(\sum_{\mathbf{b} \in \{0,1\}^{\log n}} \text{eq}(\mathbf{A}, \mathbf{b}) \cdot F_{\mathbf{b}} \right) + \xi \cdot \hat{P}(\mathbf{B}, \mathbf{W}), \\ \sigma &:= \sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\boldsymbol{\tau}, \mathbf{i}) \cdot (\mu_{\mathbf{i}} + \xi \cdot \eta_{\mathbf{i}}).\end{aligned}$$

The prover and verifier engage in a $(\log \ell)$ -round sumcheck protocol for the following claim:

$$\sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\boldsymbol{\tau}, \mathbf{i}) \cdot \hat{G}(\hat{\mathbf{F}}(\mathbf{i}), \hat{\mathbf{w}}(\mathbf{i}), \hat{\mathbf{A}}(\mathbf{i}), \hat{\mathbf{B}}(\mathbf{i})) = \sigma.$$

During the interaction, the prover sends $\log \ell$ sumcheck polynomials $\hat{h}_1, \dots, \hat{h}_{\log \ell}$ of degree $1 + \max\{\log n + 1, \log M + d\}$, and the verifier replies with randomness $\boldsymbol{\gamma} := (\gamma_1, \dots, \gamma_{\log \ell})$.

3. **New target:** The prover sends $\mu, \eta \in \mathbb{F}$. In the honest case:

$$\begin{aligned}\mu &:= \hat{g}(\hat{\mathbf{A}}(\boldsymbol{\gamma})), \\ \eta &:= \hat{P}(\hat{\mathbf{B}}(\boldsymbol{\gamma}), \mathcal{C}^{-1}(f)) \quad \text{where} \quad f := \sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\boldsymbol{\gamma}, \mathbf{i}) \cdot f_{\mathbf{i}}.\end{aligned}$$

• **Decision phase.** (The verifier rejects if any check fails.)

1. **Sumcheck decision:** The verifier checks that

$$\hat{h}_1(0) + \hat{h}_1(1) = \sigma \quad \text{and} \quad \forall j \in \{1, \dots, \log \ell - 1\}, \quad \hat{h}_{j+1}(0) + \hat{h}_{j+1}(1) = \hat{h}_j(\gamma_j).$$

2. **New target decision:** The verifier checks that $\hat{h}_{\log \ell}(\gamma_{\log \ell}) = \text{eq}(\boldsymbol{\tau}, \boldsymbol{\gamma}) \cdot (\mu + \xi \cdot \eta)$.

3. **Outputs:** Set $\boldsymbol{\gamma}' := (\text{eq}(\boldsymbol{\gamma}, \mathbf{i}))_{\mathbf{i} \in \{0,1\}^{\log \ell}}$. The verifier outputs the new explicit instance $\mathbf{x}' := (\boldsymbol{\gamma}', \hat{\mathbf{A}}(\boldsymbol{\gamma}), \mu, \hat{\mathbf{B}}(\boldsymbol{\gamma}), \eta)$ and implicit instance $\mathbf{y}' := (f_1, \dots, f_{\ell})$.

We analyze the prover time of Construction 6.3; other complexity parameters are clear.

Lemma 6.4. *The prover in Construction 6.3 can be implemented to perform*

$$O(\ell \cdot (n + d \cdot (|\hat{\mathbf{p}}| + M)))$$

field operations.

Proof. Let $j \in [\log \ell]$. In the j -th sumcheck round, let $\gamma_{j-1} := (\gamma_1, \dots, \gamma_{j-1}) \in \mathbb{F}^{j-1}$. The prover maintains five tables: $T_{j,F} \in \mathbb{F}^{(\ell/2^{j-1}) \cdot n}$, $T_{j,w} \in \mathbb{F}^{(\ell/2^{j-1}) \cdot k}$, $T_{j,A} \in \mathbb{F}^{(\ell/2^{j-1}) \cdot \log n}$, $T_{j,B} \in \mathbb{F}^{(\ell/2^{j-1}) \cdot m}$, and $T_{j,\text{eq}} \in \mathbb{F}^{\ell/2^{j-1}}$. The table entries are defined as follows: for every $\mathbf{i} \in \{0, 1\}^{\log \ell - j + 1}$,

$$\begin{aligned} T_{j,F}[\mathbf{i}, \cdot] &= \hat{\mathbf{F}}(\gamma_{j-1}, \mathbf{i}) && \in \mathbb{F}^n, \\ T_{j,w}[\mathbf{i}, \cdot] &= \hat{\mathbf{w}}(\gamma_{j-1}, \mathbf{i}) && \in \mathbb{F}^k, \\ T_{j,A}[\mathbf{i}, \cdot] &= \hat{\mathbf{A}}(\gamma_{j-1}, \mathbf{i}) && \in \mathbb{F}^{\log n}, \\ T_{j,B}[\mathbf{i}, \cdot] &= \hat{\mathbf{B}}(\gamma_{j-1}, \mathbf{i}) && \in \mathbb{F}^m, \\ T_{j,\text{eq}}[\mathbf{i}] &= \text{eq}(\tau, \gamma_{j-1}, \mathbf{i}) && \in \mathbb{F}. \end{aligned}$$

Using [VSBW13], these tables can be initialized and maintained in time that is linear in their initial size (during the entire execution of the protocol). The total number of field operations is

$$O(\ell \cdot (n + k + \log n + m + 1)) = O(\ell \cdot (n + N + \log M)).$$

Next, we show how these tables can be used to compute the sumcheck polynomials in linear time. For every $\mathbf{i} \in \{0, 1\}^{\log \ell - j}$, define $\hat{\mathbf{F}}_{j,\mathbf{i}}(\mathbf{X}) := \hat{\mathbf{F}}(\gamma_{j-1}, \mathbf{X}, \mathbf{i})$; define $\hat{\mathbf{w}}_{j,\mathbf{i}}$, $\hat{\mathbf{A}}_{j,\mathbf{i}}$, $\hat{\mathbf{B}}_{j,\mathbf{i}}$ analogously. The prover must compute the following sumcheck polynomial:

$$\hat{h}_j(\mathbf{X}) := \sum_{\mathbf{i} \in \{0, 1\}^{\log \ell - j}} \text{eq}(\tau, (\gamma_{j-1}, \mathbf{X}, \mathbf{i})) \cdot \hat{G}(\hat{\mathbf{F}}_{j,\mathbf{i}}(\mathbf{X}), \hat{\mathbf{w}}_{j,\mathbf{i}}(\mathbf{X}), \hat{\mathbf{A}}_{j,\mathbf{i}}(\mathbf{X}), \hat{\mathbf{B}}_{j,\mathbf{i}}(\mathbf{X})).$$

Fix $\mathbf{i} \in \{0, 1\}^{\log \ell - j}$. Observe that $\text{eq}(\tau, (\gamma_{j-1}, \mathbf{X}, \mathbf{i}))$ can be obtained from $T_{j,\text{eq}}$ in $O(1)$ time. We are left to show that $\hat{G}(\hat{\mathbf{F}}_{j,\mathbf{i}}(\mathbf{X}), \hat{\mathbf{w}}_{j,\mathbf{i}}(\mathbf{X}), \hat{\mathbf{A}}_{j,\mathbf{i}}(\mathbf{X}), \hat{\mathbf{B}}_{j,\mathbf{i}}(\mathbf{X}))$ can be efficiently computed; unwrapping the definition of $\hat{G}$, this is

$$\left(\sum_{\mathbf{b} \in \{0, 1\}^{\log n}} \text{eq}(\hat{\mathbf{A}}_{j,\mathbf{i}}(\mathbf{X}, \mathbf{b})) \cdot \hat{\mathbf{F}}_{j,\mathbf{i},\mathbf{b}}(\mathbf{X}) \right) + \xi \cdot \hat{P}(\hat{\mathbf{B}}_{j,\mathbf{i}}(\mathbf{X}), \hat{\mathbf{w}}_{j,\mathbf{i}}(\mathbf{X})).$$

We handle the terms separately, in both cases reducing to a strategy described in [EG23] and formally given in Claim 6.5.

Claim 6.5. *Let $\hat{\mathbf{c}}(\mathbf{X}) := (\hat{c}_1(\mathbf{X}), \dots, \hat{c}_m(\mathbf{X}))$ be a vector of linear univariate polynomials and $\hat{\mathbf{q}}(\mathbf{X}) := (\hat{q}_b(\mathbf{X}))_{b \in \{0, 1\}^m}$ be a list of univariate polynomials of degree at most d . Given $\hat{\mathbf{c}}$ and $\hat{\mathbf{q}}$ one can compute the univariate polynomial*

$$\sum_{\mathbf{b} \in \{0, 1\}^m} \text{eq}(\hat{\mathbf{c}}(\mathbf{X}), \mathbf{b}) \cdot \hat{\mathbf{q}}_{\mathbf{b}}(\mathbf{X}) \tag{3}$$

in $O(2^m \cdot d)$ field operations. Note that the above polynomial has degree at most $d + m$.

Proof. For every $\mathbf{b} \in \{0, 1\}^{m-1}$, define the degree- $(d+1)$ polynomial

$$\hat{q}_{\mathbf{b}}^{(1)} := \hat{q}_{0,\mathbf{b}}(\mathbf{X}) \cdot (1 - \hat{c}_1(\mathbf{X})) + \hat{q}_{1,\mathbf{b}}(\mathbf{X}) \cdot \hat{c}_1(\mathbf{X}).$$

The polynomial in Equation 3 is equivalent to

$$\sum_{\mathbf{b} \in \{0, 1\}^{m-1}} \text{eq}(\hat{c}_2(\mathbf{X}), \dots, \hat{c}_m(\mathbf{X}), \mathbf{b}) \cdot \hat{q}_{\mathbf{b}}^{(1)}(\mathbf{X}),$$

By recursively applying this procedure, we can efficiently obtain the polynomial. In more detail, let $\hat{\mathbf{q}}^{(0)} := \hat{\mathbf{q}}$. For $i \in [m]$, we compute, for every $\mathbf{b} \in \{0, 1\}^{m-i}$, the degree- $(d+i)$ polynomial

$$\hat{q}_{\mathbf{b}}^{(i)} := \hat{q}_{0,\mathbf{b}}^{(i-1)}(\mathbf{x}) \cdot (1 - \hat{c}_{i-1}(\mathbf{x})) + \hat{q}_{1,\mathbf{b}}^{(i-1)}(\mathbf{x}) \cdot \hat{c}_{i-1}(\mathbf{x}).$$

Note that $\hat{q}_{\mathbf{b}}^{(i)}$ can be computed from $\hat{\mathbf{q}}^{(i-1)}$ in $O(d+i)$ field operations. Subsequently, the polynomial in Equation 3 is equivalent to $\hat{q}^{(\ell)}$. The overall time (measured in field operations) is

$$\sum_{i=1}^m O(2^{m-i} \cdot (d+i)) = O(2^m \cdot d).$$

□

We use Claim 6.5 to efficiently compute $\hat{G}(\hat{\mathbf{F}}_{j,i}(\mathbf{x}), \hat{\mathbf{w}}_{j,i}(\mathbf{x}), \hat{\mathbf{A}}_{j,i}(\mathbf{x}), \hat{\mathbf{B}}_{j,i}(\mathbf{x}))$.

- The linear polynomials $\hat{\mathbf{A}}_{j,i}(\mathbf{x})$ and $\hat{\mathbf{F}}_{j,i,b}(\mathbf{x})$ can be obtained from $T_{j,A}$ and $T_{j,F}$ in $O((\ell/2^j) \cdot \log n)$ and $O((\ell/2^j) \cdot n)$ time, respectively. We use Claim 6.5 to compute the degree- $(\log n + 1)$ polynomial

$$\sum_{\mathbf{b} \in \{0,1\}^{\log n}} \text{eq}(\hat{\mathbf{A}}_{j,i}(\mathbf{x}), \mathbf{b}) \cdot \hat{\mathbf{F}}_{j,i,b}(\mathbf{x})$$

in $O(2^{\log n} \cdot 1) = O(n)$ field operations.

- The linear polynomials $\hat{\mathbf{B}}_{j,i}(\mathbf{x})$ and $\hat{\mathbf{w}}_{j,i}(\mathbf{x})$ can be obtained from $T_{j,B}$ and $T_{j,w}$ in $O((\ell/2^j) \cdot m)$ and $O((\ell/2^j) \cdot k)$ time, respectively. We split the entries of $\hat{\mathbf{B}}_{j,i}(\mathbf{x})$ into two components: the first $\log M$ entries (which correspond to the bundling randomness), denoted $\hat{\mathbf{c}}(\mathbf{x})$, and the remaining entries (which correspond to the instance), denoted $\hat{\mathbf{s}}(\mathbf{x})$. Recall from Definition 5.5 that

$$\hat{P}(\hat{\mathbf{w}}_{j,i}(\mathbf{x}), \hat{\mathbf{B}}_{j,i}(\mathbf{x})) = \sum_{\mathbf{b} \in \{0,1\}^{\log M}} \text{eq}(\hat{\mathbf{c}}(\mathbf{x}), \mathbf{b}) \cdot \hat{p}_{\mathbf{b}}(\hat{\mathbf{s}}(\mathbf{x}), \hat{\mathbf{w}}_{j,i}(\mathbf{x})). \quad (4)$$

We can compute all $\hat{p}_{\mathbf{b}}(\hat{\mathbf{s}}(\mathbf{x}), \hat{\mathbf{w}}_{j,i}(\mathbf{x}))$ polynomials in $O(d \cdot |\hat{\mathbf{p}}|)$ time (recall that $|\hat{\mathbf{p}}|$ represents the number of field operations required to evaluate the polynomial system). Claim 6.5 then shows how to compute the polynomial in Equation 4 in $O(M \cdot d)$ time.

Overall, we have shown how to compute $\hat{G}(\hat{\mathbf{F}}_i(\mathbf{x}), \hat{\mathbf{w}}_i(\mathbf{x}), \hat{\mathbf{A}}_i(\mathbf{x}), \hat{\mathbf{B}}_i(\mathbf{x}))$ in $O(n + d \cdot (|\hat{\mathbf{p}}| + M))$ time. The prover's overall time in the j -th sumcheck round is therefore $O(\ell/2^j \cdot (n + d \cdot (|\hat{\mathbf{p}}| + M)))$, and summing the complexity across all sumcheck rounds (plus the cost of maintaining tables) yields the lemma statement. □

6.2 Security

Lemma 6.6. *Suppose that:*

- $\text{PG}(2, \alpha) = (1 - \alpha, \alpha)$ is a strong proximity generator for $\mathcal{C}$ with proximity radius δ_{PG} and error err_{PG} ;
- $\delta \in (0, \min\{\delta(\mathcal{C}), \delta_{\text{PG}}(2)\})$.

Construction 6.3 is an IOR from $\mathcal{R}_{\mathcal{C}}^{\ell}$ to $\mathcal{R}_{\mathcal{C},1,\ell}$ with round-by-round knowledge soundness errors $(\varepsilon_{\text{zc}}, \varepsilon_{\text{sum},1}, \dots, \varepsilon_{\text{sum},\log \ell})$ where:

- $\varepsilon_{zc} \leq \frac{|\Lambda(\mathcal{C}^\ell, \delta)| \cdot (\log \ell + 1)}{|\mathbb{F}|}$;
- $\varepsilon_{\text{sum}, i} \leq \frac{(1 + \max\{\log n + 1, \log M + d\}) \cdot |\Lambda(\mathcal{C}^{\ell/2^{i-1}}, \delta)|}{|\mathbb{F}|} + \frac{\ell}{2^i} \cdot \text{err}_{\text{PG}}(\mathcal{C}, 2, \delta)$.

Moreover, the total extraction time is $O(\ell \cdot \text{ecor}_C)$ field operations.

Proof. First we define the knowledge state function and extractor; then we upper bound the round-by-round knowledge soundness errors. Throughout, $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$, $\mathfrak{x} = ((\boldsymbol{\alpha}_i, \mu_i, \beta_i, \eta_i))_{i \in \{0,1\}^{\log \ell}}$, and $\mathfrak{y} = (f_i)_{i \in \{0,1\}^{\log \ell}}$.

Knowledge state function and extractor. We define the knowledge state function and extractor's behavior in each round:

0. **Initial transcript:** We set $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \emptyset, \mathbf{u}) = 1$ if and only if the following hold:

- (a) $\mathbf{u} \in \Lambda(\mathcal{C}^\ell, (f_i)_{i \in \{0,1\}^{\log \ell}}, \delta)$.
- (b) For every $\mathbf{i} \in \{0,1\}^{\log \ell}$, $\hat{u}_i(\boldsymbol{\alpha}_i) = \mu_i$ and $\hat{P}(\beta_i, \mathcal{C}^{-1}(u_i)) = \eta_i$.

1. **Zerocheck randomness:** At this stage the transcript tr is empty. The verifier samples $\boldsymbol{\tau} \leftarrow \mathbb{F}^{\log \ell}$ and $\xi \leftarrow \mathbb{F}$.

Knowledge state function: We set $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \| (\boldsymbol{\tau}, \xi), \mathbf{u}) = 1$ if and only if the following hold:

- (a) $\mathbf{u} \in \Lambda(\mathcal{C}^\ell, (f_i)_{i \in \{0,1\}^{\log \ell}}, \delta)$.
- (b)

$$\sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\boldsymbol{\tau}, \mathbf{i}) \cdot \hat{G}(\hat{U}(\mathbf{i}), \hat{w}'(\mathbf{i}), \hat{A}(\mathbf{i}), \hat{B}(\mathbf{i})) = \sigma,$$

where $\hat{U}(\mathbf{I}) := \sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot u_i$ and $\hat{w}'(\mathbf{I}) := \sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot \mathcal{C}^{-1}(u_i)$.

Extractor: The extractor is $\mathbf{E}_{\text{rbr}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \| (\boldsymbol{\tau}, \xi), \mathbf{u}) := \mathbf{u}$ (the identity function on the witness).

2. **Sumcheck protocol:** At this stage the transcript has the form

$$\text{tr} = ((\boldsymbol{\tau}, \xi), (\hat{h}_i, \gamma_i)_{i < j}, \hat{h}_j).$$

The verifier samples $\gamma_j \leftarrow \mathbb{F}$. Define $\boldsymbol{\gamma}_j := (\gamma_1, \dots, \gamma_j)$. For $\mathbf{i} \in \{0,1\}^{\log \ell}$, define $f_{0,\mathbf{i}} := f_i$. For $i \in [j]$ and $\mathbf{i} \in \{0,1\}^{\log \ell - i}$, define $f_{i,\mathbf{i}} := \sum_{b \in \{0,1\}} \text{eq}(\gamma_i, b) \cdot f_{i-1,(b,\mathbf{i})}$.

Knowledge state function: We set $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \| \boldsymbol{\gamma}_j, \mathbf{u}) = 1$ if and only if the following hold:

- (a) $\mathbf{u} \in \Lambda(\mathcal{C}^{\ell/2^j}, (f_{j,\mathbf{i}})_{\mathbf{i} \in \{0,1\}^{\log \ell - j}}, \delta)$.
- (b) $\hat{h}_1(0) + \hat{h}_1(1) = \sigma$ and, for every $i < j$, $\hat{h}_{i+1}(0) + \hat{h}_{i+1}(1) = \hat{h}_i(\gamma_i)$.
- (c)

$$\sum_{\mathbf{i} \in \{0,1\}^{\log \ell - j}} \text{eq}(\boldsymbol{\tau}, (\boldsymbol{\gamma}_j, \mathbf{i})) \cdot \hat{G}(\hat{U}(\mathbf{i}), \hat{w}'(\mathbf{i}), \hat{A}(\boldsymbol{\gamma}_j, \mathbf{i}), \hat{B}(\boldsymbol{\gamma}_j, \mathbf{i})) = \hat{h}_j(\gamma_j),$$

where $\hat{U}(\mathbf{I}) := \sum_{\mathbf{i} \in \{0,1\}^{\log \ell - j}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot u_i$ and $\hat{w}'(\mathbf{I}) := \sum_{\mathbf{i} \in \{0,1\}^{\log \ell - j}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot \mathcal{C}^{-1}(u_i)$.

Extractor: The extractor $\mathbf{E}_{\text{rbr}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \| \boldsymbol{\gamma}_j, \mathbf{u})$ computes the set of indices $S \subseteq [n]$ on which $\mathbf{u}$ agrees with $(f_{j,\mathbf{i}})_{\mathbf{i} \in \{0,1\}^{\log \ell - j}}$, and outputs the erasure correction of $(f_{j,\mathbf{i}}(S))_{\mathbf{i} \in \{0,1\}^{\log \ell - (j-1)}}$ (using $\mathbf{E}_C$ per Definition 3.9); if correction fails, it aborts. $\mathbf{E}_{\text{rbr}}$ runs in time $\text{et}_{j+1} = O(\ell/2^{j-1} \cdot \text{ecor}_C)$.

3. **New target:** Before the prover's final message, the transcript has the form

$$\text{tr} = \left((\tau, \xi), (\hat{h}_i, \gamma_i)_{i \in [\log \ell]} \right).$$

The prover sends the new target values μ and η in the final message.

Knowledge state function: We set $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, \text{tr} \| (\mu, \eta), u) = 1$ if and only if the following hold:

- (a) $u \in \Lambda(\mathcal{C}, \sum_{i \in [\ell]} \gamma'_i \cdot f_i, \delta)$.
- (b) $\hat{h}_1(0) + \hat{h}_1(1) = \sigma$ and, for every $i < \log \ell$, $\hat{h}_{i+1}(0) + \hat{h}_{i+1}(1) = \hat{h}_i(\gamma_i)$.
- (c) $\hat{u}(\hat{\mathbf{A}}(\gamma)) = \mu$.
- (d) $\hat{P}(\hat{\mathbf{B}}(\gamma), \mathcal{C}^{-1}(u)) = \eta$.

Recall from Definition 4.1 (in particular, the requirement for prover moves) that we must show the following: if $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, \text{tr}, u) = 0$, then $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, \text{tr} \| (\mu, \eta), u) = 0$ for any pair of new targets (μ, η) , i.e., the verifier's output is δ -far from $\mathcal{R}_{\mathcal{C}, 1, \ell}$. Assume Item 2a holds, since otherwise we immediately have $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, \text{tr} \| (\mu, \eta), u) = 0$. Further assume Item 2b holds, since otherwise the verifier immediately rejects. Then it must be that Item 2c does not hold:

$$\text{eq}(\tau, \gamma) \cdot \left(\hat{u}(\hat{\mathbf{A}}(\gamma)) + \xi \cdot \hat{P}(\hat{\mathbf{B}}(\gamma), \mathcal{C}^{-1}(u)) \right) \neq \hat{h}_{\log \ell}(\gamma_{\log \ell}).$$

Assume that $\hat{u}(\hat{\mathbf{A}}(\gamma)) \neq \mu$ or $\hat{P}(\hat{\mathbf{B}}(\gamma), \mathcal{C}^{-1}(u)) \neq \eta$, since otherwise the verifier immediately rejects. We conclude that $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, \text{tr} \| (\mu, \eta), u) = 0$.

RBR knowledge soundness. We upper bound the round-by-round knowledge soundness errors based on the knowledge state function and extractor described above.

1. **Zerocheck randomness:** We show that

$$\Pr_{\tau, \xi} \left[\exists \mathbf{u} : \begin{array}{l} \text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, \text{tr}, \mathbf{u}) = 0 \\ \text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, \text{tr} \| (\tau, \xi), \mathbf{u}) = 1 \end{array} \right] \leq \frac{|\Lambda(\mathcal{C}^\ell, \delta)| \cdot (\log \ell + 1)}{|\mathbb{F}|}.$$

Fix $\mathbf{u} \in \Lambda(\mathcal{C}^\ell, (f_i)_{i \in \{0, 1\}^{\log \ell}}, \delta)$ and assume that $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, \text{tr}, \mathbf{u}) = 0$. We show that

$$\Pr_{\tau, \xi} [\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, \text{tr} \| (\tau, \xi), \mathbf{u}) = 1] \leq \frac{\log \ell + 1}{|\mathbb{F}|}. \quad (5)$$

The error bound follows from a union bound over the codewords in $\Lambda(\mathcal{C}^\ell, (f_i)_{i \in \{0, 1\}^{\log \ell}}, \delta)$, as these are the only choices of $\mathbf{u}$ which allow $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, \text{tr} \| (\tau, \xi), \mathbf{u}) = 1$ for some choice of (τ, ξ) . Since Item 0a holds, it must be that Item 0b does not, i.e., there exists $\mathbf{i}^* \in \{0, 1\}^{\log \ell}$ such that $\hat{u}_{i^*}(\alpha_{i^*}) \neq \mu_{i^*}$ or $\hat{P}(\beta_{i^*}, \mathcal{C}^{-1}(u_{i^*})) \neq \eta_{i^*}$. Observe that we have the following polynomial inequality (where the formal variables are $\mathbf{X}$ and $\mathbf{Y}$):

$$\begin{aligned} \sum_{i \in \{0, 1\}^{\log \ell}} \text{eq}(\mathbf{X}, \mathbf{i}) \cdot \left[\left(\sum_{b \in \{0, 1\}^{\log n}} \text{eq}(\hat{\mathbf{A}}(\mathbf{i}), \mathbf{b}) \cdot \hat{U}_b(\mathbf{i}) \right) + \mathbf{Y} \cdot \hat{P}(\hat{\mathbf{B}}(\mathbf{i}), \hat{M}(\mathbf{i})) \right] \\ \neq \sum_{i \in \{0, 1\}^{\log \ell}} \text{eq}(\mathbf{X}, \mathbf{i}) \cdot (\mu_i + \mathbf{Y} \cdot \eta_i). \end{aligned}$$

Equation 5's bound follows from Lemma 3.2.

2. **Sumcheck protocol:** Let $\mathbf{v}^* := \mathbf{E}_{\text{rbr}}(\mathbb{I}, \mathbb{X}, \mathbb{Y}, \text{tr} \parallel \gamma_j, \mathbf{u})$ denote the extractor's output on the knowledge state witness $\mathbf{u}$. We show that

$$\begin{aligned} & \Pr_{\gamma_j} \left[\exists \mathbf{u} : \begin{array}{l} \text{KState}_{\delta}(\mathbb{I}, \mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{v}^*) = 0 \\ \text{KState}_{\delta}(\mathbb{I}, \mathbb{X}, \mathbb{Y}, \text{tr} \parallel \gamma_j, \mathbf{u}) = 1 \end{array} \right] \\ & \leq \frac{(1 + \max\{\log n + 1, \log M + d\}) \cdot |\Lambda(\mathcal{C}^{\ell/2^{j-1}}, \delta)|}{|\mathbb{F}|} + \log \ell / 2^j \cdot \text{err}_{\text{PG}}(\mathcal{C}, 2, \delta). \end{aligned} \quad (6)$$

For notational convenience, we define the following for $j \in [\log \ell]$, $\mathbf{u} \in \mathcal{C}^{\ell/2^j}$, and $\mathbf{i} \in \mathbb{F}^{\log \ell - j}$:

$$\begin{aligned} \text{Smd}_j[\mathbf{u}, \mathbf{i}] &:= \text{eq}(\tau, (\gamma_j, \mathbf{i})) \cdot \hat{G}(\hat{U}(\mathbf{i}), \hat{w}'(\mathbf{i}), \hat{A}(\gamma_j, \mathbf{i}), \hat{B}(\gamma_j, \mathbf{i})), \\ \text{Sum}_j[\mathbf{u}] &:= \sum_{\mathbf{i} \in \{0,1\}^{\log \ell - j}} \text{Smd}_j[\mathbf{u}, \mathbf{i}], \end{aligned}$$

where $\hat{U}(\mathbf{I}) := \sum_{\mathbf{i} \in \{0,1\}^{\log \ell - j}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot u_{\mathbf{i}}$ and $\hat{w}'(\mathbf{I}) := \sum_{\mathbf{i} \in \{0,1\}^{\log \ell - j}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot \mathcal{C}^{-1}(u_{\mathbf{i}})$.

Observe that in order for Equation 6's event to hold, Item 2b must hold. Hence, Equation 6's probability is at most

$$\Pr_{\gamma_j} \left[\exists \mathbf{u} \in \Lambda(\mathcal{C}^{\ell/2^j}, \mathbf{f}_j, \delta) : \begin{array}{l} (\Delta(\mathbf{v}^*, \mathbf{f}_{j-1}) > \delta \vee \text{Sum}_{j-1}[\mathbf{v}^*] \neq \hat{h}_j(0) + \hat{h}_j(1)) \\ \wedge \text{Sum}_j[\mathbf{u}] = \hat{h}_j(\gamma_j) \end{array} \right]. \quad (7)$$

Claim 6.7. *It holds that*

$$\begin{aligned} & \Pr_{\gamma_j} \left[\exists \mathbf{v} \in \Lambda(\mathcal{C}^{\ell/2^{j-1}}, \mathbf{f}_{j-1}, \delta) : \begin{array}{l} \text{Sum}_{j-1}[\mathbf{v}] \neq \hat{h}_j(0) + \hat{h}_j(1) \\ \wedge \sum_{\mathbf{i} \in \{0,1\}^{\log \ell - j}} \text{Smd}_{j-1}[\mathbf{v}, (\gamma_j, \mathbf{i})] = \hat{h}_j(\gamma_j) \end{array} \right] \\ & \leq \frac{(1 + \max\{\log n + 1, \log M + d\}) \cdot |\Lambda(\mathcal{C}^{\log \ell / 2^{k-1}}, \delta)|}{|\mathbb{F}|}. \end{aligned}$$

Proof. Fix $\mathbf{v} \in \Lambda(\mathcal{C}^{\ell/2^{j-1}}, \mathbf{f}_{j-1}, \delta)$ and assume that $\text{Sum}_{j-1}[\mathbf{v}] \neq \hat{h}_j(0) + \hat{h}_j(1)$. Observe that we have the following polynomial inequality (where the formal variable is $\mathbf{X}$):

$$\sum_{\mathbf{i} \in \{0,1\}^{\log \ell - j}} \text{Smd}_{j-1}[\mathbf{v}, (\mathbf{X}, \mathbf{i})] \neq \hat{h}_j(\mathbf{X}),$$

since the two polynomials do not sum to the same value over $\{0,1\}$. The claim follows from Lemma 3.2 and a union bound over the codewords in $\Lambda(\mathcal{C}^{\ell/2^{j-1}}, \mathbf{f}_{j-1}, \delta)$. $\square$

Claim 6.8. *It holds that*

$$\Pr_{\gamma_j} \left[\exists S \subseteq [n], \mathbf{u} \in \mathcal{C}^{\ell/2^j} : \begin{array}{l} |S| \geq (1 - \delta) \cdot n \\ \wedge \mathbf{u}(S) = \mathbf{f}_j(S) \\ \wedge \mathbf{v}^*(S) \neq \mathbf{f}_{j-1}(S) \end{array} \right] \leq \ell / 2^j \cdot \text{err}_{\text{PG}}(\mathcal{C}, 2, \delta).$$

Proof. Recall that we are assuming $\text{PG}(2, \alpha) = (1 - \alpha, \alpha)$ is a strong proximity generator for $\mathcal{C}$ with proximity radius δ_{PG} and $\delta < \delta_{\text{PG}}(2)$. From Definition 3.15 and a union bound over each $u_{\mathbf{i}}$, we have

$$\Pr_{\gamma_j} \left[\exists S \subseteq [n], \mathbf{u} \in \mathcal{C}^{\ell/2^j} : \begin{array}{l} |S| \geq (1 - \delta) \cdot n \\ \wedge \mathbf{u}(S) = \mathbf{f}_j(S) \\ \wedge \forall \mathbf{v} \in \mathcal{C}^{\ell/2^{j-1}}, \mathbf{v}(S) \neq \mathbf{f}_{j-1}(S) \end{array} \right] \leq \ell / 2^j \cdot \text{err}_{\text{PG}}(\mathcal{C}, 2, \delta). \quad (8)$$

Fix γ_j such that Equation 8's event does not hold. Then for arbitrary S and $\mathbf{u}$ such that $|S| \geq (1 - \delta) \cdot n$ and $\mathbf{u}(S) = \mathbf{f}_j(S)$, it must be that $\mathbf{f}_{j-1}$ on agrees with $\mathcal{C}^{\ell/2^{j-1}}$ on S . Recall that we are assuming $\delta < \delta(\mathcal{C})$; this guarantees that erasure correction succeeds. In particular, we have $\mathbf{v}^*(S) = \mathbf{f}_{j-1}(S)$, and the claim follows. $\square$

Fix γ_j such that the events of Claims 6.7 and 6.8 do not hold; by a union bound, there are at least

$$\left(1 - \frac{(1 + \max\{\log n + 1, \log M + d\}) \cdot |\Lambda(\mathcal{C}^{\log \ell/2^{k-1}}, \delta)|}{|\mathbb{F}|} - \ell/2^j \cdot \text{err}_{\text{PG}}(\mathcal{C}, 2, \delta)\right) \cdot |\mathbb{F}|$$

such choices for γ_j . Consider arbitrary $\mathbf{u} \in \Lambda(\mathcal{C}^{\ell/2^j}, \mathbf{f}_j, \delta)$ satisfying $\text{Sum}[\mathbf{u}] = \hat{h}_j(\gamma_j)$; this is required for Equation 7's event to hold. Let $S \subseteq [n]$ be set of indices on which $\mathbf{u}$ agrees with $\mathbf{f}_j$; since $\mathbf{u}$ is δ -close to $\mathbf{f}_j$, it must be that $|S| \geq (1 - \delta) \cdot n$. Since γ_j is such that Claim 6.8's event does not hold, it must be that $\mathbf{v}^*(S) = \mathbf{f}_{j-1}(S)$. Observe the following:

- $\Delta(\mathbf{v}^*, \mathbf{f}_{j-1}) \leq \delta$, since they agree on S .
- For all $\mathbf{i} \in \{0, 1\}^{\log \ell - j}$, both u_i and the codeword

$$\sum_{b \in \{0, 1\}} \text{eq}(\gamma_j, b) \cdot v_{b, \mathbf{i}}^*$$

agree with $f_{j, \mathbf{i}}$ on S . Recall that we are assuming $\delta < \delta(\mathcal{C})$; this guarantees that these codewords are in fact the same. It follows that

$$\hat{h}_j(\gamma_j) = \sum_{\mathbf{i} \in \{0, 1\}^{\log \ell - j}} \text{Smd}_j[\mathbf{u}, \mathbf{i}] = \sum_{\mathbf{i} \in \{0, 1\}^{\log \ell - j}} \text{Smd}_{j-1}[\mathbf{v}^*, (\gamma_j, \mathbf{i})].$$

Since γ_j is such that Claim 6.7's event does not hold, it must be that $\text{Sum}_{j-1}[\mathbf{v}^*] = \hat{h}_j(0) + \hat{h}_j(1)$.

Hence, Equation 7's event cannot hold and the bound follows. $\square$

7 Codeword batching

Theorem 7.1. *Fix a linear code $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ and parameters $\ell, r, s, t \in \mathbb{N}$. Construction 7.2 is a public-coin IOR from $\mathcal{R}_{\mathcal{C}, r, \ell}$ (Definition 6.1) to $\mathcal{R}_{\mathcal{C}, r+s+t}$ (Definition 5.7) with the following parameters.*

- **Rounds.** *The protocol has 2 rounds.*
- **Proof length.** *The prover sends $n + s$ field elements.*
- **Query complexity.** *The verifier makes $\ell \cdot t$ queries.*
- **Prover time.** *The prover performs $O((\ell + s) \cdot n)$ field operations.*
- **Verifier time.** *The verifier performs $O(\ell \cdot t)$ field operations.*

Moreover, the IOR outputs t binary evaluations points and $r + s$ non-binary evaluations points.

For every $\delta \in (0, 1)$, if the field $\mathbb{F}$ and repetition parameters t, s are such that

$$|\mathbb{F}| \geq 2^{\lambda/s-1} \cdot |\Lambda(\mathcal{C}, \delta)|^{2/s} \cdot \log n \quad \text{and} \quad t \geq \frac{\lambda}{-\log(1-\delta)},$$

the IOR has round-by-round knowledge soundness error $\leq 2^{-\lambda}$. Moreover, the round-by-round extractor is the identity function on the witness.

In Section 7.1 we describe the protocol and in Section 7.2 we prove its security.

7.1 Protocol

Construction 7.2. We describe the IOR from $\mathcal{R}_{\mathcal{C}, r, \ell}$ (Definition 6.1) to $\mathcal{R}_{\mathcal{C}, r+s+t}$ (Definition 5.7).

- **Inputs.** The verifier is given the explicit instance $\mathbf{x} = (\gamma, (\alpha_i, \mu_i)_{i \in [r]}, \beta, \eta)$ and oracle access to the implicit instance $\mathbf{y} = (f_1, \dots, f_\ell)$. In the honest case, $(\mathbf{i}, \mathbf{x}, \mathbf{y}) \in \mathcal{L}(\mathcal{R}_{\mathcal{C}, r, \ell})$.
- **Protocol:**
 1. **New oracle:** The prover sends $f: [n] \rightarrow \mathbb{F}$. In the honest case, $f := \sum_{i \in [\ell]} \gamma_i \cdot f_i$.
 2. **OOD samples:** The verifier samples $\alpha_{r+1}, \dots, \alpha_{r+s} \leftarrow \mathbb{F}^{\log n}$.
 3. **OOD answers:** The prover sends $\mu_{r+1}, \dots, \mu_{r+s} \in \mathbb{F}$. In the honest case, $\mu_{r+k} := \hat{f}(\alpha_{r+k})$ for every $k \in [s]$.
 4. **Shift queries:** The verifier samples $x_1, \dots, x_t \leftarrow [n]$. For $k \in [t]$, define $\alpha_{r+s+k} := \text{binary}(x_k) \in \mathbb{F}^{\log n}$.
 5. **Outputs:** For every $k \in [t]$, the verifier queries $f_1, \dots, f_\ell$ at x_k and computes

$$\mu_{r+s+k} := \sum_{i \in [\ell]} \gamma_i \cdot f_i(x_k).$$

The verifier outputs the new explicit instance $\mathbf{x}' := ((\alpha_i, \mu_i)_{i \in [r+s+t]}, \beta, \eta)$ and implicit instance $\mathbf{y}' := f$.

7.2 Security

Lemma 7.3. *Construction 7.2 is an IOR from $\mathcal{R}_{\mathcal{C},r,\ell}$ to $\mathcal{R}_{\mathcal{C},r+s+t}$ with round-by-round knowledge soundness errors $(\varepsilon_{\text{out}}, \varepsilon_{\text{shift}})$ where:*

- $\varepsilon_{\text{out}} \leq \frac{|\Lambda(\mathcal{C}, \delta)|^2}{2} \cdot \left(\frac{\log n}{|\mathbb{F}|}\right)^s$;
- $\varepsilon_{\text{shift}} \leq (1 - \delta)^t$.

Moreover, the round-by-round extractor is the identity function on the witness.

Proof. First we define the knowledge state function and extractor; then we upper bound the round-by-round knowledge soundness errors. Throughout, $\mathfrak{i} = (\hat{p}, M, N, k)$, $\mathfrak{x} = (\gamma, (\alpha_i, \mu_i)_{i \in [r]}, \beta, \eta)$, and $\mathfrak{y} = (f_i)_{i \in [\ell]}$.

Knowledge state function and extractor. We define the knowledge state function and extractor's behavior in each round:

0. **Initial transcript:** We set $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \emptyset, u) = 1$ if and only if $u \in \Lambda(\mathcal{C}, \sum_{i \in [\ell]} \gamma_i \cdot f_i, \delta)$ and $u \in \mathcal{C}[(\alpha_i, \mu_i)_{i \in [r]}, (\hat{P}, \beta, \eta)]$.

1. **OOD samples:** At this stage the transcript has the form $\text{tr} = (f)$. The verifier samples $\alpha_{r+1}, \dots, \alpha_{r+s}$.

Knowledge state function: We set $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \| (\alpha_{r+1}, \dots, \alpha_{r+s}), u) = 1$ if and only if one of the following holds:

- (a) $u \in \Lambda(\mathcal{C}, \sum_{i \in [\ell]} \gamma_i \cdot f_i, \delta)$ and $u \in \mathcal{C}[(\alpha_i, \mu_i)_{i \in [r]}, (\hat{P}, \beta, \eta)]$.
- (b) There exist distinct $v, v' \in \Lambda(\mathcal{C}, f, \delta)$ such that $\hat{v}(\alpha_{r+k}) = \hat{v}'(\alpha_{r+k})$ for every $k \in [s]$.

Extractor: The extractor is $\mathbf{E}_{\text{rbr}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \| (\alpha_{r+1}, \dots, \alpha_{r+s}), u) := u$ (the identity function on the witness).

2. **Shift queries:** At this stage the transcript has the form

$$\text{tr} = (f, (\alpha_{r+1}, \dots, \alpha_{r+s}), (\mu_{r+1}, \dots, \mu_{r+s})) .$$

The verifier samples $x_1, \dots, x_t \leftarrow [n]$. Define $\alpha_{r+s+1}, \dots, \alpha_{r+s+t}$ and $\mu_{r+s+1}, \dots, \mu_{r+s+t}$ as in Construction 7.2.

Knowledge state function: We set $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \| (x_1, \dots, x_t), u) = 1$ if and only if $u \in \Lambda(\mathcal{C}, f, \delta)$ and $u \in \mathcal{C}[(\alpha_i, \mu_i)_{i \in [r+s+t]}, (\hat{P}, \beta, \eta)]$. This corresponds to whether the verifier's output is δ -close to $\mathcal{R}_{\mathcal{C}, r+s+t}$.

Extractor: The extractor is $\mathbf{E}_{\text{rbr}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \| (x_1, \dots, x_t), u) := u$ (the identity function on the witness).

RBR knowledge soundness. We upper bound the round-by-round knowledge soundness errors based on the knowledge state function and extractor described above.

1. **OOD samples:** We argue that

$$\Pr_{\alpha_{r+1}, \dots, \alpha_{r+s}} \left[\exists u : \begin{array}{l} \text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, u) = 0 \\ \text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \| (\alpha_{r+1}, \dots, \alpha_{r+s}), u) = 1 \end{array} \right] \leq \frac{|\Lambda(\mathcal{C}, \delta)|^2}{2} \cdot \left(\frac{\log n}{|\mathbb{F}|}\right)^s .$$

If $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, u) = 0$, then Item 1a cannot hold. Hence it suffices to bound the probability that Item 1b holds. Let ZE be the zero-evader from Lemma 3.5; observe that, for $v \in \mathcal{C}$ and $\alpha \in \mathbb{F}^{\log n}$, $\hat{v}(\alpha) = \langle \text{ZE}(\alpha), \mathcal{C}^{-1}(v) \rangle$. Then the bound follows from Lemma 3.14.

2. **Shift queries.** We argue that

$$\Pr_{x_1, \dots, x_t} \left[\exists u : \begin{array}{l} \text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, u) = 0 \\ \text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel (x_1, \dots, x_t), u) = 1 \end{array} \right] \leq (1 - \delta)^t.$$

Define $\mathcal{S}_{\text{ood}} := \{u \in \Lambda(\mathcal{C}, f, \delta) : \forall k \in [s], \hat{u}(\alpha_{r+k}) = \mu_{r+k}\}$ and $g := \sum_{i \in [\ell]} \gamma_i \cdot f_i$. We consider the following cases.

- *There are no codewords in $\Lambda(\mathcal{C}, f, \delta) \cap \mathcal{S}_{\text{ood}}$.* For every $u \in \Lambda(\mathcal{C}, f, \delta)$, there exists $k \in [s]$ such that $\hat{u}(\alpha_{r+k}) \neq \mu_{r+k}$. Hence, we unconditionally have $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel (x_1, \dots, x_t), u) = 0$.
- *There is exactly one codeword $u \in \Lambda(\mathcal{C}, f, \delta) \cap \mathcal{S}_{\text{ood}}$.* Fix $u \in \Lambda(\mathcal{C}, f, \delta) \cap \mathcal{S}_{\text{ood}}$ and assume that $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, u) = 0$. We show that

$$\Pr_{x_1, \dots, x_t} [\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel (x_1, \dots, x_t), u) = 1] \leq (1 - \delta)^t. \quad (9)$$

The error bound follows from the fact that this is the only possible choice of u which allows $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel (x_1, \dots, x_t), u) = 1$ for some choice of $(x_1, \dots, x_t)$. Assume that $u \in \mathcal{C}[(\alpha_i, \mu_i)_{i \in [r]}, (\bar{P}, \beta, \eta)]$, since otherwise Equation 9's event cannot hold. Since Item 1a does not hold, it must be that $\Delta(u, g) > \delta$. Equation 9's bound follows from the fact that, except with probability at most $(1 - \delta)^t$, for at least one of the randomly sampled locations $x_1, \dots, x_t \leftarrow [n]$, we have $\hat{u}(\text{binary}(x_i)) = u(x_i) \neq g(x_i) = \mu_{r+s+i}$.

- *There are multiple codewords in $\Lambda(\mathcal{C}, f, \delta) \cap \mathcal{S}_{\text{ood}}$.* Item 1b holds so we unconditionally have $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, u) = 1$.

□

8 Multilinear constraint batching

Theorem 8.1. *Fix a linear code $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ and $r \in 2^{\mathbb{N}}$. Construction 8.2 is a public-coin IOR from $\mathcal{R}_{\mathcal{C},r}$ (Definition 5.7) to $\mathcal{R}_{\mathcal{C},1} = \mathcal{R}_{\mathcal{C}}$ (Definition 5.7) with the following parameters.*

- **Rounds.** *The protocol has $\log n + 1$ rounds.*
- **Proof length.** *The prover sends $3 \cdot \log n + 1$ field elements.*
- **Query complexity.** *The verifier makes no queries.*
- **Prover time.** *The prover performs $O(\max\{1, r - t\} \cdot n + t \cdot \log n)$ field operations, where $t \in \{0, \dots, r\}$ is the number of evaluations claims in the boolean hypercube (see Lemma 8.3).*
- **Verifier time.** *The verifier performs $O(r \cdot \log n)$ field operations.*

For every $\delta \in (0, 1)$, if

$$|\mathbb{F}| \geq 2^\lambda \cdot |\Lambda(\mathcal{C}, \delta)| \cdot \max\{2, \log r\},$$

the IOR has round-by-round knowledge soundness error $\leq 2^{-\lambda}$. Moreover, the round-by-round extractor is the identity function on the witness.

In Section 8.1 we describe the protocol and in Section 8.2 we prove its security.

8.1 Protocol

Construction 8.2. We describe the IOR from $\mathcal{R}_{\mathcal{C},r}$ (Definition 5.7) to $\mathcal{R}_{\mathcal{C},1} = \mathcal{R}_{\mathcal{C}}$ (Definition 5.7). We present the protocol in terms of a general zero-evader $\mathbf{ZE}: D_{\mathbf{ZE}} \rightarrow \mathbb{F}^r$ (Definition 3.4), which in the theorem we fix to the zero-evader $\mathbf{ZE}: \mathbb{F}^{\log r} \rightarrow \mathbb{F}^r$ whose mapping is $\tau \mapsto (\mathbf{eq}(\tau, \mathbf{i}))_{\mathbf{i} \in \{0,1\}^{\log r}}$.

- **Inputs.** The verifier is given the explicit instance $\mathbf{x} = ((\alpha_i, \mu_i)_{\mathbf{i} \in \{0,1\}^{\log r}}, \beta, \eta)$ and oracle access to the implicit instance $\mathbf{y} = f: [n] \rightarrow \mathbb{F}$. In the honest case, $(\mathbf{i}, \mathbf{x}, \mathbf{y}) \in \mathcal{L}(\mathcal{R}_{\mathcal{C},r})$.
- **Interaction phase.**
 1. **Zerocheck randomness:** The verifier samples and sends $\rho \leftarrow D_{\mathbf{ZE}}$. Set $\xi := \mathbf{ZE}(\rho)$.
 2. **Sumcheck batching:** Define the following:

$$\mathbf{eq}^*(\mathbf{x}) := \sum_{\mathbf{i} \in \{0,1\}^{\log r}} \xi_{\mathbf{i}} \cdot \mathbf{eq}(\alpha_{\mathbf{i}}, \mathbf{x}), \quad \sigma := \sum_{\mathbf{i} \in \{0,1\}^{\log r}} \xi_{\mathbf{i}} \cdot \mu_{\mathbf{i}}.$$

The prover and verifier engage in a $(\log n)$ -round sumcheck protocol for the following claim:

$$\sum_{\mathbf{a} \in \{0,1\}^{\log n}} \mathbf{eq}^*(\mathbf{a}) \cdot \hat{f}(\mathbf{a}) = \sigma.$$

During the interaction, the prover sends quadratic sumcheck polynomials $\hat{h}_1, \dots, \hat{h}_{\log n}$, and the verifier sends randomness $\gamma \in \mathbb{F}^{\log n}$.

3. **New target:** The prover sends $\mu \in \mathbb{F}$. In the honest case, $\mu := \hat{f}(\gamma)$.
- **Decision phase.** (The verifier rejects if any check fails.)
 1. **Sumcheck decision:** The verifier checks that

$$\hat{h}_1(0) + \hat{h}_1(1) = \sigma \quad \text{and} \quad \forall j \in \{1, \dots, \log n - 1\}, \quad \hat{h}_{j+1}(0) + \hat{h}_{j+1}(1) = \hat{h}_j(\gamma_j).$$

2. **New target decision:** The verifier checks that $\hat{h}_{\log n}(\gamma_{\log n}) = \text{eq}^*(\gamma) \cdot \mu$.
3. **Outputs:** Set $\alpha := \gamma$. The verifier outputs the new explicit instance $x' := (\alpha, \mu, \beta, \eta)$ and implicit instance $y' := f$.

We analyze the prover time of Construction 8.2; other complexity parameters are clear.

Lemma 8.3. *The prover in Construction 8.2 can be implemented to perform*

$$O(\max\{1, s\} \cdot n + t \cdot \log n)$$

field operations, where $t := |\{i : \alpha_i \in \{0, 1\}^{\log n}\}|$ and $s := r - t$.

Proof. We use a technique from [CBBZ23, Sec 3.8.1]. We partition the index set $\{0, 1\}^{\log r}$ into $I_{\text{ID}} := \{i : \alpha_i \in \{0, 1\}^{\log n}\}$ and $I_{\text{OOD}} := \{0, 1\}^{\log r} \setminus I_{\text{ID}}$. Let $j \in [\log n]$. In the j -th sumcheck round, define $\gamma_{j-1} = (\gamma_1, \dots, \gamma_{j-1})$. The prover must compute the following sumcheck polynomial:

$$\hat{h}_j(\mathbf{X}) = \sum_{\mathbf{a} \in \{0, 1\}^{\log n - j}} \text{eq}^*(\gamma_{j-1}, \mathbf{X}, \mathbf{a}) \cdot \hat{f}(\gamma_{j-1}, \mathbf{X}, \mathbf{a}).$$

Rewriting the equation, we get

$$\begin{aligned} \hat{h}_j(\mathbf{X}) &= \sum_{\mathbf{a} \in \{0, 1\}^{\log n - j}} \hat{f}(\gamma_{j-1}, \mathbf{X}, \mathbf{a}) \cdot \sum_{i \in \{0, 1\}^{\log r}} \xi_i \cdot \text{eq}(\alpha_i, \gamma_{j-1}, \mathbf{X}, \mathbf{a}) \\ &= \sum_{\mathbf{a} \in \{0, 1\}^{\log n - j}} \hat{f}(\gamma_{j-1}, \mathbf{X}, \mathbf{a}) \cdot \left(\sum_{i \in I_{\text{OOD}}} \xi_i \cdot \text{eq}(\alpha_i, \gamma_{j-1}, \mathbf{X}, \mathbf{a}) + \sum_{i \in I_{\text{ID}}} \xi_i \cdot \text{eq}(\alpha_i, \gamma_{j-1}, \mathbf{X}, \mathbf{a}) \right). \end{aligned}$$

We show that the following linear polynomials can be efficiently computed.

- The polynomials $(\hat{f}(\gamma_{j-1}, \mathbf{X}, \mathbf{a}))_{\mathbf{a} \in \{0, 1\}^{\log n - j}}$ can be obtained from a table that is maintained across rounds, as in [VSBW13]. The table is initially of size n and is initialized with f ; in the j -th sumcheck round, it is updated (and halved in size) via a linear-time computation that depends on γ_j . This takes $O(n)$ field operations across all rounds.
- The polynomials $(\sum_{i \in I_{\text{OOD}}} \xi_i \cdot \text{eq}(\alpha_i, \gamma_{j-1}, \mathbf{X}, \mathbf{a}))_{\mathbf{a} \in \{0, 1\}^{\log n - j}}$ can be obtained from $s = |I_{\text{OOD}}|$ tables that are maintained across rounds, as in [VSBW13]. For every $i \in I_{\text{OOD}}$, the i -th table is initialized with $\{\xi_i \cdot \text{eq}(\alpha_i, \mathbf{a})\}_{\mathbf{a} \in \{0, 1\}^{\log n}}$; in the j -th sumcheck round, it is updated (and halved in size) via a linear-time computation that depends on γ_j . This takes $O(s \cdot n)$ field operations across all rounds.
- The polynomials $(\sum_{i \in I_{\text{ID}}} \xi_i \cdot \text{eq}(\alpha_i, \gamma_{j-1}, \mathbf{X}, \mathbf{a}))_{\mathbf{a} \in \{0, 1\}^{\log n - j}}$ can be obtained in $O(n + t \cdot \log n)$ field operations across all rounds, using a technique from [CBBZ23, Sec 3.8.1]. The key observation is that, since $\alpha_i \in \{0, 1\}^{\log n}$, there are at most t values of $\mathbf{a} \in \{0, 1\}^{\log n - j}$ for which $\sum_{i \in I_{\text{ID}}} \xi_i \cdot \text{eq}(\alpha_i, \gamma_{j-1}, \mathbf{X}, \mathbf{a})$ is non-zero. The non-zero polynomials can be maintained and updated dynamically in each rounds.

Observe that $\hat{h}_j(\mathbf{X})$ is the sum of the following quadratic polynomials, which we show can be efficiently computed from the linear polynomials.

- The polynomial

$$\sum_{\mathbf{a} \in \{0,1\}^{\log n-j}} \hat{f}(\gamma_{j-1}, \mathbf{X}, \mathbf{a}) \cdot \sum_{i \in I_{\text{OOD}}} \xi_i \cdot \text{eq}(\alpha_i, \gamma_{i-1}, \mathbf{X}, \mathbf{a})$$

can be obtained in $O(n/2^j \cdot s)$ field operations.

- The polynomial

$$\sum_{\mathbf{a} \in \{0,1\}^{\log n-j}} \hat{f}(\gamma_{j-1}, \mathbf{X}, \mathbf{a}) \cdot \sum_{i \in I_{\text{ID}}} \xi_i \cdot \text{eq}(\alpha_i, \gamma_{i-1}, \mathbf{X}, \mathbf{a})$$

can be obtained in $O(t)$ field operations by restricting the sum to the (at most t values of) $\mathbf{a} \in \{0,1\}^{\log n-j}$ for which $\sum_{i \in I_{\text{ID}}} \xi_i \cdot \text{eq}(\alpha_i, \gamma_{j-1}, \mathbf{X}, \mathbf{a})$ is non-zero.

The prover's overall time in the j -th sumcheck round is therefore $O(n/2^j \cdot s + t)$, and summing the complexity across all sumcheck rounds (plus the cost of maintaining tables) yields the lemma statement. $\square$

8.2 Security

Lemma 8.4. *Suppose that the zero-evader ZE has error $\varepsilon_{\text{zero}}$. Construction 8.2 is an IOR from $\mathcal{R}_{\mathcal{C},r}$ to $\mathcal{R}_{\mathcal{C}}$ with round-by-round knowledge soundness errors $(\varepsilon_{\text{zc}}, \varepsilon_{\text{sum}}, \dots, \varepsilon_{\text{sum}})$ where:*

- $\varepsilon_{\text{zc}} \leq |\Lambda(\mathcal{C}, \delta)| \cdot \varepsilon_{\text{zero}};$
- $\varepsilon_{\text{sum}} \leq |\Lambda(\mathcal{C}, \delta)| \cdot \frac{2}{|\mathbb{F}|}.$

Moreover, the round-by-round extractor is the identity function on the witness.

Proof. First we define the knowledge state function and extractor; then we upper bound the round-by-round knowledge soundness errors. Throughout, $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$, $\mathfrak{x} = ((\alpha_i, \mu_i)_{i \in [r]}, \beta, \eta)$, and $\mathfrak{y} = f$.

Knowledge state function and extractor. We define the knowledge state function and extractor's behavior in each round.

0. **Initial transcript:** We set $\text{KState}_{\delta}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \emptyset, u) = 1$ if and only if the following hold:

- (a) $u \in \Lambda(\mathcal{C}, f, \delta)$ and $u \in \mathcal{C}[(\hat{P}, \beta, \eta)]$.
- (b) For every $i \in \{0,1\}^{\log r}$, $\hat{u}(\alpha_i) = \mu_i$.

1. **Zerocheck randomness:** At this stage, the transcript tr is empty. The verifier samples $\rho \leftarrow D_{\text{ZE}}$.

Knowledge state function: We set $\text{KState}_{\delta}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \rho, u) = 1$ for some $u \in \mathcal{C}$ if and only if the following hold:

- (a) $u \in \Lambda(\mathcal{C}, f, \delta)$ and $u \in \mathcal{C}[(\hat{P}, \beta, \eta)]$.
- (b) $\sum_{i \in \{0,1\}^{\log r}} \xi_i \cdot \hat{u}(\alpha_i) = \sigma$.

Extractor: The extractor $\mathbf{E}_{\text{rbr}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \rho, u) := u$ is the identity function on the witness.

2. **Sumcheck rounds:** At this stage, the transcript has the form

$$\text{tr} = \left(\rho, (\hat{h}_i, \gamma_i)_{i < j}, \hat{h}_j \right).$$

The verifier samples $\gamma_j \leftarrow \mathbb{F}$. Define $\gamma_j = (\gamma_1, \dots, \gamma_j)$, and eq^* be defined as in Construction 8.2.

Knowledge state function: We set $\text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr} \parallel \gamma_i, u) = 1$ if and only if the following hold:

- (a) $u \in \Lambda(\mathcal{C}, f, \delta)$ and $u \in \mathcal{C}[(\hat{P}, \beta, \eta)]$.
- (b) $\hat{h}_1(0) + \hat{h}_1(1) = \sigma$, and, for $i < j$, $\hat{h}_{i+1}(0) + \hat{h}_{i+1}(1) = \hat{h}_i(\gamma_i)$.
- (c) $\sum_{i \in \{0,1\}^{\log r - i}} \text{eq}^*(\gamma_j, \mathfrak{i}) \cdot \hat{u}(\gamma_j, \mathfrak{i}) = \hat{h}_j(\gamma_j)$.

Extractor: The extractor $\mathbf{E}_{\text{rbr}}(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr} \parallel \gamma_i, u) := u$ is the identity function on the witness.

3. **New target:** Before the prover's final message, the transcript has the form

$$\text{tr} = \left(\rho, (\hat{h}_j, \gamma_j)_{j \in [\log n]} \right).$$

The prover sends the new target value μ in the final message.

Knowledge state function: We set $\text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr} \parallel \mu, u) = 0$ if any only if the following hold:

- (a) $u \in \Lambda(\mathcal{C}, f, \delta)$ and $u \in \mathcal{C}[(\hat{P}, \beta, \eta)]$.
- (b) $\hat{h}_1(0) + \hat{h}_1(1) = \sigma$, and, for $i < \log n$, $\hat{h}_{i+1}(0) + \hat{h}_{i+1}(1) = \hat{h}_i(\gamma_i)$.
- (c) $\hat{u}(\gamma) = \mu$.

Recall from Definition 4.1 (in particular, the requirement for prover moves) that we must show the following: if $\text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr}, u) = 0$, then $\text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr} \parallel \mu, u) = 0$ for any new target μ , i.e., the verifier's output is δ -far from $\mathcal{R}_\mathcal{C}$. Assume Item 2a holds, since otherwise we immediately have $\text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr} \parallel \mu, u) = 0$. Further assume Item 2b holds, since otherwise the verifier immediately rejects. Then it must be that Item 2c does not hold:

$$\text{eq}^*(\gamma) \cdot \hat{u}(\gamma) \neq \hat{h}_{\log n}(\gamma_{\log n}).$$

Assume that $\hat{u}(\gamma) \neq \mu$, since otherwise the verifier immediately rejects. We conclude that $\text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr} \parallel \mu, u) = 0$.

RBR knowledge soundness. We upper bound the round-by-round knowledge soundness errors based on the knowledge state function and extractor described above.

1. **Zerocheck randomness:** We show that

$$\Pr_{\rho} \left[\exists u : \begin{array}{l} \text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr}, u) = 0 \\ \text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr} \parallel \rho, u) = 1 \end{array} \right] \leq |\Lambda(\mathcal{C}, \delta)| \cdot \varepsilon_{\text{zero}}.$$

Fix $u \in \Lambda(\mathcal{C}, f, \delta)$ and assume that $\text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr}, u) = 0$. We show that

$$\Pr_{\rho} [\text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr} \parallel \rho, u) = 1] \leq \varepsilon_{\text{zero}}. \quad (10)$$

The error bound follows from a union bound over the codewords in $\Lambda(\mathcal{C}, f, \delta)$, as these are the only choices of u which allow $\text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr} \parallel \rho, u) = 1$ for some choice of ρ . Assume Item 0a holds, since otherwise we immediately have $\text{KState}_\delta(\mathfrak{i}, \mathbb{z}, \mathbb{y}, \text{tr} \parallel \rho, u) = 0$. Then it must be that Item 0b does not hold, i.e., there exists $\mathfrak{i} \in \{0, 1\}^{\log r}$ such that $\hat{u}(x_{\mathfrak{i}}) \neq \mu_{\mathfrak{i}}$. Equation 10 follows from the fact that ZE is a zero-evader.

2. **Sumcheck rounds:** We show that

$$\Pr_{\gamma_i} \left[\exists u : \begin{array}{l} \text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, u) = 0 \\ \text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \gamma_j, u) = 1 \end{array} \right] \leq |\Lambda(\mathcal{C}, \delta)| \cdot \frac{2}{|\mathbb{F}|}.$$

Fix $u \in \Lambda(\mathcal{C}, f, \delta)$ and assume that $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, u) = 0$. We show that

$$\Pr_{\gamma_j} [\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \gamma_j, u) = 1] \leq \frac{2}{|\mathbb{F}|}. \quad (11)$$

The error bound follows from a union bound over the codewords in $\Lambda(\mathcal{C}, f, \delta)$, as these are the only choices of u which allow $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \gamma_j, u) = 1$ for some choice of γ_j . We handle the case where $j > 1$; the case where $j = 1$ is identical up to minor syntactic modifications. Assume Items 2a and 2b hold, since otherwise we immediately have $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr} \parallel \gamma_j, u) = 0$. Then it must be that Item 2c does not hold. Observe that we have the following polynomial inequality (where the formal variable is $\mathbf{X}$):

$$\sum_{\mathbf{i} \in \{0,1\}^{\log n - j}} \text{eq}^*(\gamma_{j-1}, \mathbf{X}, \mathbf{i}) \cdot \hat{u}(\gamma_{j-1}, \mathbf{X}, \mathbf{i}) \not\equiv \hat{h}_j(\mathbf{X}),$$

since the right hand side sums to $\hat{h}_{j-1}(\gamma_{j-1})$ whereas the left hand side does not by virtue of Item 2c not holding. Equation 11's bound follows from Lemma 3.2.

□

9 Combining the IORs

We obtain an IOR from $\mathcal{R}_C^\ell$ to $\mathcal{R}_C$ by combining the IORs described in Section 6, Section 7, Section 8. Below we state and prove the corresponding theorem. For ease of understanding, we additionally provide in one description the IOR resulting from this combination (Construction 9.4) as well as the corresponding straightline state-restoration extractor (Construction 9.5).

Theorem 9.1. *Fix a linear code $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ and $\ell \in 2^{\mathbb{N}}$, and let $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$ be of degree d . Recall that $\kappa := N - k$. Fix parameters $s, t \in \mathbb{N}$ such that $1 + s + t \in 2^{\mathbb{N}}$.*

Construction 9.4 is a public-coin IOR from $\mathcal{R}_C^\ell$ (Definition 5.7) to $\mathcal{R}_C$ (Definition 5.7) with the following parameters.

- Rounds. *The protocol has $\log \ell + \log n + 3$ rounds.*
- Proof length. *The prover sends $n + \log \ell \cdot (1 + \max\{\log n + 1, \log M + d\}) + 3 \cdot \log n + s + 3$ field elements.*
- Query complexity. *The verifier makes $\ell \cdot t$ queries.*
- Prover time. *The prover performs $O(\ell \cdot (n + d \cdot (|\hat{\mathbf{p}}| + M)) + s \cdot n + t \cdot \log n)$ field operations.*
- Verifier time. *The verifier performs $O(\ell \cdot (\log n + \log M + \kappa + t) + \log \ell \cdot d + (t + s) \cdot \log n)$ field operations.*

Suppose that $\text{PG}(2, \alpha) = (1 - \alpha, \alpha)$ is a strong proximity generator for $\mathcal{C}$ with proximity radius δ_{PG} and error err_{PG} . For every $\delta \in (0, \min\{\delta(\mathcal{C}), \delta_{\text{PG}}(2)\})$, if

$$|\mathbb{F}| \geq 2^\lambda \cdot \max \left\{ \begin{array}{l} (\log \ell + 1) \cdot |\Lambda(\mathcal{C}^\ell, \delta)|, \\ (1 + \max\{\log n + 1, \log M + d\}) \cdot |\Lambda(\mathcal{C}^\ell, \delta)| + \ell/2 \cdot \text{err}_{\text{PG}}(\mathcal{C}, 2, \delta) \cdot |\mathbb{F}|, \\ 2^{\lambda/s-1} \cdot |\Lambda(\mathcal{C}, \delta)|^{2/s} \cdot \log n, \\ |\Lambda(\mathcal{C}, \delta)| \cdot \max\{2, \log(1 + s + t)\} \end{array} \right\}, \quad (12)$$

and

$$t \geq \frac{\lambda}{-\log(1 - \delta)}, \quad (13)$$

the IOR has round-by-round knowledge soundness error $\leq 2^{-\lambda}$. Moreover, the total extraction time is $O(\ell \cdot \text{ecor}_C)$ field operations.

Proof. The IOR from $\mathcal{R}_C^\ell$ to $\mathcal{R}_C$ is the result of concatenating IORs described in prior sections.

- The IOR in Section 6 (Construction 6.3), which reduces from $\mathcal{R}_C^\ell$ to $\mathcal{R}_{C,1,\ell}$.
- The IOR in Section 7 (Construction 7.2), which reduces $\mathcal{R}_{C,1,\ell}$ to $\mathcal{R}_{C,1+s+t}$.
- The IOR in Section 8 (Construction 8.2), which reduces $\mathcal{R}_{C,1+s+t}$ to $\mathcal{R}_C$.

The claimed efficiency and security of this IOR is the result of careful bookkeeping across the three relevant theorems (Theorems 6.2, 7.1 and 8.1). $\square$

Theorem 2 in Section 1.1 is an informal statement that summarizes the IORs provided by Theorem 5.8 (IOR from $\mathcal{R}_\odot(\mathbb{F})$ to $\mathcal{R}_C$, given Remark 5.9) and Theorem 9.1 (IOR from $\mathcal{R}_C^\ell$ to $\mathcal{R}_C$).

Remark 9.2. Theorem 9.1 depends on the codeword length n , but not the encoding time of $\mathcal{C}$. The encoding time only matters for Theorem 5.8.

Remark 9.3. Theorem 9.1 mandates that $n, M, \ell, 1 + s + t \in 2^{\mathbb{N}}$. The restriction can be straightforwardly lifted by standard techniques (e.g., adding dummy claims) with only a constant impact on the asymptotics.

Lifting abstractions. The IOR from $\mathcal{R}_c^\ell$ to $\mathcal{R}_c$ underlying Theorem 9.1 is obtained by combining three IORs. Below we make explicit the resulting IOR by lifting abstraction barriers between the IORs. After that we make explicit the straightline state-restoration extractor that is obtained by concatenating each round's round-by-round extractor.

Construction 9.4. We describe the IOR from $\mathcal{R}_c^\ell$ to $\mathcal{R}_c$ underlying Theorem 9.1.

- **Inputs.** For $\mathbf{i} \in \{0, 1\}^{\log \ell}$, the verifier receives $(\alpha_{\mathbf{i}}, \mu_{\mathbf{i}}, \beta_{\mathbf{i}}, \eta_{\mathbf{i}})$ and has oracle access to $f_{\mathbf{i}}: [n] \rightarrow \mathbb{F}$. In the honest case, for every $\mathbf{i} \in \{0, 1\}^{\log \ell}$, $(\mathbf{i}, (\alpha_{\mathbf{i}}, \mu_{\mathbf{i}}, \beta_{\mathbf{i}}, \eta_{\mathbf{i}}), f_{\mathbf{i}}) \in \mathcal{L}(\mathcal{R}_c)$.
- **Interaction phase.**

1. **Zerocheck randomness:** The verifier samples $\tau \leftarrow \mathbb{F}^{\log \ell}$ and $\omega \leftarrow \mathbb{F}$.
2. **Sumcheck protocol:** Define the following $(\log \ell)$ -variate multilinear polynomials:

$$\begin{aligned}
\hat{F}(\mathbf{I}) &:= \sum_{\mathbf{i} \in \{0, 1\}^{\log \ell}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot f_{\mathbf{i}} && \in (\mathbb{F}^{<2}[\mathbf{I}])^n, \\
\hat{w}(\mathbf{I}) &:= \sum_{\mathbf{i} \in \{0, 1\}^{\log \ell}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot \mathcal{C}^{-1}(f_{\mathbf{i}}) && \in (\mathbb{F}^{<2}[\mathbf{I}])^k, \\
\hat{A}(\mathbf{I}) &:= \sum_{\mathbf{i} \in \{0, 1\}^{\log \ell}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot \alpha_{\mathbf{i}} && \in (\mathbb{F}^{<2}[\mathbf{I}])^{\log n}, \\
\hat{B}(\mathbf{I}) &:= \sum_{\mathbf{i} \in \{0, 1\}^{\log \ell}} \text{eq}(\mathbf{I}, \mathbf{i}) \cdot \beta_{\mathbf{i}} && \in (\mathbb{F}^{<2}[\mathbf{I}])^m.
\end{aligned} \tag{14}$$

Note that $\hat{w}$ is well-defined in the honest case and, assuming the code is systematic, equivalent to the first k entries of $\hat{F}$. Define the following:

$$\begin{aligned}
\hat{G}(\mathbf{F}, \mathbf{W}, \mathbf{A}, \mathbf{B}) &:= \left(\sum_{\mathbf{b} \in \{0, 1\}^{\log n}} \text{eq}(\mathbf{A}, \mathbf{b}) \cdot F_{\mathbf{b}} \right) + \omega \cdot \hat{P}(\mathbf{B}, \mathbf{W}), \\
\sigma^{(1)} &:= \sum_{\mathbf{i} \in \{0, 1\}^{\log \ell}} \text{eq}(\tau, \mathbf{i}) \cdot (\mu_{\mathbf{i}} + \omega \cdot \eta_{\mathbf{i}}).
\end{aligned} \tag{15}$$

The prover and verifier engage in a $(\log \ell)$ -round sumcheck protocol for the following claim:

$$\sum_{\mathbf{i} \in \{0, 1\}^{\log \ell}} \text{eq}(\tau, \mathbf{i}) \cdot \hat{G}(\hat{F}(\mathbf{i}), \hat{w}(\mathbf{i}), \hat{A}(\mathbf{i}), \hat{B}(\mathbf{i})) = \sigma^{(1)}.$$

During the interaction, the prover sends $\log \ell$ sumcheck polynomials $\hat{h}_1^{(1)}, \dots, \hat{h}_{\log \ell}^{(1)}$ of degree $1 + \max\{\log n + 1, \log M + d\}$, and the verifier replies with randomness $\gamma := (\gamma_1, \dots, \gamma_{\log \ell}) \in \mathbb{F}^{\log \ell}$. The prover and verifier set

$$\zeta_0 := \hat{A}(\gamma), \quad \beta := \hat{B}(\gamma).$$

3. **New oracle and target:** The prover sends $f: [n] \rightarrow \mathbb{F}$ and $\nu_0, \eta \in \mathbb{F}$. In the honest case,

$$f := \sum_{i \in \{0,1\}^{\log \ell}} \text{eq}(\gamma, i) \cdot f_i, \quad \nu_0 := \hat{f}(\zeta_0), \quad \eta := \hat{P}(\beta, \mathcal{C}^{-1}(f)).$$

4. **OOD samples:** The verifier samples $\zeta_1, \dots, \zeta_s \leftarrow \mathbb{F}^{\log n}$.

5. **OOD answers:** The prover sends $\nu_1, \dots, \nu_s \in \mathbb{F}$. In the honest case, for every $k \in [s]$, $\nu_k := \hat{f}(\zeta_k)$.

6. **Shift queries and zerocheck randomness:** The verifier samples $x_1, \dots, x_t \leftarrow [n]$ and $\xi \leftarrow \mathbb{F}^{\log r}$. For every $k \in [t]$, the verifier queries $(f_i)_{i \in \{0,1\}^{\log \ell}}$ at x_k and sets

$$\nu_{s+k} := \sum_{i \in \{0,1\}^{\log \ell}} \text{eq}(\gamma, i) \cdot f_i(x_k).$$

Set $r := 1 + s + t$. For $k \in [t]$, define $\zeta_{s+k} := \text{binary}(x_k) \in \mathbb{F}^{\log n}$. In the rest of the protocol, we abuse notation by writing $\zeta_{\text{binary}(i)} := \zeta_i$ and $\nu_{\text{binary}(i)} := \nu_i$ for $i \in \{0, \dots, r-1\}$.

7. **Sumcheck batching:** Define the following:

$$\text{eq}^*(\mathbf{X}) := \sum_{i \in \{0,1\}^{\log r}} \text{eq}(i, \xi) \cdot \text{eq}(\zeta_i, \mathbf{X}), \quad \sigma^{(2)} := \sum_{i \in \{0,1\}^{\log r}} \text{eq}(i, \xi) \cdot \nu_i. \quad (16)$$

The prover and verifier engage in a $(\log n)$ -round sumcheck protocol for the following claim:

$$\sum_{\mathbf{a} \in \{0,1\}^{\log n}} \text{eq}^*(\mathbf{a}) \cdot \hat{f}(\mathbf{a}) = \sigma^{(2)}.$$

During the interaction, the prover sends quadratic sumcheck polynomials $\hat{h}_1^{(2)}, \dots, \hat{h}_{\log n}^{(2)}$, and the verifier sends randomness $\alpha \in \mathbb{F}^{\log n}$.

8. **New target:** The prover sends $\mu \in \mathbb{F}$. In the honest case, $\mu := \hat{f}(\alpha)$.

• **Decision phase.** (The verifier rejects if any check fails.)

1. **Sumcheck decisions:** The verifier checks that

- $\hat{h}_1^{(1)}(0) + \hat{h}_1^{(1)}(1) = \sigma^{(1)}$ and $\hat{h}_1^{(2)}(0) + \hat{h}_1^{(2)}(1) = \sigma^{(2)}$;
- $\forall j \in \{1, \dots, \log \ell - 1\}$, $\hat{h}_{j+1}^{(1)}(0) + \hat{h}_{j+1}^{(1)}(1) = \hat{h}_j^{(1)}(\gamma_j)$; and
- $\forall j \in \{1, \dots, \log n - 1\}$, $\hat{h}_{j+1}^{(2)}(0) + \hat{h}_{j+1}^{(2)}(1) = \hat{h}_j^{(2)}(\alpha_j)$.

2. **New target decision:** The verifier checks that

$$\hat{h}_{\log \ell}^{(1)}(\gamma_{\log \ell}) = \text{eq}(\tau, \gamma) \cdot (\nu_0 + \omega \cdot \eta) \quad \text{and} \quad \hat{h}_{\log n}^{(2)}(\alpha_{\log n}) = \mu \cdot \text{eq}^*(\alpha).$$

3. **Outputs:** The verifier outputs the new explicit instance $\mathbf{x}' := (\alpha, \mu, \beta, \eta)$ and implicit instance $\mathbf{y}' := f$.

Construction 9.5. We describe the straightline state-restoration extractor $\mathbf{E}_{\text{sr}}$ for the IOR from $\mathcal{R}_c^\ell$ to $\mathcal{R}_c$ in Construction 9.4 (see Definition B.2). Below, $\mathfrak{i} = (\hat{\mathbf{p}}, M, N, k)$, $\mathbf{x} = ((\alpha_i, \mu_i, \beta_i, \eta_i))_{i \in \{0,1\}^{\log \ell}}$, and $\mathbf{y} = (f_i)_{i \in \{0,1\}^{\log \ell}}$.

$\mathbf{E}_{\text{sr}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, (\pi_i)_{i \in [k]}, \mathfrak{w}', (\rho_i)_{i \in [k]}, \text{tr}_{\text{sr}})$:

1. Parse the verifier randomness $(\rho_i)_{i \in [k]}$ according to the different types of randomness per round according to Construction 9.4:

$$((\boldsymbol{\tau}, \boldsymbol{\omega}), \gamma_1, \dots, \gamma_{\log \ell}, (\boldsymbol{\sigma}_1, \dots, \boldsymbol{\sigma}_s), (x_1, \dots, x_t, \boldsymbol{\xi}), x_1, \dots, x_{\log n}).$$

Let $\boldsymbol{\gamma} := (\gamma_1, \dots, \gamma_{\log \ell}) \in \mathbb{F}^{\log \ell}$ be the first sumcheck protocol's randomness.

2. Identify the subset of indices $S \subseteq [n]$ on which $\mathfrak{w}' \in \mathcal{C}$ agrees with $\sum_{i \in \{0,1\}^{\log \ell}} \text{eq}(\boldsymbol{\gamma}, \mathfrak{i}) \cdot f_i$.
3. For every $\mathfrak{i} \in \{0,1\}^{\log \ell}$, erasure correct the restriction $f_i(S)$ using $\mathbf{E}_{\mathcal{C}}$ (see Definition 3.9) to obtain a codeword $u_{\mathfrak{i}} \in \mathcal{C}$; if correction fails, the extractor aborts.
4. Output $\mathfrak{w} := (u_{\mathfrak{i}})_{\mathfrak{i} \in \{0,1\}^{\log \ell}}$.

Construction 9.5 is functionally equivalent to the concatenation of each round's round-by-round extractors, consistent with Construction B.5 (used to prove Theorem B.4). Indeed, the round-by-round extractors for Construction 7.2 and Construction 8.2 are the identity function, and the above extractor results from the composition of the round-by-round extractors for Construction 6.3.

10 Unbounded accumulation from any linear code

We present the accumulation scheme implied by our IORs. In Section 10.1 we recall the definition of an accumulation scheme in an oracle model (such as the ROM) for a relation $\mathcal{R}$; our definition is specialized to straightline extraction. Then in Section 10.2 we state our main result about an accumulation scheme for the PESAT relation $\mathcal{R}_\odot(\mathbb{F})$ (Definition 5.2). Note that in this section we consider *ternary* relations $\mathcal{R}$ (such as $\mathcal{R}_\odot(\mathbb{F})$), which consist of triples $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w})$; since the verifier does not have query access to its inputs, there is no implicit instance $\mathfrak{y}$.

10.1 Accumulation schemes in oracle models

A tuple of algorithms $\text{ACC} = (\mathbf{I}_{\text{ACC}}, \mathbf{P}_{\text{ACC}}, \mathbf{V}_{\text{ACC}}, \mathbf{D}_{\text{ACC}})$ is an *accumulation scheme* in the oracle model $\mathcal{D}$ for a (ternary) relation $\mathcal{R}$ if it satisfies completeness and knowledge soundness as defined below.

Definition 10.1. *ACC is (perfectly) complete if, for every security parameter λ_{ro} for the oracle, index $\mathfrak{i}$, instance-witness pairs $(\mathfrak{x}_i, \mathfrak{w}_i)_{i \in [\ell_1]}$, and accumulators $(\text{acc}_j)_{j \in [\ell_2]}$,*

$$\Pr \left[\begin{array}{l} \forall i \in [\ell_1] : (\mathfrak{i}, \mathfrak{x}_i, \mathfrak{w}_i) \in \mathcal{R} \\ \wedge \forall j \in [\ell_2] : \mathbf{D}_{\text{ACC}}^{\text{ro}}(\mathfrak{i}, \text{acc}_j) = 1 \\ \Downarrow \\ \mathbf{V}_{\text{ACC}}^{\text{ro}} \left(\begin{array}{c} \text{vk}_{\text{ACC}}, (\mathfrak{x}_i)_{i \in [\ell_1]}, \\ (\text{acc}_j \cdot \mathfrak{x})_{j \in [\ell_2]}, \\ \text{acc} \cdot \mathfrak{x}, \text{pf} \end{array} \right) = 1 \\ \wedge \mathbf{D}_{\text{ACC}}^{\text{ro}}(\mathfrak{i}, \text{acc}) = 1 \end{array} \right] = 1.$$

Definition 10.2. *ACC has straightline knowledge soundness error κ_{ACC} with extraction time et_{ACC} if there exists a relaxed decider $\hat{\mathbf{D}}_{\text{ACC}}$ (i.e., $\mathbf{D}_{\text{ACC}}$ accepting implies $\hat{\mathbf{D}}_{\text{ACC}}$ accepting) and deterministic extractor $\mathbf{E}_{\text{ACC}}$ such that, for every security parameter λ_{ro} for the oracle, index size bound $n_{\mathfrak{i}}$, instance size bound $n_{\mathfrak{x}}$, accumulator size bound n_{acc} , number of instance-witness pairs ℓ_1 , number of accumulators ℓ_2 , and stateful adversary $\tilde{\mathbf{P}}$ that performs at most t_{ro} queries to the random oracle, $\mathbf{E}_{\text{ACC}}$ runs in time at most $\text{et}_{\text{ACC}}(\lambda_{\text{ro}}, t_{\text{ro}}, n_{\mathfrak{i}}, n_{\mathfrak{x}}, n_{\text{acc}}, \ell_1, \ell_2)$ and*

$$\Pr \left[\begin{array}{l} |\mathfrak{i}| \leq n_{\mathfrak{i}} \\ \wedge \forall i \in [\ell_1] : |\mathfrak{x}_i| \leq n_{\mathfrak{x}} \\ \wedge \forall j \in [\ell_2] : |\text{acc}_j \cdot \mathfrak{x}| \leq n_{\text{acc}} \\ \wedge \mathbf{V}_{\text{ACC}}^{\text{ro}} \left(\begin{array}{c} \text{vk}_{\text{ACC}}, (\mathfrak{x}_i)_{i \in [\ell_1]}, \\ (\text{acc}_j \cdot \mathfrak{x})_{j \in [\ell_2]}, \\ \text{acc} \cdot \mathfrak{x}, \text{pf} \end{array} \right) = 1 \\ \wedge \hat{\mathbf{D}}_{\text{ACC}}^{\text{ro}}(\mathfrak{i}, \text{acc}) = 1 \\ \Downarrow \\ \exists i \in [\ell_1] : (\mathfrak{i}, \mathfrak{x}_i, \mathfrak{w}_i) \notin \mathcal{R} \\ \vee \exists j \in [\ell_2] : \hat{\mathbf{D}}_{\text{ACC}}^{\text{ro}}(\mathfrak{i}, \text{acc}_j) \neq 1 \end{array} \right]$$

is at most $\kappa_{\text{ACC}}(\lambda_{\text{ro}}, t_{\text{ro}}, n_{\mathfrak{i}}, n_{\mathfrak{x}}, n_{\text{acc}}, \ell_1, \ell_2)$.

10.2 WARP: our accumulation scheme

Our IORs directly imply a corresponding accumulation scheme in (a suitable configuration of) the random oracle model (ROM), which we call WARP.

Building blocks. The theorem below is stated with respect to building blocks from prior work.

- **Merkle commitments.** We rely on a Merkle commitment scheme MT in the ROM, as described in [CY24]. This is a tuple of algorithms

$$\text{MT} = (\text{MT.Commit}, \text{MT.Open}, \text{MT.Check})$$

in a setting where the random oracle ro_{MT} is sampled from $\mathcal{U}(\sigma_{\text{MT}})$ (the uniform distribution over functions from $\{0, 1\}^*$ to $\{0, 1\}^{\sigma_{\text{MT}}}$). We rely on the (multi-)extraction property of MT , which is quantified via two functions as follows (under minor conditions on the input parameters):

- $\kappa_{\text{MT}}(\sigma_{\text{MT}}, t_{\text{MT}}, \ell_{\text{MT}}, n_{\text{MT}}, k_{\text{MT}}) = O(\frac{t_{\text{MT}}^2}{2^{\sigma_{\text{MT}}}})$ is the (multi-)extraction error; and
- $\text{et}_{\text{MT}}(\sigma_{\text{MT}}, \Sigma_{\text{MT}}, \ell_{\text{MT}}, t_{\text{MT}}, n_{\text{MT}}) = O(n_{\text{MT}} \cdot t_{\text{MT}} \cdot \ell_{\text{MT}} \cdot (\log |\Sigma_{\text{MT}}| + \sigma_{\text{MT}}))$ is the extraction time.

The input parameters are the following: (i) σ_{MT} is the output size of ro_{MT} ; (ii) t_{MT} is the adversary's query bound to ro_{MT} ; (iii) Σ_{MT} is the message alphabet; (iv) ℓ_{MT} is the message length; (v) n_{MT} is the number of commitments output by the adversary; and (vi) k_{MT} is the number of openings output by the adversary. See [CY24] for more details; here we ignore the salt size (it is 0) because we do not rely on hiding properties of MT .

- **Fiat–Shamir transformation.** We rely on a Fiat–Shamir transformation FS in the ROM for suitably deriving the randomness for the verifiers in the used IORs. For convenience of presentation (and implementation), we use the duplex-sponge variant in [CO25], which is a tuple of algorithms

$$\text{FS} = (\text{FS.Init}, \text{FS.Absorb}, \text{FS.Squeeze})$$

where FS.Init initializes a state given an input string, FS.Absorb updates the state given an input string, and FS.Squeeze outputs derived randomness given an input state. This duplex-sponge variant is in a setting where the random oracle is an ideal permutation, and its efficiency and security depend on the permutation's alphabet Σ_{FS} , capacity c_{FS} , and rate r_{FS} . Accordingly, we consider a random oracle ro_{FS} sampled from $\mathcal{S}(\Sigma_{\text{FS}}, c_{\text{FS}}, r_{\text{FS}})$, the uniform distribution over permutations on $\Sigma_{\text{FS}}^{r_{\text{FS}} + c_{\text{FS}}}$. The security of FS is quantified via two functions as follows (under minor conditions on the input parameters):

- $\kappa_{\text{FS}}(t_{\text{FS}}) = O(\frac{t_{\text{FS}}^2}{|\Sigma_{\text{FS}}|^{c_{\text{FS}}}})$ is the extraction error; and
- $\text{et}_{\text{FS}}(t_{\text{FS}}, n) = O(t_{\text{FS}}^2 \cdot \log t_{\text{FS}} \cdot ((r_{\text{FS}} + c_{\text{FS}}) \cdot \log |\Sigma_{\text{FS}}| + n))$ is the extraction time.

The input parameters are the following: (i) t_{FS} is the adversary's query bound to ro_{FS} ; (ii) n is a bound on instance size. See [CO25] for more details.

One technicality is that FS needs to know how to convert from prover messages to strings over Σ_{FS} and from strings over Σ_{FS} to verifier messages. We leave this implicit, and assume that this information is hardcoded, i.e., FS knows in each round (determined by the sequence of absorbs/squeezes) what is to be absorbed/squeezed. Explicitly handling these translations involves specifying a *codec*, and accounting for negligible errors introduced by translation biases.

Our result. We consider the random oracle model given by the following distribution:

$$\mathcal{D}(\lambda_{\text{ro}}) := \left\{ \text{ro} : \begin{array}{l} \text{ro}_{\text{MT}} \leftarrow \mathcal{U}(\sigma_{\text{MT}}) \\ \text{ro}_{\text{FS}} \leftarrow \mathcal{S}(\Sigma_{\text{FS}}, c_{\text{FS}}, r_{\text{FS}}) \\ \text{ro} := (\text{ro}_{\text{MT}}, \text{ro}_{\text{FS}}) \end{array} \right\},$$

where we think of $\sigma_{\text{MT}}, \Sigma_{\text{FS}}, c_{\text{FS}}, r_{\text{FS}}$ as (implicitly) functions of the security parameter λ_{ro} . Note that $t_{\text{MT}} + t_{\text{FS}} \leq t_{\text{ro}}$, where $t_{\text{MT}}, t_{\text{FS}}, t_{\text{ro}}$ are the adversary's query bounds to $\text{ro}_{\text{MT}}, \text{ro}_{\text{FS}}, \text{ro}$ respectively. We keep track of the query bounds to each oracle for the sake of concrete security bounds. Finally, note that $\mathcal{D}(\lambda_{\text{ro}})$ can be “implemented” via a single random oracle sampled from $\mathcal{U}(\lambda_{\text{ro}})$ (a “plain” ROM) but we do not do so since the above oracle distribution is closer to practice.¹¹

Theorem 10.3. Fix $\mathbb{F}, \mathcal{C}, \mathbb{I}, s, t$ as in Theorem 9.1; recall that $\kappa := N - k$. Construction 10.4 is an accumulation scheme in the ROM

$$\text{ACC} = (\mathbf{I}_{\text{ACC}}, \mathbf{P}_{\text{ACC}}, \mathbf{V}_{\text{ACC}}, \mathbf{D}_{\text{ACC}})$$

for the PESAT relation $\mathcal{R}_{\odot}(\mathbb{F})$ (Definition 5.2) with the following parameters. (Below ℓ_1 and ℓ_2 are the number of statements and accumulators being accumulated, respectively; we let $\ell := \ell_1 + \ell_2$.)

- Prover cost:
 - $O(\ell_1 \cdot \text{enc}_{\mathcal{C}} + \ell \cdot (n + d \cdot (|\hat{\mathbf{p}}| + M)) + s \cdot n + t \cdot \log n)$ field operations,
 - absorbing/squeezing $O(\ell_1 \cdot \log M + \log \ell \cdot \max\{\log n, \log M + d\} + s \cdot \log n + \log(t + s))$ field elements and $O(\ell_1 \cdot n_{\mathbf{x}} + \ell_2 \cdot n_{\text{acc}} + \sigma_{\text{MT}} + t \cdot \log n)$ bits via queries to the FS random oracle, and
 - $O(n)$ queries to the MT random oracle of size $\ell_1 \cdot \log |\mathbb{F}|$ and σ_{MT} .
- Verifier cost:
 - $O(\ell \cdot (\log n + \log M + \kappa + t) + \log \ell \cdot d + (t + s) \cdot \log n)$ field operations,
 - absorbing/squeezing the same information as the prover (via queries to the FS random oracle),
 - $O(t)$ queries to the MT random oracle of size $\ell_1 \cdot \log |\mathbb{F}|$,
 - $O(\ell_2 \cdot t)$ queries to the MT random oracle of size $\log |\mathbb{F}|$, and
 - $O(\ell_2 \cdot t \cdot \log n)$ queries to the MT random oracle of size σ_{MT} .
- Decider cost:
 - $O(|\hat{\mathbf{p}}| + \text{enc}_{\mathcal{C}})$ field operations, and
 - $O(n)$ queries to the MT random oracle of size $\log |\mathbb{F}|$ and σ_{MT} .
- Security. If Equations 1, 12 and 13 (in Theorems 5.8 and 9.1) hold then ACC has straightline knowledge soundness error

$$\begin{aligned} \kappa_{\text{ACC}}(\lambda_{\text{ro}}, t_{\text{ro}}, n_{\mathbb{I}}, n_{\mathbf{x}}, n_{\text{acc}}, \ell_1, \ell_2) &\leq (t_{\text{FS}} + \log \ell + \log n + 4) \cdot 2^{-\lambda} \\ &\quad + \kappa_{\text{MT}}(\sigma_{\text{MT}}, t_{\text{MT}}, n, t_{\text{FS}} \cdot (\ell_2 + 2), \ell_2 + 2) \\ &\quad + \kappa_{\text{FS}}(t_{\text{FS}}) \end{aligned} \tag{17}$$

with extraction time

$$\begin{aligned} \text{et}_{\text{ACC}} &\leq O(\ell_1 \cdot \text{dec}_{\mathcal{C}} + \ell \cdot \text{ecor}_{\mathcal{C}}) \text{ field operations} \\ &\quad + \text{et}_{\text{MT}}(\sigma_{\text{MT}}, \mathbb{F}^{\ell_1}, n, t_{\text{MT}}, t_{\text{FS}} \cdot (\ell_2 + 2)) + \text{et}_{\text{FS}}(t_{\text{FS}}, \ell_1 \cdot n_{\mathbf{x}} + \ell_2 \cdot n_{\text{acc}}) \text{ bit operations.} \end{aligned} \tag{18}$$

Theorem 1 in Section 1.1 is an informal statement that summarizes ACC from Theorem 10.3, after having fixed a linear code $\mathcal{C}$ with linear encoding time and constant relative distance over a sufficiently large finite field (e.g., the expander code in [GLSTW23] and the repeat-accumulate-accumulate code in [BCFRZ25]).

We prove Theorem 10.3 by following the strategy in [BMNW25b], which adapts the IOP-to-SNARG transformation of [BCS16] to the setting of accumulation schemes. We construct an

¹¹Implementing ro_{FS} in the plain ROM can be done via a Feistel network [FNS75], while ro_{MT} can be implemented with standard input/output extension techniques. Domain separation should be used to separate the two oracles.

IOR from $\mathcal{R}_\circ(\mathbb{F})^{\ell_1} \times \mathcal{R}_c^{\ell_2}$ to $\mathcal{R}_c$ by composing ℓ_1 parallel invocations of Construction 5.10 with 1 invocation of Construction 9.4 where $\ell := \ell_1 + \ell_2$. [BMNW25b] shows how to compile such an IOR into a corresponding non-interactive reduction, which can be recast as an accumulation scheme for $\mathcal{R}_\circ(\mathbb{F})$.¹² For convenience, in Construction 10.4 we make the construction explicit, and then elaborate on the (efficiency and security) parameters claimed in the theorem statement.

Construction 10.4 (WARP). Below we describe each algorithm of the accumulation scheme

$$\text{ACC} = (\mathbf{I}_{\text{ACC}}, \mathbf{P}_{\text{ACC}}, \mathbf{V}_{\text{ACC}}, \mathbf{D}_{\text{ACC}}).$$

The construction relies on a Merkle commitment scheme MT and a Fiat–Shamir transformation FS, which are the two building blocks described above.

- $\mathbf{I}_{\text{ACC}}^{\text{ro}}(\mathfrak{i})$
 1. Parse $\mathfrak{i}$ as a PESAT index $(\hat{\mathbf{p}}, M, N, k)$.
 2. Set $\text{st}_{\text{FS}} := \text{FS.Init}^{\text{roFS}}(\mathfrak{i})$.
 3. Set $\text{pk}_{\text{ACC}} := (\text{st}_{\text{FS}}, \hat{\mathbf{p}}, M, N, k)$.
 4. Set $\text{vk}_{\text{ACC}} := (\text{st}_{\text{FS}}, M, N, k)$.
 5. Return $(\text{pk}_{\text{ACC}}, \text{vk}_{\text{ACC}})$.
- $\mathbf{P}_{\text{ACC}}^{\text{ro}}(\text{pk}_{\text{ACC}}, (\mathfrak{x}_i, \mathfrak{w}_i)_{i \in [\ell_1]}, (\text{acc}_j)_{j \in [\ell_2]}):$
 1. **Parsing phase.**
 - (a) **Index:** parse pk_{ACC} as $(\text{st}_{\text{FS}}, \hat{\mathbf{p}}, M, N, k)$.
 - (b) **Statements:** for every $i \in [\ell_1]$, parse $\mathfrak{x}_i$ as $\mathbf{x}_i \in \mathbb{F}^{N-k}$ and $\mathfrak{w}_i$ as $\mathbf{w}_i \in \mathbb{F}^k$.
 - (c) **Accumulators:** for every $j \in [\ell_2]$, parse $\text{acc}_j.\mathfrak{x}$ as $(\text{rt}_j, \boldsymbol{\alpha}_{\ell_1+j}, \mu_{\ell_1+j}, \boldsymbol{\beta}_{\ell_1+j}, \eta_{\ell_1+j})$ and $\text{acc}_j.\mathfrak{w}$ as $(\text{td}_j, f_{\ell_1+j}, \mathbf{w}_{\ell_1+j})$.
 - (d) **Absorb parameters:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}(((\mathfrak{x}_i)_{i \in [\ell_1]}, (\text{acc}_j.\mathfrak{x})_{j \in [\ell_2]}), \text{st}_{\text{FS}})$.
 2. **PESAT reduction.**
 - (a) **Encode witnesses:** for every $i \in [\ell_1]$, set $f_i := \mathcal{C}(\mathbf{w}_i)$.
 - (b) **Evaluation claims:** for every $i \in [\ell_1]$, set $\boldsymbol{\alpha}_i := 0^{\log n}$ and $\mu_i := \hat{f}_i(\boldsymbol{\alpha}_i)$.¹³
 - (c) **Commit to witnesses:** set $(\text{rt}_0, \text{td}_0) := \text{MT.Commit}^{\text{roMT}}((f_1, \dots, f_{\ell_1}))$.
 - (d) **Absorb commitment and code evaluations:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}((\text{rt}_0, (\mu_i)_{i \in [\ell_1]}), \text{st}_{\text{FS}})$.
 - (e) **Zerocheck randomness:** set $((\boldsymbol{\tau}_1, \dots, \boldsymbol{\tau}_{\ell_1}), \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $\boldsymbol{\tau}_i \in \mathbb{F}^{\log M}$ for $i \in [\ell_1]$.
 - (f) **Bundle evaluations:** for every $i \in [\ell_1]$, set $\boldsymbol{\beta}_i := (\mathbf{x}_i, \boldsymbol{\tau}_i)$ and $\eta_i := 0$.
 3. **Constrained code accumulation.**
 - (a) **Zerocheck randomness:** set $((\boldsymbol{\tau}, \omega), \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $\boldsymbol{\tau} \in \mathbb{F}^{\log \ell}$ and $\omega \in \mathbb{F}$.
 - (b) For every $i \in [\ell]$, define $f_{\text{binary}(i-1)} := f_i$, $\mathbf{w}_{\text{binary}(i-1)} := \mathbf{w}_i$, $\boldsymbol{\alpha}_{\text{binary}(i-1)} := \boldsymbol{\alpha}_i$, $\mu_{\text{binary}(i-1)} := \mu_i$, $\boldsymbol{\beta}_{\text{binary}(i-1)} := \boldsymbol{\beta}_i$, $\eta_{\text{binary}(i-1)} := \eta_i$.

¹²In [BMNW25b], accumulation schemes are defined for non-interactive arguments (rather than relations). In our case the non-interactive argument is trivial: the prover sends the witness and the verifier checks it.

¹³Construction 5.10 is a reduction from $\mathcal{R}_\circ(\mathbb{F})$ to $\mathcal{R}_{c,0}$, while Construction 9.4 instead expects as input instances of $\mathcal{R}_c = \mathcal{R}_{c,1}$. Here, the prover converts an instance of $\mathcal{R}_{c,0}$ into an instance of $\mathcal{R}_c$ by revealing the evaluation claims $(\mu_i)_{i \in [\ell_1]}$, as mentioned in Remark 5.9.

- (c) **Sumcheck protocol:** define the $(\log \ell)$ -variate multilinear polynomials $\hat{F}(\mathbf{I})$, $\hat{w}(\mathbf{I})$, $\hat{A}(\mathbf{I})$, $\hat{B}(\mathbf{I})$ as in Equation 14 and $\hat{G}(\mathbf{F}, \mathbf{W}, \mathbf{A}, \mathbf{B})$ as in Equation 15.
- (d) For every $i \in [\log \ell]$:
- i. Set $\gamma_{i-1} := (\gamma_1, \dots, \gamma_{i-1}) \in \mathbb{F}^{i-1}$.
 - ii. **Sumcheck polynomial:** compute

$$\hat{h}_i^{(1)}(\mathbf{X}) := \sum_{\mathbf{i} \in \{0,1\}^{\log \ell - i}} \text{eq}(\tau, (\gamma_i, \mathbf{X}, \mathbf{i})) \cdot \hat{G}(\hat{F}(\gamma_i, \mathbf{X}, \mathbf{i}), \hat{w}(\gamma_i, \mathbf{X}, \mathbf{i}), \hat{A}(\gamma_i, \mathbf{X}, \mathbf{i}), \hat{B}(\gamma_i, \mathbf{X}, \mathbf{i})).$$

- iii. **Absorb sumcheck polynomial:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}(\hat{h}_i^{(1)}, \text{st}_{\text{FS}})$.
 - iv. **Sumcheck randomness:** set $(\gamma_i, \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $\gamma_i \in \mathbb{F}$.
- (e) **New oracle and target:** set $\gamma := (\gamma_1, \dots, \gamma_{\log \ell})$, $\zeta_0 := \hat{A}(\gamma)$, $\beta := \hat{B}(\gamma)$ and

$$f := \sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\gamma, \mathbf{i}) \cdot f_{\mathbf{i}}, \quad \mathbf{w} := \sum_{\mathbf{i} \in \{0,1\}^{\log \ell}} \text{eq}(\gamma, \mathbf{i}) \cdot \mathbf{w}_{\mathbf{i}}, \quad \nu_0 := \hat{f}(\zeta_0), \quad \eta := \hat{P}(\beta, \mathbf{w}).$$

- (f) **New commitment:** compute $(\text{rt}, \text{td}) := \text{MT.Commit}^{\text{roMT}}(f)$.
- (g) **Absorb new commitment and target:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}((\text{rt}, \nu_0, \eta), \text{st}_{\text{FS}})$.
- (h) **OOD samples:** set $((\zeta_1, \dots, \zeta_s), \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $\zeta_i \in \mathbb{F}^{\log n}$ for $i \in [s]$.
- (i) **OOD answers:** for every $i \in [s]$, set $\nu_i := \hat{f}(\zeta_i)$.
- (j) **Absorb OOD answers:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}((\nu_1, \dots, \nu_s), \text{st}_{\text{FS}})$.
- (k) **Shift queries and zerocheck randomness:** set $((x_1, \dots, x_t, \xi), \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $x_i \in [n]$ for $i \in [t]$ and $\xi \in \mathbb{F}^{\log r}$. For every $k \in [t]$, define $\zeta_{s+k} := \text{binary}(x_k) \in \mathbb{F}^{\log n}$. Set $r := 1 + s + t$. In the rest of the protocol, we abuse notation by writing $\zeta_{\text{binary}(i)} := \zeta_i$ and $\nu_{\text{binary}(i)} := \nu_i$ for $i \in \{0, \dots, r-1\}$.
- (l) For every $i \in [\log n]$:
- i. **Sumcheck polynomials:** compute

$$\hat{h}_i^{(2)}(\mathbf{X}) := \sum_{\mathbf{i} \in \{0,1\}^{\log n - i}} \text{eq}^*(\alpha_1, \dots, \alpha_{i-1}, \mathbf{X}, \mathbf{i}) \cdot \hat{f}(\alpha_1, \dots, \alpha_{i-1}, \mathbf{X}, \mathbf{i}),$$

where eq^* is defined as in Equation 16.

- ii. **Absorb sumcheck polynomials:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}(\hat{h}_i^{(2)}, \text{st}_{\text{FS}})$.
 - iii. **Sumcheck randomness:** set $(\alpha_i, \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $\alpha_i \in \mathbb{F}$.
- (m) **New target:** set $\alpha := (\alpha_1, \dots, \alpha_{\log n})$ and compute $\mu := \hat{f}(\alpha)$.
- (n) **Compute authentication paths:**
- $\text{auth}_0 := \text{MT.Open}(\text{td}_0, (x_1, \dots, x_t))$,
 - $\forall j \in [\ell_2], \text{auth}_j := \text{MT.Open}(\text{td}_j, (x_1, \dots, x_t))$.

4. **Return** (acc, pf) where $\text{acc}.\mathbb{X} := (\text{rt}, \alpha, \mu, \beta, \eta)$, $\text{acc}.\mathbb{W} := (\text{td}, f, \mathbf{w})$, and

$$\text{pf} := \left(\text{rt}_0, (\mu_i)_{i \in [\ell_1]}, (\hat{h}_i^{(1)})_{i \in [\log \ell]}, \nu_0, (\nu_i)_{i \in [s]}, (\hat{h}_i^{(2)})_{i \in [\log n]}, \text{auth}_0, (\text{auth}_j)_{j \in [\ell_2]}, (f_i(x_j))_{i \in [\ell], j \in [t]} \right).$$

- $\mathbf{V}_{\text{ACC}}^{\text{ro}}(\text{vk}_{\text{ACC}}, (\mathbb{X}_i)_{i \in [\ell_1]}, (\text{acc}_j.\mathbb{X})_{j \in [\ell_2]}, \text{acc}.\mathbb{X}, \text{pf})$:

1. **Parsing phase.**

- (a) **Verification key:** parse vk_{ACC} as $(\text{st}_{\text{FS}}, M, N, k)$.
- (b) **Instances:** for every $i \in [\ell_1]$ parse $\mathbb{x}_i$ as $\mathbf{x}_i \in \mathbb{F}^{N-k}$.
- (c) **Accumulators:** for every $j \in [\ell_2]$ parse $\text{acc}_j.\mathbb{x}$ as $(\text{rt}_j, \alpha_{\ell_1+j}, \mu_{\ell_1+j}, \beta_{\ell_1+j}, \eta_{\ell_1+j})$.
- (d) **Final accumulator:** parse $\text{acc}.\mathbb{x}$ as $(\text{rt}, \alpha, \mu, \beta, \eta)$.
- (e) **Accumulation proof:** parse pf as

$$\left(\text{rt}_0, (\mu_i)_{i \in [\ell_1]}, (\hat{h}_i^{(1)})_{i \in [\log \ell]}, \nu_0, (\nu_i)_{i \in [s]}, (\hat{h}_i^{(2)})_{i \in [\log n]}, \text{auth}_0, (\text{auth}_j)_{j \in [\ell_2]}, (v_{i,j})_{i \in [t], j \in [\ell]} \right).$$

- (f) **Absorb parameters:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}((\mathbb{x}_i)_{i \in [\ell_1]}, (\text{acc}_j.\mathbb{x})_{j \in [\ell_2]}, \text{st}_{\text{FS}})$

2. **Derive randomness.**

- (a) **Absorb commitment:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}(\text{rt}_0, \text{st}_{\text{FS}})$.
- (b) **Zerocheck:** set $((\tau_1, \dots, \tau_{\ell_1}), \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $\tau_i \in \mathbb{F}^{\log M}$ for $i \in [\ell_1]$.
- (c) **Absorb code evaluations:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}((\mu_i)_{i \in [\ell_1]}, \text{st}_{\text{FS}})$.
- (d) **Zerocheck:** set $((\tau, \omega), \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $\tau \in \mathbb{F}^{\log \ell}$ and $\omega \in \mathbb{F}$.
- (e) **Sumcheck:** For every $i \in [\log \ell]$:
 - i. **Absorb sumcheck polynomials:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}(\hat{h}_i^{(1)}, \text{st}_{\text{FS}})$.
 - ii. **Sumcheck randomness:** set $(\gamma_i, \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $\gamma_i \in \mathbb{F}$.
- (f) **Absorb new commitment and target:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}(\text{rt}, \nu_0, \eta), \text{st}_{\text{FS}})$.
- (g) **OOD samples:** set $((\zeta_1, \dots, \zeta_s), \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $\zeta_i \in \mathbb{F}^{\log n}$ for $i \in [s]$.
- (h) **Absorb OOD answers:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}((\nu_1, \dots, \nu_s), \text{st}_{\text{FS}})$.
- (i) **Shift queries and zerocheck:** set $((x_1, \dots, x_t, \xi), \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $x_i \in [n]$ for $i \in [t]$ and $\xi \in \mathbb{F}^{\log r}$.
- (j) **Sumcheck:** For every $i \in [\log n]$:
 - i. **Absorb sumcheck polynomial:** set $\text{st}_{\text{FS}} := \text{FS.Absorb}^{\text{roFS}}(\hat{h}_i^{(2)}, \text{st}_{\text{FS}})$.
 - ii. **Sumcheck randomness:** set $(\alpha_i, \text{st}_{\text{FS}}) := \text{FS.Squeeze}^{\text{roFS}}(\text{st}_{\text{FS}})$ where $\alpha_i \in \mathbb{F}$.

3. **Derive values.**

- (a) For every $i \in [\ell]$, set $\alpha_{\text{binary}(i-1)} := \alpha_i$, $\mu_{\text{binary}(i-1)} := \mu_i$, $\beta_{\text{binary}(i-1)} := \beta_i$, and $\eta_{\text{binary}(i-1)} := \eta_i$.
- (b) Set $\gamma = (\gamma_1, \dots, \gamma_{\log \ell})$ and compute $\zeta_0 := \hat{A}(\gamma)$ where $\hat{A}(\mathbf{I})$ is defined as in Equation 14. For every $k \in [t]$, set

$$\nu_{s+k} := \sum_{i \in \{0,1\}^{\log \ell}} \text{eq}(\gamma, i) \cdot v_{k,i},$$

and define $\zeta_{s+k} := \text{binary}(x_k) \in \mathbb{F}^{\log n}$.

- (c) Set $r := 1 + s + t$. In the rest of the protocol, we abuse notation by writing $\zeta_{\text{binary}(i)} := \zeta_i$ and $\nu_{\text{binary}(i)} := \nu_i$ for $i \in \{0, \dots, r-1\}$.
- (d) Set $\sigma^{(1)} := \sum_{i \in \{0,1\}^{\log \ell}} \text{eq}(\tau, i) \cdot (\mu_i + \omega \cdot \eta_i)$ and $\sigma^{(2)} := \sum_{i \in \{0,1\}^{\log r}} \text{eq}(i, \xi) \cdot \nu_i$.

4. **Decision phase.** (The verifier rejects if any check fails.)

- (a) **New code evaluation point:** check that $\alpha = (\alpha_1, \dots, \alpha_{\log n})$.

- (b) **New circuit evaluation point:** check that $\beta = \hat{B}(\gamma)$, where $\hat{B}(\mathbf{I})$ is defined as in Equation 14.
- (c) **Check authentication paths:** check that
 - $\text{MT.Check}^{\text{roMT}}(\text{rt}_0, (x_i)_{i \in [t]}, ((v_{i,j})_{j \in [\ell_1]})_{i \in [t]}, \text{auth}_0) = 1$; and
 - $\forall j \in [\ell_2], \text{MT.Check}^{\text{roMT}}(\text{rt}_j, (x_i)_{i \in [t]}, (v_{i, \ell_1+j})_{i \in [t]}, \text{auth}_j) = 1$.
- (d) **Sumcheck decisions:** check that
 - $\hat{h}_1^{(1)}(0) + \hat{h}_1^{(1)}(1) = \sigma^{(1)}$ and $\hat{h}_1^{(2)}(0) + \hat{h}_1^{(2)}(1) = \sigma^{(2)}$;
 - $\forall j \in \{1, \dots, \log \ell - 1\}, \hat{h}_{j+1}^{(1)}(0) + \hat{h}_{j+1}^{(1)}(1) = \hat{h}_j^{(1)}(\gamma_j)$; and
 - $\forall j \in \{1, \dots, \log n - 1\}, \hat{h}_{j+1}^{(2)}(0) + \hat{h}_{j+1}^{(2)}(1) = \hat{h}_j^{(2)}(\alpha_j)$.
- (e) **New target decision:** check that

$$\hat{h}_{\log \ell}^{(1)}(\gamma_{\log \ell}) = \text{eq}(\tau, \gamma) \cdot (\nu_0 + \omega \cdot \eta), \quad \hat{h}_{\log n}^{(2)}(\alpha_{\log n}) = \mu \cdot \text{eq}^*(\alpha),$$

where eq^* is defined as in Equation 16.

- $\mathbf{D}_{\text{ACC}}^{\text{ro}}(\mathfrak{i}, \text{acc})$:
 1. Parse $\mathfrak{i}$ as $(\hat{\mathbf{p}}, M, N, k)$.
 2. Parse acc.x as $(\text{rt}, \alpha, \mu, \beta, \eta)$ and acc.w as $(\text{td}, f, \mathbf{w})$.
 3. Check the following:
 - (a) $(\text{rt}, \text{td}) = \text{MT.Commit}^{\text{roMT}}(f)$.
 - (b) $\hat{f}(\alpha) = \mu$, $\hat{P}(\beta, \mathbf{w}) = \eta$, and $f = \mathcal{C}(\mathbf{w})$.

Proof of Theorem 10.3. The accumulation scheme WARP in Construction 10.4 is the result of applying the (adapted) IOP-to-SNARG compilation [BCS16; BMNW25b] to an IOR from $\mathcal{R}_{\odot}(\mathbb{F})^{\ell_1} \times \mathcal{R}_{\mathcal{C}}^{\ell_2}$ to $\mathcal{R}_{\mathcal{C}}$, which we call IOR_{WARP} and now describe. This IOR results from concatenating two IORs:

- An IOR from $\mathcal{R}_{\odot}(\mathbb{F})^{\ell_1} \times \mathcal{R}_{\mathcal{C}}^{\ell_2}$ to $\mathcal{R}_{\mathcal{C}}^{\ell_1} \times \mathcal{R}_{\mathcal{C}}^{\ell_2} = \mathcal{R}_{\mathcal{C}}^{\ell}$, which is obtained by considering ℓ_1 parallel invocations of the IOR from $\mathcal{R}_{\odot}(\mathbb{F})$ to $\mathcal{R}_{\mathcal{C}}$ in Construction 5.10 (given Remark 5.9’s adaptation).
- The IOR from $\mathcal{R}_{\mathcal{C}}^{\ell}$ to $\mathcal{R}_{\mathcal{C}}$ in Construction 9.4.

We discuss the efficiency and security of WARP.

Efficiency. The efficiency parameters come from careful bookkeeping, and here we only highlight an efficiency feature coming from parallel execution. Since IOR_{WARP} involves ℓ_1 parallel instances of the IOR in Construction 5.10, the accumulation prover $\mathbf{P}_{\text{ACC}}$ commits to the ℓ_1 encoded witnesses via a *single* Merkle commitment with alphabet $\mathbb{F}^{\ell_1}$ (by stacking the encoded witnesses), instead of with ℓ_1 Merkle commitments with alphabet $\mathbb{F}$. Hence, the accumulation proof pf contains t authentication paths to this single Merkle commitment, as well as t authentication paths to the Merkle commitments in the ℓ_2 input accumulators. $\mathbf{V}_{\text{ACC}}$ then must only verify those authentication paths, via $O(\ell_2 \cdot t \cdot \log n)$ queries to the MT random oracle (which in the theorem statement we split according to query size). Note that if IOR_{WARP} were to run Construction 5.10 serially (rather than in parallel) then $\mathbf{V}_{\text{ACC}}$ would instead perform $O(\ell \cdot t \cdot \log n)$ queries to the MT random oracle.

Security. The security bounds follow from [BMNW25b; CY24; CO25]. The error term κ_{ACC} for ACC in Equation 17 is obtained as follows.

- The first error term is the state-restoration knowledge soundness error of IOR_{WARP} :

$$\begin{aligned}
& \varepsilon_{\text{sr}}(t_{\text{FS}}) \\
& \leq (t_{\text{FS}} + k) \cdot \varepsilon_{\text{rbr}} && \text{(by Theorem B.4)} \\
& \leq (t_{\text{FS}} + k) \cdot 2^{-\lambda} && \text{(since } \varepsilon_{\text{rbr}} \leq 2^{-\lambda} \text{)} \\
& = (t_{\text{FS}} + \log \ell + \log n + 4) \cdot 2^{-\lambda}. && \text{(since } k = \log \ell + \log n + 4 \text{)}
\end{aligned}$$

We outline why $\varepsilon_{\text{rbr}} \leq 2^{-\lambda}$.

The concatenation of two IORs (with matching output and input relations) yields an IOR with round-by-round knowledge soundness error equal to the maximum of the round-by-round knowledge soundness errors of the two IORs. IOR_{WARP} is an IOR from $\mathcal{R}_{\odot}(\mathbb{F})^{\ell_1} \times \mathcal{R}_c^{\ell_2}$ to $\mathcal{R}_c$ obtained as the concatenation of two IORs, each having error $\leq 2^{-\lambda}$, as discussed next.

- From $\mathcal{R}_{\odot}(\mathbb{F})^{\ell_1} \times \mathcal{R}_c^{\ell_2}$ to $\mathcal{R}_c^{\ell_1} \times \mathcal{R}_c^{\ell_2} = \mathcal{R}_c^{\ell}$. Construction 5.10 is an IOR from $\mathcal{R}_{\odot}(\mathbb{F})$ to $\mathcal{R}_c$ with error $\leq 2^{-\lambda}$ (Equation 1 holds). Repeating ℓ_1 times in parallel this IOR yields an IOR from $\mathcal{R}_{\odot}(\mathbb{F})^{\ell_1}$ to $\mathcal{R}_c^{\ell_1}$ with the same error: if at least one of the ℓ_1 input instances is not in $\mathcal{L}(\mathcal{R}_{\odot}(\mathbb{F}))$ then, except with a small error, at least one of the ℓ_1 output instances is far from $\mathcal{R}_c$; the same error is achieved by defining the knowledge state function of new IOR to be the disjunction of base IOR’s knowledge state function on the individual instances. Separately, the ℓ_2 instances for $\mathcal{R}_c^{\ell_2}$ are left untouched (in particular, no error is incurred for this).
 - From $\mathcal{R}_c^{\ell}$ to $\mathcal{R}_c$. Construction 9.4 is an IOR from $\mathcal{R}_c^{\ell}$ to $\mathcal{R}_c$ with error $\leq 2^{-\lambda}$ (Equations 12 and 13 hold).
- The second error term is the (multi-)extraction error $\kappa_{\text{MT}}(\sigma_{\text{MT}}, t_{\text{MT}}, n, t_{\text{FS}} \cdot (\ell_2 + 2), \ell_2 + 2)$ of the Merkle commitment scheme MT, where:
 - σ_{MT} is the output size of ro_{MT} ;
 - t_{MT} is the adversary’s query bound to ro_{MT} ;
 - n is the message length;
 - $t_{\text{FS}} \cdot (\ell_2 + 2)$ is the number of Merkle commitments output by the adversary; and
 - $\ell_2 + 2$ is the number of Merkle commitment extractions.

The reduction to state-restoration knowledge soundness of IOR_{WARP} produces, for each of $\leq t_{\text{FS}}$ state-restoration moves, $\ell_2 + 2$ IOR oracles (ℓ_2 oracles come from the ℓ_2 input $\mathcal{R}_c$ instances, one oracle over $\mathbb{F}^{\ell_1}$ is produced from the parallel executions of Construction 5.10, and one oracle comes from the function batching protocol Construction 6.3 executed within Construction 9.4). Overall, the reduction outputs $t_{\text{FS}} \cdot (\ell_2 + 2)$ Merkle commitments and performs $\ell_2 + 2$ Merkle commitment extractions, for Merkle commitments of size n .

- The third error term $\kappa_{\text{FS}}(t_{\text{FS}})$ is the Fiat–Shamir transformation extraction error.

Finally, the extraction time et_{ACC} for ACC in Equation 18 is obtained as follows. The accumulation extractor $\mathbf{E}_{\text{ACC}}$ runs in turn three extractors.

1. A Fiat–Shamir extractor that, given the malicious prover’s query-answer trace, reconstructs the “virtual” moves of the adversary in a state-restoration game for an interactive protocol where only Merkle commitments are used. This takes $\text{et}_{\text{FS}}(t_{\text{FS}}, \ell_1 \cdot n_{\text{x}} + \ell_2 \cdot n_{\text{acc}})$ bit operations.

2. A Merkle commitment extractor that is invoked on the moves of the adversary in the interactive protocol state-restoration game, to obtain corresponding moves for the IOR state restoration game. This takes $\text{et}_{\text{MT}}(\sigma_{\text{MT}}, \mathbb{F}^{\ell_1}, n, t_{\text{MT}}, t_{\text{FS}} \cdot (\ell_2 + 2))$ bit operations.
3. A state-restoration extractor $\mathbf{E}_{\text{sr}}$ for IOR_{WARP} that invokes two further state-restoration extractors: the extractor described in Construction 9.5 (which uses $O(\ell \cdot \text{ecor}_{\mathcal{C}})$ field operations) and ℓ_1 instances of the extractor in Construction 5.10 (each of which uses $O(\text{dec}_{\mathcal{C}})$ field operations). Overall, this takes $O(\ell_1 \cdot \text{dec}_{\mathcal{C}} + \ell \cdot \text{ecor}_{\mathcal{C}})$ field operations.

□

A Succinct arguments vs. accumulation schemes

A key efficiency metric of PCD is the *recursion overhead*: the additional computation whose correctness is proven beyond the PCD predicate’s correctness. The recursion overhead differs across PCD constructions. Below ℓ refers to the arity of the PCD predicate.

1. For PCD based on a succinct non-interactive argument $\text{ARG} = (\mathbf{P}_{\text{ARG}}, \mathbf{V}_{\text{ARG}})$, the recursion overhead is ℓ executions of $\mathbf{V}_{\text{ARG}}$.
2. For PCD based on an accumulation scheme $\text{ACC} = (\mathbf{P}_{\text{ACC}}, \mathbf{V}_{\text{ACC}}, \mathbf{D}_{\text{ACC}})$, the recursion overhead is 1 execution of $\mathbf{V}_{\text{ACC}}$ on ℓ instance-witness pairs and ℓ accumulators.

The latter is typically more efficient, and motivates the study of accumulation schemes. A detailed quantitative comparison, however, is more nuanced due to the fact that a succinct non-interactive argument leads to PCD via an alternate “indirect” construction that yields different tradeoffs.

A succinct non-interactive argument $\text{ARG} = (\mathbf{P}_{\text{ARG}}, \mathbf{V}_{\text{ARG}})$ implies a corresponding accumulation scheme $\text{ACC} = (\mathbf{P}_{\text{ACC}}, \mathbf{V}_{\text{ACC}}, \mathbf{D}_{\text{ACC}})$ with a trivial decider $\mathbf{D}_{\text{ACC}}$: (i) $\mathbf{P}_{\text{ACC}}$ runs $\mathbf{P}_{\text{ARG}}$ on the (conjunction of) ℓ instance-witness pairs; (ii) $\mathbf{V}_{\text{ACC}}$ runs $\mathbf{V}_{\text{ARG}}$ on the ℓ instances; and (iii) $\mathbf{D}_{\text{ACC}}$ always accepts. One can then construct PCD from this accumulation scheme ACC (following Item 2 above).¹⁴

In particular, if $\mathbf{P}_{\text{ARG}}$ is linear time then so is $\mathbf{P}_{\text{ACC}}$, which directly yields constructions of linear-time accumulation schemes from corresponding linear-time succinct non-interactive arguments [BCGGHJ17; BCG20; BCL22; XZS22; RR22; GLSTW23; BCFRRZ25; NST24; BMMS25]. The corresponding accumulation schemes are *incomparable* to WARP in the following sense. (Below λ is the security parameter and, for concreteness, we consider the PESAT relation from Definition 1.)

- The verifier $\mathbf{V}_{\text{ACC}}$ of accumulation schemes obtained via direct “non-succinct” constructions typically runs in time $\ell \cdot f(\lambda, k)$ for some function f (think $f = O_\lambda(\log k)$).
- The verifier $\mathbf{V}_{\text{ACC}}$ of accumulation schemes obtained from succinct arguments typically run in time $g(\lambda, \ell \cdot (M + N))$ for some function $g = \omega(f)$.

These costs are incomparable in that the first approach is better for small to medium arities ℓ (e.g., IVC corresponds to $\ell = 1$), whereas the second approach is better for larger arities ℓ . In this paper we focus on direct accumulation schemes (and the regime where they are better).

¹⁴This indirect construction is *not the same* as the direct PCD construction: recursive proof composition of ARG yields *succinct* PCD proofs whereas PCD based on ACC does not. On the other hand, using ACC leads to a smaller recursion overhead, as we informally explain next. Let $\mathbf{V}_{\text{ARG}}(|C|)$ denote the complexity of $\mathbf{V}_{\text{ARG}}$ on a circuit of size $|C|$. In the direct PCD construction, the recursion overhead is $\ell \cdot \mathbf{V}_{\text{ARG}}(|C|)$. In the indirect PCD construction the recursion overhead is $\mathbf{V}_{\text{ARG}}(\ell \cdot |C|)$. Hence the indirect construction may be beneficial to recursion overhead: $\mathbf{V}_{\text{ARG}}(\ell \cdot |C|)$ can be smaller than $\ell \cdot \mathbf{V}_{\text{ARG}}(|C|)$ depending on the arity ℓ .

B RBR knowledge soundness implies SR knowledge soundness

We prove that (our relaxation of) round-by-round knowledge soundness (Section 4) implies straightline state-restoration knowledge soundness. For notational convenience, let $\mathcal{R}_{\leq \delta}$ denote the set of statements up to δ -far from some relation $\mathcal{R}$:

$$\mathcal{R}_{\leq \delta} := \{(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \mathfrak{w}) \mid \exists \mathfrak{y}_\star \text{ s.t. } (\mathfrak{i}, \mathfrak{x}, \mathfrak{y}_\star, \mathfrak{w}) \in \mathcal{R} \text{ and } \Delta(\pi_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}), \pi_{\mathcal{R}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}_\star)) \leq \delta\}.$$

This matches the definition of a knowledge state function, in the sense that if $\text{tr} = \emptyset$ is the empty transcript, then $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, \mathfrak{w}) = 1$ if and only if $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \mathfrak{w}) \in \mathcal{R}_{\leq \delta}$. Similarly, if tr is a full transcript, then $\text{KState}_\delta(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \text{tr}, \mathfrak{w}) = 1$ if and only if $\mathbf{V}$ outputs $(\mathfrak{x}', \mathfrak{y}')$ such that $(\mathfrak{i}, \mathfrak{x}', \mathfrak{y}', \mathfrak{w}) \in \mathcal{R}'_{\leq \delta}$.

Definition B.1. *The IOR state restoration game with salt size s , functions $\text{rnd} = (\text{rnd}_i)_{i \in [k]}$ and adversary $\tilde{\mathbf{P}}_{\text{sr}}$, denoted $\text{SRGame}(s, \text{rnd}, \tilde{\mathbf{P}}_{\text{sr}})$, is defined as follows.*

1. Repeat the following until $\tilde{\mathbf{P}}_{\text{sr}}$ exits the loop:
 - (a) $\tilde{\mathbf{P}}_{\text{sr}}$ sends a move $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, (\pi_1, \dots, \pi_i), (\sigma_1, \dots, \sigma_i))$ where $i \in [k]$, $(\pi_1, \dots, \pi_i)$ are prover messages for the first i rounds, and $(\sigma_1, \dots, \sigma_i)$ are salt strings in $\{0, 1\}^s$.
 - (b) Set $\rho_i := \text{rnd}_i(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, (\pi_1, \dots, \pi_i), (\sigma_1, \dots, \sigma_i))$.
 - (c) Give ρ_i as response to $\tilde{\mathbf{P}}_{\text{sr}}$.
2. $\tilde{\mathbf{P}}_{\text{sr}}$ outputs $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, (\pi_i)_{i \in [k]}, (\sigma_i)_{i \in [k]}, \mathfrak{w}')$ where $(\pi_i)_{i \in [k]}$ are prover messages for all rounds, $(\sigma_i)_{i \in [k]}$ are salt strings in $\{0, 1\}^s$, and $\mathfrak{w}'$ is a (candidate) witness.
3. For every $i \in [k]$, set $\rho_i := \text{rnd}_i(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, (\pi_1, \dots, \pi_i), (\sigma_1, \dots, \sigma_i))$.
4. Output $(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, (\pi_i)_{i \in [k]}, (\sigma_i)_{i \in [k]}, \mathfrak{w}', (\rho_i)_{i \in [k]})$.

We denote by tr_{sr} the list of move-response pairs of the form $((\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, (\pi_1, \dots, \pi_i), (\sigma_1, \dots, \sigma_i)), \rho_i)$ in the loop.

Definition B.2. *A k -round IOR $(\mathbf{I}, \mathbf{P}, \mathbf{V})$ from a relation $\mathcal{R}$ to $\mathcal{R}'$ with proximity radius $\delta^\star$ has **straightline state-restoration knowledge soundness** for a statement set Z with error ε_{sr} and extraction time et if there exists a deterministic extractor $\mathbf{E}_{\text{sr}}$ that runs in time at most et with the following property: for every salt size $s \in \mathbb{N}$, move budget $t \in \mathbb{N}$, proximity bound $\delta \in (0, \delta^\star)$, and t -move deterministic state-restoration prover $\tilde{\mathbf{P}}_{\text{sr}}$,*

$$\Pr \left[\begin{array}{l} (\mathfrak{i}, \mathfrak{x}, \mathfrak{y}) \in Z \\ \wedge (\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, \mathfrak{w}) \notin \mathcal{R}_{\leq \delta} \\ \wedge (\mathfrak{i}, \mathfrak{x}', \mathfrak{y}', \mathfrak{w}') \in \mathcal{R}'_{\leq \delta} \end{array} \middle| \begin{array}{l} \text{rnd} = (\text{rnd}_i)_{i \in [k]} \leftarrow \mathcal{U}((r_i)_{i \in [k]}) \\ (\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, (\pi_i)_{i \in [k]}, (\sigma_i)_{i \in [k]}, \mathfrak{w}', (\rho_i)_{i \in [k]}) \xleftarrow{\text{tr}_{\text{sr}}} \\ \text{SRGame}(s, \text{rnd}, \tilde{\mathbf{P}}_{\text{sr}}) \\ (\mathfrak{x}', \mathfrak{y}') \leftarrow \mathbf{V}^{\mathfrak{y}, (\pi_i)_{i \in [k]}}(\mathbf{I}(\mathfrak{i}), \mathfrak{x}, (\rho_i)_{i \in [k]}) \\ \mathfrak{w} \leftarrow \mathbf{E}_{\text{sr}}(\mathfrak{i}, \mathfrak{x}, \mathfrak{y}, (\pi_i)_{i \in [k]}, (\sigma_i)_{i \in [k]}, \mathfrak{w}', (\rho_i)_{i \in [k]}, \text{tr}_{\text{sr}}) \end{array} \right] \leq \varepsilon_{\text{sr}}(s, t, \delta).$$

(Note that $\mathbf{E}_{\text{sr}}$ does not need $\mathfrak{x}'$ and $\mathfrak{y}'$ as inputs, as they are determined by $\mathbf{E}_{\text{sr}}$'s other inputs.)

Remark B.3. [BMNW25b] generalizes the notion of straightline state-restoration knowledge soundness [BCS16; CY24] from IOPs to IORs. There are minor differences between Definitions B.1 and B.2 and those in [BMNW25b].

- [BMNW25b] consider an IOR indexer in full generality, which outputs a new index for $\mathcal{R}'$ and an index string to which the verifier has oracle access. These additional features do not meaningfully affect the state restoration definitions.

- We give the state-restoration extractor the random challenges from the state-restoration game's output transcript, whereas [BMNW25b] does not.
- [BMNW25b] additionally allows the state-restoration prover to erase characters in the new implicit instance y' . This is a technical detail required to model the capabilities of a non-interactive prover (which can analogously omit Merkle path openings). For proximity claims (and more generally, *monotone relations* as defined in [BMNW25b]), erasing characters cannot help the state-restoration prover. In this case, any bound on the state-restoration error in Definition B.2 translates into a bound on the state-restoration error in [BMNW25b]'s definition.

Theorem B.4. *Let $\text{IOR} = (\mathbf{I}, \mathbf{P}, \mathbf{V})$ be a k -round IOR with round-by-round knowledge soundness with errors $(\varepsilon_1, \dots, \varepsilon_k)$ and extraction times $(\text{et}_1, \dots, \text{et}_k)$ (Definition 4.2). Let Z be an arbitrary statement set and define $\varepsilon_{\text{rbr}}(\delta) := \max_{i \in [k], (\mathbb{i}, \mathbb{x}, \mathbb{y}) \in Z} \varepsilon_i(\mathbb{i}, \mathbb{x}, \mathbb{y}, \delta)$. Then IOR has straightline state-restoration knowledge soundness for the statement set Z (Definition B.2) with error*

$$\varepsilon_{\text{sr}}(s, t, \delta) \leq (t + k) \cdot \varepsilon_{\text{rbr}}(\delta)$$

and extraction time $\text{et} \leq \sum_{i \in [k]} \text{et}_i$. (The upper bounds are unaffected by salt size.)

Construction B.5. Let $\mathbf{E}_{\text{rbr}}$ be the round-by-round knowledge extractor for $(\mathbf{I}, \mathbf{P}, \mathbf{V})$. We construct a state-restoration extractor $\mathbf{E}_{\text{sr}}$ as follows.

- $\mathbf{E}_{\text{sr}}(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_i)_{i \in [k]}, (\sigma_i)_{i \in [k]}, \mathbb{w}', (\rho_i)_{i \in [k]}, \text{tr}_{\text{sr}})$:
1. Define $\mathbb{w}_k := \mathbb{w}'$.
 2. For $i = k, \dots, 1$: compute $\mathbb{w}_{i-1} \leftarrow \mathbf{E}_{\text{rbr}}(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \rho_1, \dots, \pi_i, \rho_i), \mathbb{w}_i)$.
 3. Output $\mathbb{w} := \mathbb{w}_0$.

The time of $\mathbf{E}_{\text{sr}}$ equals the sum of the k invocations of $\mathbf{E}_{\text{rbr}}$.

Proof. The state-restoration game's output is of the form $(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_i)_{i \in [k]}, (\sigma_i)_{i \in [k]}, \mathbb{w}', (\rho_i)_{i \in [k]})$. Define the *full trace* to be the concatenation of the state-restoration game trace and the move-response pairs

$$\left((\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \dots, \pi_i), (\sigma_1, \dots, \sigma_i)), \rho_i \right)_{i \in [k]},$$

namely, the move-response pairs obtained from each prefix of the state-restoration game's output transcript. The full trace contains at most $t + k$ moves.

For each $a \in [t + k]$, we define the indicator variable X_a as follows. Consider the a -th unique move in the full trace, denoted by

$$\left((\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \dots, \pi_i), (\sigma_1, \dots, \sigma_i)), \rho_i \right).$$

For $j < i$, let $\rho_j := \text{rnd}_j(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \dots, \pi_j), (\sigma_1, \dots, \sigma_j))$. We let $X_a = 1$ if and only if there exists $\mathbb{w}$ such that the following holds:

- $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \rho_1, \dots, \pi_i), \mathbf{E}_{\text{rbr}}(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \rho_1, \dots, \pi_i, \rho_i), \mathbb{w})) = 0$.
- $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \rho_1, \dots, \pi_i, \rho_i), \mathbb{w}) = 1$.

Let $X := \bigvee_{a \in [t+k]} X_a$. For every $i \in [k]$ and $a \in [t + k]$, we define $Y_{i,a}$ to be the indicator variable for the event where the a -th unique move in the full trace is for round i .

Claim B.6. Consider the state-restoration experiment in Definition B.2, where the state-restoration game outputs $(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_i)_{i \in [k]}, (\sigma_i)_{i \in [k]}, \mathbb{w}', (\rho_i)_{i \in [k]})$, the IOR verifier outputs $(\mathbb{x}', \mathbb{y}')$, and the state-restoration extractor outputs $\mathbb{w}$. If $(\mathbb{i}, \mathbb{x}, \mathbb{y}, \mathbb{w}) \notin \mathcal{R}_{\leq \delta}$ and $(\mathbb{i}, \mathbb{x}', \mathbb{y}', \mathbb{w}') \in \mathcal{R}'_{\leq \delta}$, then $X = 1$.

Proof. For each $i \in [k]$, let $a^{(i)} \in [t + k]$ be such that the $a^{(i)}$ -th unique move in the full trace is equal to

$$((\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \dots, \pi_i), (\sigma_1, \dots, \sigma_i)), \rho_i);$$

this is guaranteed to exist by construction of the full trace. Assume that $(\mathbb{i}, \mathbb{x}, \mathbb{y}, \mathbb{w}) \notin \mathcal{R}_{\leq \delta}$ and $(\mathbb{i}, \mathbb{x}', \mathbb{y}', \mathbb{w}') \in \mathcal{R}'_{\leq \delta}$. We show there exists $i \in [k]$ where $X_{a^{(i)}} = 1$; the claim follows.

Define $\mathbb{w}_k := \mathbb{w}'$. Since $(\mathbb{i}, \mathbb{x}', \mathbb{y}', \mathbb{w}') \in \mathcal{R}_{\leq \delta}$, by the definition of a knowledge state function (Definition 4.1), it must be that $\text{KState}(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \rho_1, \dots, \pi_k, \rho_k), \mathbb{w}_k) = 1$. Define $\mathbb{w}_{k-1} := \mathbf{E}_{\text{rbr}}(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \rho_1, \dots, \pi_k, \rho_k), \mathbb{w}_k)$. If $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \rho_1, \dots, \pi_k), \mathbb{w}_{k-1}) = 0$, then $X_{a^{(k)}} = 1$ and we are done. Otherwise, by the definition of a knowledge state function (in particular, the requirement for prover moves), it must be that $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \rho_1, \dots, \pi_{k-1}, \rho_{k-1}), \mathbb{w}_{k-1}) = 1$.

This argument continues until we arrive at the situation where $\text{KState}_\delta(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \rho_1), \mathbb{w}_1) = 1$. Define $\mathbb{w}_0 := \mathbf{E}_{\text{rbr}}(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \rho_1), \mathbb{w}_1)$ and observe that the state-restoration extractor outputs $\mathbb{w}$ which is precisely equal to $\mathbb{w}_0$. Since $(\mathbb{i}, \mathbb{x}, \mathbb{y}, \mathbb{w}) \notin \mathcal{R}_{\leq \delta}$, it must be that $\text{KState}(\mathbb{i}, \mathbb{x}, \mathbb{y}, \emptyset, \mathbb{w}_0) = 0$. Then $X_{a^{(1)}} = 1$ and we are done. $\square$

Claim B.6 implies that the state-restoration knowledge error is at most

$$\Pr \left[\begin{array}{l} (\mathbb{i}, \mathbb{x}, \mathbb{y}) \in Z \\ X = 1 \end{array} \left| \begin{array}{l} \text{rnd} = (\text{rnd}_i)_{i \in [k]} \leftarrow \mathcal{U}((r_i)_{i \in [k]}) \\ (\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_i)_{i \in [k]}, (\sigma_i)_{i \in [k]}, \mathbb{w}', (\rho_i)_{i \in [k]}) \xleftarrow{\text{tr}_{\text{sr}}} \\ \text{SRGame}(\text{rnd}, \mathbf{P}_{\text{sr}}) \\ (\mathbb{x}', \mathbb{y}') \leftarrow \mathbf{V}^{\mathbb{y}, (\pi_i)_{i \in [k]}}(\mathbf{I}(\mathbb{i}), \mathbb{x}, (\rho_i)_{i \in [k]}) \\ \mathbb{w} \leftarrow \mathbf{E}_{\text{sr}}(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_i)_{i \in [k]}, (\sigma_i)_{i \in [k]}, \mathbb{w}', (\rho_i)_{i \in [k]}, \text{tr}_{\text{sr}}) \end{array} \right. \right].$$

Fix any $i \in [k]$ and $a \in [t + k]$, and consider the execution of the state-restoration prover conditioned on $Y_{i,a} = 1$. Note that the event $Y_{i,a}$ depends only on the moves and responses up to the a -th unique move, not including its response. In the a -th move, the state-restoration prover outputs some move $(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \dots, \pi_i), (\sigma_1, \dots, \sigma_i))$. Since the state-restoration game has not yet answered this move (here we use the fact that the move is unique), the prover's output is independent of the response $\rho_i := \text{rnd}_i(\mathbb{i}, \mathbb{x}, \mathbb{y}, (\pi_1, \dots, \pi_i), (\sigma_1, \dots, \sigma_i))$. By the definition of round-by-round knowledge soundness (Definition 4.2), we have

$$\Pr[(\mathbb{i}, \mathbb{x}, \mathbb{y}) \in Z \wedge X_a = 1 \mid Y_{i,a} = 1] \leq \varepsilon_{\text{rbr}}(\delta).$$

Since this bound holds for any $i \in [k]$, it follows that $\Pr[(\mathbb{i}, \mathbb{x}, \mathbb{y}) \in Z \wedge X_a = 1] \leq \varepsilon_{\text{rbr}}$. Taking a union bound over $a \in [t + k]$ yields the desired bound. $\square$

C Comparison with prior notions of RBR knowledge soundness

We prove that our variant of round-by-round knowledge soundness (Section 4) is a relaxation of prior definitions [CMS19; BMNW25b], in the sense that any IOR that is knowledge sound under prior definitions is also knowledge sound under our variant (with the same error). As this is not essential to our results, we use the informal definitions from Section 2.8.

Theorem C.1 (informal). *If an IOR has round-by-round knowledge soundness error ε_{rbr} per Definition 2.5, then the IOR has round-by-round knowledge soundness error ε_{rbr} per Definition 2.7.*

Proof. Fix an IOR from $\mathcal{R}$ to $\mathcal{R}'$ with proximity radius δ^* . Let **State** be a state function and **E**_{old} be an extractor under which the IOR has round-by-round knowledge soundness error ε_{rbr} , per Definition 2.5.

We define a knowledge state function **KState** as follows.

- *Empty transcript:* if $\text{tr} = \emptyset$ is the empty transcript, then $\text{KState}_\delta(\mathbf{z}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if there exists $\mathbf{y}_\star \in B_\delta(\mathbf{y})$ with $(\mathbf{z}, \mathbf{y}_\star, \mathbf{w}) \in \mathcal{R}$.
- *Partial transcript:* If tr is a transcript that is neither empty nor full, then $\text{KState}_\delta(\mathbf{z}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $\text{State}_\delta(\mathbf{z}, \mathbf{y}, \text{tr}) = 1$ or there exists $\mathbf{y}_\star \in B_\delta(\mathbf{y})$ with $(\mathbf{z}, \mathbf{y}_\star, \mathbf{w}) \in \mathcal{R}$.
- *Full transcript:* if $\text{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\text{KState}_\delta(\mathbf{z}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $\mathbf{V}^{\mathbf{y}, (\pi_i)_{i \in [k]}}(\mathbf{z}, (\rho_i)_{i \in [k]})$ outputs $(\mathbf{z}', \mathbf{y}')$ such that there exists $\mathbf{y}'_\star \in B_\delta(\mathbf{y}')$ with $(\mathbf{z}', \mathbf{y}'_\star, \mathbf{w}) \in \mathcal{R}'$.

We also define the extractor $\mathbf{E}_{\text{new}}(\mathbf{z}, \mathbf{y}, \text{tr} \parallel \rho_i, \mathbf{w}) := \mathbf{E}_{\text{old}}(\mathbf{z}, \mathbf{y}, \text{tr})$, which runs the original extractor while ignoring the knowledge state witness (and the verifier's most recent message).

Claim C.2. *For every $\delta \in (0, \delta^*)$, statement $(\mathbf{z}, \mathbf{y})$, interaction transcript tr where the prover is about to move, and knowledge state witness $\mathbf{w}$, the following holds. If $\text{KState}_\delta(\mathbf{z}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$, then $\text{KState}_\delta(\mathbf{z}, \mathbf{y}, \text{tr} \parallel \pi, \mathbf{w}) = 0$ for every prover message π .*

Proof. Suppose that $\text{KState}_\delta(\mathbf{z}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$ and let π be an arbitrary prover message. Then for every $\mathbf{y}_\star \in B_\delta(\mathbf{y})$, $(\mathbf{z}, \mathbf{y}_\star, \mathbf{w}) \notin \mathcal{R}$. Moreover, $\text{State}(\mathbf{z}, \mathbf{y}, \text{tr}) = 0$ in both of the following cases.

- If $\text{tr} = \emptyset$ is the empty transcript then $\text{State}(\mathbf{z}, \mathbf{y}, \text{tr}) = 0$ follows from Definition 2.5 (in particular, the requirement for empty transcripts).
- If tr is not empty then, since the prover is about to move, tr is not full and $\text{State}(\mathbf{z}, \mathbf{y}, \text{tr}) = 0$ follows from the fact that $\text{KState}_\delta(\mathbf{z}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$.

From Definition 2.5 (in particular, the requirement for prover moves), it must be that $\text{State}(\mathbf{z}, \mathbf{y}, \text{tr} \parallel \pi) = 0$. We conclude that $\text{KState}_\delta(\mathbf{z}, \mathbf{y}, \text{tr} \parallel \pi, \mathbf{w}) = 0$. $\square$

Claim C.3. *For every $\delta \in (0, \delta^*)$, statement $(\mathbf{z}, \mathbf{y})$, interaction transcript tr where the verifier is about to move, and knowledge state witness $\mathbf{w}$,*

$$\Pr_{\rho_i} \left[\exists \mathbf{w} : \begin{array}{l} \text{KState}_\delta(\mathbf{z}, \mathbf{y}, \text{tr}, \mathbf{E}_{\text{new}}(\mathbf{z}, \mathbf{y}, \text{tr} \parallel \rho_i, \mathbf{w})) = 0 \\ \wedge \text{KState}_\delta(\mathbf{z}, \mathbf{y}, \text{tr} \parallel \rho_i, \mathbf{w}) = 1 \end{array} \right] \leq \varepsilon_{\text{rbr}}.$$

Proof. After unwrapping definitions, the claim's probability is equivalent to

$$\Pr_{\rho_i} \left[\begin{array}{l} \text{State}_\delta(\mathbf{z}, \mathbf{y}, \text{tr}) = 0 \\ \exists \mathbf{w} : \wedge \text{KState}_\delta(\mathbf{z}, \mathbf{y}, \text{tr} \parallel \rho_i, \mathbf{w}) = 1 \\ \wedge \forall \mathbf{y}_\star \in B_\delta(\mathbf{y}), (\mathbf{z}, \mathbf{y}_\star, \mathbf{E}_{\text{old}}(\mathbf{z}, \mathbf{y}, \text{tr} \parallel \rho_i)) \notin \mathcal{R} \end{array} \right].$$

If $\text{KState}_\delta(\mathbb{z}, \mathbb{y}, \text{tr} \parallel \rho_i, \mathbf{w}) = 1$ for some $\mathbf{w}$, then $\text{State}_\delta(\mathbb{z}, \mathbb{y}, \text{tr} \parallel \rho_i) = 1$. Hence, the above probability is upper bounded by

$$\Pr_{\rho_i} \left[\begin{array}{l} \text{State}_\delta(\mathbb{z}, \mathbb{y}, \text{tr}) = 0 \\ \wedge \text{State}_\delta(\mathbb{z}, \mathbb{y}, \text{tr} \parallel \rho_i) = 1 \\ \wedge \forall \mathbb{y}_\star \in B_\delta(\mathbb{y}), (\mathbb{z}, \mathbb{y}_\star, \mathbf{E}_{\text{old}}(\mathbb{z}, \mathbb{y}, \text{tr} \parallel \rho_i)) \notin \mathcal{R} \end{array} \right].$$

The claim follows from the fact that the IOR has round-by-round knowledge soundness error ε_{rbr} per Definition 2.5 under **State** and **E_{old}**. $\square$

By construction, **KState** satisfies the requirements for empty and full transcripts in Definition 2.7. In conjunction with Claims C.2 and C.3, this shows that the IOR has round-by-round knowledge soundness error ε_{rbr} per Definition 2.7. $\square$

D Extension fields

Throughout the paper we considered a PESAT relation $\mathcal{R}_\circ(\mathbb{F})$ and a linear code $\mathcal{C}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ defined over a field $\mathbb{F}$ that is required to be large enough for soundness. Here we discuss how to adapt our two main IORs to the setting of *field extensions*: the PESAT relation $\mathcal{R}_\circ(\mathbb{B})$ and linear code $\mathcal{C}_\mathbb{B}: \mathbb{B}^k \rightarrow \mathbb{B}^n$ are defined over a base field $\mathbb{B}$ (e.g., $\mathbb{B} = \mathbb{F}_2$), and we wish to benefit from a larger field extension $\mathbb{F}$ (e.g., $\mathbb{F} = \mathbb{F}_{2^{160}}$) for soundness. We wish to adjust our IORs so that the verifier samples field elements from $\mathbb{F}$ (rather than $\mathbb{B}$) while using $\mathbb{B}$ for the reduction from PESAT.

D.1 Definitions

We review definitions about extension fields and codes defined over extension fields.

Definition D.1. A tuple $(\mathbb{B}, \mathbb{F}, e, \psi, \varphi)$ is a **extension field presentation** if:

- $\mathbb{B}, \mathbb{F}$ are fields;
- $\psi: \mathbb{B} \hookrightarrow \mathbb{F}$ is an injective field homomorphism;
- $e \in \mathbb{N}$ is the dimension of $\mathbb{F}$ as a $\mathbb{B}$ -vector space;
- $\varphi: \mathbb{F} \rightarrow \mathbb{B}^e$ is a $\mathbb{B}$ -linear isomorphism.

For each $i \in [e]$, $\varphi_i: \mathbb{F} \rightarrow \mathbb{B}$ denotes the i -th component of φ . An extension field presentation $(\mathbb{B}, \mathbb{F}, e, \psi, \varphi)$ is **systematic** if, for every $x \in \mathbb{B}$, $\varphi(\psi(x)) = (x, 0, \dots, 0)$.

We also apply ψ to vectors of base field elements and $\varphi, \varphi_1, \dots, \varphi_e$ to vectors of extension field elements by applying each of these maps componentwise.

An extension code defines a code over a field extension for a linear code over the base field.

Definition D.2. Let $\mathcal{C}_\mathbb{B}: \mathbb{B}^k \rightarrow \mathbb{B}^n$ be a linear code. The **extension code of $\mathcal{C}_\mathbb{B}$ over $\mathbb{F}$** (with respect to the extension field presentation $(\mathbb{B}, \mathbb{F}, e, \psi, \varphi)$) is the linear code $\mathcal{C}_\mathbb{F}: \mathbb{F}^k \rightarrow \mathbb{F}^n$ defined as

$$\mathcal{C}_\mathbb{F}(\mathbf{v}) := \varphi^{-1}\left(\mathcal{C}_\mathbb{B}(\varphi_1(\mathbf{v})), \dots, \mathcal{C}_\mathbb{B}(\varphi_e(\mathbf{v}))\right).$$

If the extension field presentation is systematic, then $\mathcal{C}_\mathbb{F}(\psi(\mathbf{v})) = \psi(\mathcal{C}_\mathbb{B}(\mathbf{v}))$ for any $\mathbf{v} \in \mathbb{B}^k$.

The distance of the extension code $\mathcal{C}_\mathbb{F}$ equals the distance of the given code $\mathcal{C}_\mathbb{B}$, i.e., $\delta(\mathcal{C}_\mathbb{B}) = \delta(\mathcal{C}_\mathbb{F})$ [DP25, Thm 3.2].

The list size of the extension code is the list size of the e -interleaved base code (which in turn is related to the list size of the base code by Lemma 3.13).

Lemma D.3. Let $\mathcal{C}_\mathbb{B}: \mathbb{B}^k \rightarrow \mathbb{B}^n$ be a linear code and $(\mathbb{B}, \mathbb{F}, e, \psi, \varphi)$ be a extension field presentation. For every $\delta \in (0, 1)$, $|\Lambda(\mathcal{C}_\mathbb{F}, \delta)| = |\Lambda(\mathcal{C}_\mathbb{B}^e, \delta)|$.

Proof. We prove two inequalities.

- $|\Lambda(\mathcal{C}_\mathbb{F}, \delta)| \leq |\Lambda(\mathcal{C}_\mathbb{B}^e, \delta)|$.

Let $f: [n] \rightarrow \mathbb{F}$ and $u \in \Lambda(\mathcal{C}_\mathbb{F}, f, \delta)$. For every $i \in [e]$, define $f_i := \varphi_i \circ f$ and $u_i := \varphi_i \circ u$. We show that $(u_i)_{i \in [e]} \in \Lambda(\mathcal{C}_\mathbb{B}^e, (f_i)_{i \in [e]}, \delta)$. For every $i \in [e]$, observe that $u_i \in \mathcal{C}_\mathbb{B}$. Let $\mathbf{v} \in \mathbb{F}^k$ be such that $u = \mathcal{C}_\mathbb{F}(\mathbf{v})$. Then,

$$u_i = \varphi_i \circ u = \varphi_i \circ \mathcal{C}_\mathbb{F}(\mathbf{v}) = \varphi_i \circ \varphi^{-1}\left(\mathcal{C}_\mathbb{B}(\varphi_1(\mathbf{v})), \dots, \mathcal{C}_\mathbb{B}(\varphi_e(\mathbf{v}))\right) = \mathcal{C}_\mathbb{B}(\varphi_i(\mathbf{v})).$$

Since $\Delta(f, u) \leq \delta$, there exists $S \subseteq [n]$ with $|S| \geq (1 - \delta) \cdot n$ such that $f(x) = u(x)$ for every $x \in S$. Then, for every $x \in S$ and $i \in [e]$, $f_i(x) = \varphi_i \circ f(x) = \varphi_i \circ u(x) = u_i(x)$. We conclude that $\Delta((f_i)_{i \in [e]}, (u_i)_{i \in [e]}) \leq \delta$.

- $|\Lambda(\mathcal{C}_{\mathbb{F}}, \delta)| \geq |\Lambda(\mathcal{C}_{\mathbb{B}}^e, \delta)|$.

Let $(f_i)_{i \in [e]}: [n] \rightarrow \mathbb{B}^e$ and $(u_i) \in \Lambda(\mathcal{C}^e, (f_i)_{i \in [e]}, \delta)$. Define $f := \varphi^{-1} \circ (f_1, \dots, f_e)$ and $u := \varphi^{-1} \circ (u_1, \dots, u_e)$. We show that $u \in \Lambda(\mathcal{C}_{\mathbb{F}}, f, \delta)$. Observe that $u \in \mathcal{C}_{\mathbb{F}}$: for every $i \in [e]$, let $\mathbf{v}_i$ be such that $u_i = \mathcal{C}_{\mathbb{B}}(\mathbf{v}_i)$ and define $\mathbf{v} := \varphi^{-1}(\mathbf{v}_1, \dots, \mathbf{v}_e)$; then,

$$\mathcal{C}_{\mathbb{F}}(\mathbf{v}) = \varphi^{-1}(\mathcal{C}_{\mathbb{B}}(\varphi_1(\mathbf{v})), \dots, \mathcal{C}_{\mathbb{B}}(\varphi_e(\mathbf{v}))) = \varphi^{-1}(\mathcal{C}_{\mathbb{B}}(\mathbf{v}_1), \dots, \mathcal{C}_{\mathbb{B}}(\mathbf{v}_e)) = \varphi^{-1}(u_1, \dots, u_e) = u.$$

Since $\Delta((f_i)_{i \in [e]}, (u)_{i \in [e]}) \leq \delta$, there exists $S \subseteq [n]$ with $|S| \geq (1 - \delta) \cdot n$ such that $f_i(x) = u_i(x)$ for every $x \in S$ and $i \in [e]$. Since $f(x) = \varphi^{-1}(f_1(x), \dots, f_e(x)) = \varphi^{-1}(u_1(x), \dots, u_e(x)) = u(x)$ for every $x \in S$, we conclude that $\Delta(f, u) \leq \delta$.

□

D.2 Adapting the IORs

Let $(\mathbb{B}, \mathbb{F}, e, \psi, \varphi)$ be a systematic extension field presentation, and let $\mathcal{C}_{\mathbb{B}}: \mathbb{B}^k \rightarrow \mathbb{B}^n$ be a linear code with extension code $\mathcal{C}_{\mathbb{F}}$ over $\mathbb{F}$. We denote by $\mathcal{R}_{\mathcal{C}_{\mathbb{B}}, r}^{(\mathbb{F})}$ the relaxation of $\mathcal{R}_{\mathcal{C}_{\mathbb{B}}, r}$ (Definition 5.7) where $((\alpha_i, \mu_i)_{i \in [r]}, \beta, \eta)$ are defined over $\mathbb{F}$ instead of $\mathbb{B}$.

Reduction from PESAT. We adapt Construction 5.10 to be an IOR from $\mathcal{R}_{\odot}(\mathbb{B})$ to $\mathcal{R}_{\mathcal{C}_{\mathbb{B}}, 0}^{(\mathbb{F})}$ as follows. The prover encodes the witness $\mathbf{w} \in \mathbb{B}^k$ using $\mathcal{C}_{\mathbb{B}}$; this involves $O(\text{enc}_{\mathcal{C}_{\mathbb{B}}})$ operations in the *base field* $\mathbb{B}$, and the sent oracle $f: [n] \rightarrow \mathbb{B}$ is over the alphabet $\mathbb{B}$. The IOR verifier samples $\tau \leftarrow \mathbb{F}^{\log M}$. In the security analysis, Equation 1 in Theorem 5.8 changes to the following condition:

$$|\mathbb{F}| \geq 2^\lambda \cdot \log M \cdot |\Lambda(\mathcal{C}_{\mathbb{B}}, \delta)|.$$

Constrained code reduction. We adapt Construction 9.4 to be an IOR from $(\mathcal{R}_{\mathcal{C}_{\mathbb{B}}, 0}^{(\mathbb{F})})^{\ell_1} + \mathcal{R}_{\mathcal{C}_{\mathbb{F}}}^{\ell_2}$ to $\mathcal{R}_{\mathcal{C}_{\mathbb{F}}}$. This is identical to the IOR from $\mathcal{R}_{\mathcal{C}_{\mathbb{F}}}^{\ell}$ to $\mathcal{R}_{\mathcal{C}_{\mathbb{F}}}$ where $\ell := \ell_1 + \ell_2$, except some of the input oracles f_i are over the alphabet $\mathbb{B}$ instead of $\mathbb{F}$. Accordingly, the IOR verifier applies ψ to convert them to elements in the extension field $\mathbb{F}$. This is sound since, by virtue of the presentation being systematic, $\mathcal{C}_{\mathbb{F}}(\psi(\mathbf{v})) = \psi(\mathcal{C}_{\mathbb{B}}(\mathbf{v}))$ for every $\mathbf{v} \in \mathbb{B}^k$. The security analysis depends on the list size of $\mathcal{C}_{\mathbb{F}}$, which Lemma D.3 relates to the list size of (the e -interleaving of) $\mathcal{C}_{\mathbb{B}}$.

D.3 Adapting the accumulation scheme

Theorem 10.3 and Construction 10.4 can be adapted to handle base and extension fields by compiling the adapted IORs described in Appendix D.2. The soundness analysis is identical, and follows the sketch in Section 10.2. Concretely, this results in a verifier for ACC similar to that in the (unmodified) WARP construction, which additionally checks that opened locations are in the relevant alphabet of the Merkle commitment (base field elements in $\mathbb{B}$ or extension field elements in $\mathbb{F}$).

Remark D.4. Construction 10.4 benefits particularly from IOR_{WARP} running Construction 5.10 in parallel (rather than sequentially). Sequentially, committing to ℓ_1 encoded witnesses over the base field would involve $O(\ell_1 \cdot n)$ random oracle queries of size $\log |\mathbb{B}|$. By running Construction 5.10 in parallel instead, this cost is reduced to $O(n)$ random oracle queries of size $\ell_1 \cdot \log |\mathbb{B}|$. If $\ell_1 \approx e$, then this corresponds to the cost of committing to a *single* encoded witness over the extension field.

Acknowledgments

Alessandro Chiesa and Giacomo Fenzi are supported in part by the Ethereum Foundation and the Sui Foundation. Benedikt Bünz and William Wang are supported in part by Alpen Labs, Chaincode, Google, and the Sui Foundation.

References

- [ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. “STIR: Reed–Solomon Proximity Testing with Fewer Queries”. In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO ’24. 2024, pp. 380–413.
- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. “WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification”. In: *Proceedings of the 44th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’25. 2025.
- [AHIKV17] Benny Applebaum, Naama Haramaty, Yuval Ishai, Eyal Kushilevitz, and Vinod Vaikuntanathan. “Low-Complexity Cryptographic Hash Functions”. In: *Proceedings of the 8th Innovations in Theoretical Computer Science Conference*. ITCS ’17. 2017, pp. 1–31.
- [AHIV17] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup”. In: *Proceedings of the 24th ACM Conference on Computer and Communications Security*. CCS ’17. 2017, pp. 2087–2104.
- [Ajt96] Miklós Ajtai. “Generating hard instances of lattice problems”. In: *Proceedings of the 28th Annual ACM Symposium on the Theory of Computing*. STOC ’96. 1996, pp. 99–108.
- [BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. “Fast Reed–Solomon Interactive Oracle Proofs of Proximity”. In: *Proceedings of the 45th International Colloquium on Automata, Languages and Programming*. ICALP ’18. 2018, 14:1–14:17.
- [BC23] Benedikt Bünz and Binyi Chen. “Protostar: Generic Efficient Accumulation/Folding for Special-Sound Protocols”. In: *Proceedings of the 29th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’23. 2023, pp. 77–110.
- [BC24] Dan Boneh and Binyi Chen. *LatticeFold: A Lattice-based Folding Scheme and its Applications to Succinct Proof Systems*. Cryptology ePrint Archive, Paper 2024/257. 2024.
- [BC25] Dan Boneh and Binyi Chen. *Faster, Simpler, Shorter Lattice-Based Folding for Succinct Proof Systems*. Cryptology ePrint Archive, Paper 2025/247. 2025.
- [BCCT13] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. “Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data”. In: *Proceedings of the 45th ACM Symposium on the Theory of Computing*. STOC ’13. 2013, pp. 111–120.
- [BCFRRZ25] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. “Blaze: Fast SNARKs from Interleaved RAA Codes”. In: *Proceedings of the 44th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’25. 2025.
- [BCG20] Jonathan Bootle, Alessandro Chiesa, and Jens Groth. “Linear-Time Arguments with Sublinear Verification from Tensor Codes”. In: *Proceedings of the 18th Theory of Cryptography Conference*. TCC ’20. 2020, pp. 19–46.
- [BCG24] Elette Boyle, Ran Cohen, and Aarushi Goel. “Breaking the $O(\sqrt{n})$ -Bit Barrier: Byzantine Agreement with Polylog Bits Per Party”. In: *Journal of Cryptology* 37.1 (2024), p. 2.

- [BCGGHJ17] Jonathan Bootle, Andrea Cerulli, Essam Ghadafi, Jens Groth, Mohammad Hajiabadi, and Sune K. Jakobsen. “Linear-Time Zero-Knowledge Proofs for Arithmetic Circuit Satisfiability”. In: *Proceedings of the 23rd International Conference on the Theory and Applications of Cryptology and Information Security*. ASIACRYPT ’17. 2017, pp. 336–365.
- [BCGGRS19] Eli Ben-Sasson, Alessandro Chiesa, Lior Goldberg, Tom Gur, Michael Riabzev, and Nicholas Spooner. “Linear-Size Constant-Query IOPs for Delegating Computation”. In: *Proceedings of the 17th Theory of Cryptography Conference*. TCC ’19. 2019, pp. 494–521.
- [BCGTV13] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. “SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge”. In: *Proceedings of the 33rd Annual International Cryptology Conference*. CRYPTO ’13. 2013, pp. 90–108.
- [BCIKS20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. “Proximity Gaps for Reed–Solomon Codes”. In: *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science*. FOCS ’20. 2020, pp. 900–909.
- [BCL22] Jonathan Bootle, Alessandro Chiesa, and Siqi Liu. “Zero-Knowledge IOPs with Linear-Time Prover and Polylogarithmic-Time Verifier”. In: *Proceedings of the 41st Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’22. 2022, pp. 275–304.
- [BCLMS21] Benedikt Bünz, Alessandro Chiesa, William Lin, Pratyush Mishra, and Nicholas Spooner. “Proof-Carrying Data Without Succinct Arguments”. In: *Proceedings of the 41st Annual International Cryptology Conference*. CRYPTO ’21. 2021, pp. 681–710.
- [BCMS20] Benedikt Bünz, Alessandro Chiesa, Pratyush Mishra, and Nicholas Spooner. “Recursive Proof Composition from Accumulation Schemes”. In: *Proceedings of the 18th Theory of Cryptography Conference*. TCC ’20. 2020, pp. 1–18.
- [BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. “Interactive Oracle Proofs”. In: *Proceedings of the 14th Theory of Cryptography Conference*. TCC ’16-B. 2016, pp. 31–60.
- [BCTV14] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. “Scalable Zero Knowledge via Cycles of Elliptic Curves”. In: *Proceedings of the 34th Annual International Cryptology Conference*. CRYPTO ’14. 2014, pp. 276–294.
- [BGH19] Sean Bowe, Jack Grigg, and Daira Hopwood. *Halo: Recursive Proof Composition without a Trusted Setup*. Cryptology ePrint Archive, Report 2019/1021. 2019.
- [BGKS20] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. “DEEP-FRI: Sampling Outside the Box Improves Soundness”. In: *Proceedings of the 11th Innovations in Theoretical Computer Science Conference*. ITCS ’20. 2020, 5:1–5:32.
- [BMMS25] Anubhav Baweja, Pratyush Mishra, Tushar Mopuri, and Matan Shtepel. *FICS and FACS: Fast IOPs and Accumulation via Code-Switching*. Cryptology ePrint Archive, Report 2025/737. 2025.
- [BMNW25a] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. “Accumulation Without Homomorphism”. In: *Proceedings of the 16th Innovations in Theoretical Computer Science Conference*. ITCS ’25. 2025, 23:1–23:25.
- [BMNW25b] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. “Arc: Accumulation for Reed–Solomon Codes”. In: *Proceedings of the 45th Annual International Cryptology Conference*. CRYPTO ’25. 2025.
- [BMRS20] Joseph Bonneau, Izaak Meckler, Vanishree Rao, and Evan Shapiro. *Coda: Decentralized Cryptocurrency at Scale*. Cryptology ePrint Archive, Report 2020/352. 2020.

- [CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. “HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates”. In: *Proceedings of the 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’23. 2023, pp. 499–530.
- [CCDW20] Weikeng Chen, Alessandro Chiesa, Emma Dauterman, and Nicholas P. Ward. *Reducing Participation Costs via Incremental Verification for Ledger Systems*. Cryptology ePrint Archive, Report 2020/1522. 2020.
- [CCHLRR18] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, and Ron D. Rothblum. *Fiat–Shamir From Simpler Assumptions*. Cryptology ePrint Archive, Report 2018/1004. 2018.
- [CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. “Succinct Arguments in the Quantum Random Oracle Model”. In: *Proceedings of the 17th Theory of Cryptography Conference*. TCC ’19. 2019, pp. 1–29.
- [CO25] Alessandro Chiesa and Michele Orrù. *A Fiat–Shamir Transformation From Duplex Sponges*. Cryptology ePrint Archive, Report 2025/536. 2025.
- [CT10] Alessandro Chiesa and Eran Tromer. “Proof-Carrying Data and Hearsay Arguments from Signature Cards”. In: *Proceedings of the 1st Symposium on Innovations in Computer Science*. ICS ’10. 2010, pp. 310–331.
- [CTV13] Stephen Chong, Eran Tromer, and Jeffrey A. Vaughan. *Enforcing Language Semantics Using Proof-Carrying Data*. Cryptology ePrint Archive, Report 2013/513. 2013.
- [CTV15] Alessandro Chiesa, Eran Tromer, and Madars Virza. “Cluster Computing in Zero Knowledge”. In: *Proceedings of the 34th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’15. 2015, pp. 371–403.
- [CY24] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024. URL: <https://github.com/hash-based-snargs-book>.
- [DI14] Erez Druk and Yuval Ishai. “Linear-time encodable codes meeting the Gilbert–Varshamov bound and their cryptographic applications”. In: *Proceedings of the 5th Innovations in Theoretical Computer Science Conference*. ITCS ’14. 2014, pp. 169–182.
- [DMS24] Michel Dellepère, Pratyush Mishra, and Alireza Shirzad. *Garuda and Pari: Faster and Smaller SNARKs via Equi-efficient Polynomial Commitments*. Cryptology ePrint Archive, Paper 2024/1245. 2024.
- [DP24] Benjamin E. Diamond and Jim Posen. “Proximity Testing with Logarithmic Randomness”. In: *IACR Communications in Cryptology* 1.1 (2024).
- [DP25] Benjamin Diamond and Jim Posen. “Succinct Arguments over Towers of Binary Fields”. In: *Proceedings of the 44th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’25. 2025.
- [EG23] Liam Eagen and Ariel Gabizon. *ProtoGalaxy: Efficient ProtoStar-style folding of multiple instances*. Cryptology ePrint Archive, Report 2023/1106. 2023.
- [FKNP24] Giacomo Fenzi, Christian Knabenhans, Ngoc Khanh Nguyen, and Duc Tu Pham. “Lova: Lattice-Based Folding Scheme from Unstructured Lattices”. In: *Proceedings of the 30th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’24. 2024, pp. 303–326.
- [FNS75] Horst Feistel, William A. Notz, and J. Lynn Smith. “Some cryptographic techniques for machine-to-machine data communications”. In: *Proceedings of the IEEE* 63.11 (1975), pp. 1545–1554.

- [GGR11] Parikshit Gopalan, Venkatesan Guruswami, and Prasad Raghavendra. “List Decoding Tensor Products and Interleaved Codes”. In: *SIAM Journal on Computing* 40.5 (2011). Preliminary version appeared in STOC ’09., pp. 1432–1462.
- [GI05] Venkatesan Guruswami and Piotr Indyk. “Linear-time encodable/decodable codes with near-optimal rate”. In: *IEEE Transactions on Information Theory* 51.10 (2005). Preliminary version appeared in STOC ’03., pp. 3393–3400.
- [GKL24] Yiwen Gao, Haibin Kan, and Yuan Li. *Linear Proximity Gap for Linear Codes within the 1.5 Johnson Bound*. Cryptology ePrint Archive, Report 2024/1810. 2024.
- [GLSTW23] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. “Brakedown: Linear-time and field-agnostic SNARKs for R1CS”. In: *Proceedings of the 43rd Annual International Cryptology Conference*. CRYPTO ’23. 2023.
- [GRS12] Venkatesan Guruswami, Atri Rudra, and Madhu Sudan. *Essential coding theory*. Vol. 2. 1. 2012.
- [KB20] Assimakis Kattis and Joseph Bonneau. *Proof of Necessary Work: Succinct State Verification with Fairness Guarantees*. Cryptology ePrint Archive, Report 2020/190. 2020.
- [KS24] Abhiram Kothapalli and Srinath T. V. Setty. “HyperNova: Recursive Arguments for Customizable Constraint Systems”. In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO ’24. 2024, pp. 345–379.
- [KST22] Abhiram Kothapalli, Srinath T. V. Setty, and Ioanna Tzialla. “Nova: Recursive Zero-Knowledge Arguments from Folding Schemes”. In: *Proceedings of the 42nd Annual International Cryptology Conference*. CRYPTO ’22. 2022, pp. 359–388.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. “Constant-Size Commitments to Polynomials and Their Applications”. In: *Proceedings of the 16th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’10. 2010, pp. 177–194.
- [Min] Mina. <https://minaprotocol.com/>.
- [NDCTB24] Wilson D. Nguyen, Trisha Datta, Binyi Chen, Nirvan Tyagi, and Dan Boneh. “Mangrove: A Scalable Framework for Folding-Based SNARKs”. In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO ’24. 2024, pp. 308–344.
- [NS25] Wilson Nguyen and Srinath Setty. *Neo: Lattice-based folding scheme for CCS over small fields and pay-per-bit commitments*. Cryptology ePrint Archive, Paper 2025/294. 2025.
- [NST24] Vineet Nair, Ashish Sharma, and Bhargav Thankey. *BrakingBase - a linear prover, poly-logarithmic verifier, field agnostic polynomial commitment scheme*. Cryptology ePrint Archive, Report 2024/1825. 2024.
- [NT16] Assa Naveh and Eran Tromer. “PhotoProof: Cryptographic Image Authentication for Any Set of Permissible Transformations”. In: *Proceedings of the 2016 IEEE Symposium on Security and Privacy*. SP ’16. 2016, pp. 255–271.
- [Ped92] Torben P. Pedersen. “Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing”. In: *Proceedings of the 11th Annual International Cryptology Conference*. CRYPTO ’91. 1992, pp. 129–140.
- [Pip80] Nicholas Pippenger. “On the Evaluation of Powers and Monomials”. In: *SIAM Journal on Computing* 9.2 (1980), pp. 230–250.
- [RR22] Noga Ron-Zewi and Ron D. Rothblum. “Proving as Fast as Computing: Succinct Arguments with Constant Prover Overhead”. In: *Proceedings of the 54th ACM Symposium on the Theory of Computing*. STOC ’22. 2022, pp. 1353–1363.

- [RRR16] Omer Reingold, Ron Rothblum, and Guy Rothblum. “Constant-Round Interactive Proofs for Delegating Computation”. In: *Proceedings of the 48th ACM Symposium on the Theory of Computing*. STOC ’16. 2016, pp. 49–62.
- [STW23] Srinath Setty, Justin Thaler, and Riad Wahby. *Customizable constraint systems for succinct arguments*. Cryptology ePrint Archive, Report 2023/552. 2023.
- [Spi96] Daniel A. Spielman. “Linear-time encodable and decodable error-correcting codes”. In: *IEEE Transactions on Information Theory* 42.6 (1996). Preliminary version appeared in STOC ’95., pp. 1723–1731.
- [VSBW13] Victor Vu, Srinath Setty, Andrew J. Blumberg, and Michael Walfish. “A hybrid architecture for interactive verifiable computation”. In: *Proceedings of the 34th IEEE Symposium on Security and Privacy*. Oakland ’13. 2013, pp. 223–237.
- [Val08] Paul Valiant. “Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency”. In: *Proceedings of the 5th Theory of Cryptography Conference*. TCC ’08. 2008, pp. 1–18.
- [XZS22] Tiancheng Xie, Yupeng Zhang, and Dawn Song. “Orion: Zero Knowledge Proof with Linear Prover Time”. In: *Proceedings of the 42nd Annual International Cryptology Conference*. CRYPTO ’22. 2022, pp. 299–328.
- [ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. “BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes”. In: *Proceedings of the 44th Annual International Cryptology Conference*. Vol. 14929. CRYPTO ’24. 2024, pp. 138–169.
- [ZYORJ24] Chenru Zhang, Xiao Yang, David Oswald, Mark Ryan, and Philipp Jovanovic. *Eva: Efficient IVC-Based Authentication of Lossy-Encoded Videos*. Cryptology ePrint Archive, Report 2024/1436. 2024.
- [Zei24] Hadas Zeilberger. *Khatam: Reducing the Communication Complexity of Code-Based SNARKs*. Cryptology ePrint Archive, Report 2024/1843. 2024.